GIÁO TRÌNH TỰ HỌC TIẾP NỐI TẬP 1

PYTHON TỪ TRONG CHĂN

TẬP 2

Hiểu bản chất — Thử bằng đôi tay — Tự xây lời giải

Cấu trúc dữ liệu, thuật toán, tệp, lỗi, hướng đối tượng, kiểm thử và ứng dụng thực tế

Nguyễn Huỳnh Đăng Nhựt

Bản thảo hoàn chỉnh — 2026

Lời nói đầu

Nếu Tập 1 là lúc chúng ta kéo chiếc chăn lên, đặt máy tính trước mặt và viết những dòng lệnh đầu tiên, thì Tập 2 bắt đầu ở một câu hỏi khó hơn: *làm thế nào để những dòng lệnh rời rạc trở thành một chương trình thực sự?*

Bạn đã biết biến lưu dữ liệu ở đâu, điều kiện giúp chương trình lựa chọn ra sao, vòng lặp thực hiện một công việc nhiều lần như thế nào và hàm giúp chia bài toán lớn thành những phần nhỏ. Những kiến thức ấy giống như bảng chữ cái. Ta có thể đọc từng chữ, nhưng để viết được một câu chuyện, ta còn phải biết tổ chức ý, giữ mạch, sửa lỗi và kết nối các phần với nhau.

Trong cuốn sách này, dữ liệu không còn chỉ là một con số hay một dòng chữ đứng riêng. Chúng ta sẽ học cách giữ cả danh sách điểm, một tập hợp từ khóa, hồ sơ của nhiều học sinh, nội dung của một tệp và các bản ghi trong cơ sở dữ liệu. Chương trình cũng không còn sống đúng một lần rồi biến mất khi cửa sổ được đóng. Nó sẽ biết đọc, biết ghi, biết phát hiện dữ liệu không hợp lệ, biết tự kiểm tra và biết giao tiếp với những chương trình khác.

Tập 2 vẫn giữ con đường quen thuộc của *Python từ trong chăn*:

Một tình huống gần gũi ⟶ Khái niệm ⟶ Ví dụ chạy được ⟶ Mổ xẻ từng bước ⟶ Sai rồi sửa ⟶ Tự làm một sản phẩm

Cuốn sách không được viết để bạn đọc thật nhanh. Một đoạn mã chỉ thực sự trở thành kiến thức của bạn khi bạn đã gõ lại, thay đổi dữ liệu, cố tình làm nó sai và tự giải thích được vì sao kết quả xuất hiện. Vì thế, ở cuối mỗi chương có ba tầng bài tập. Tầng thứ nhất giúp nhớ công cụ; tầng thứ hai buộc bạn chọn công cụ; tầng thứ ba yêu cầu bạn thiết kế một lời giải không có sẵn khuôn mẫu. Đừng bỏ qua tầng đầu vì thấy nó dễ, cũng đừng vội nhìn gợi ý khi tầng cuối khiến bạn im lặng vài phút.

Các ví dụ ưu tiên thư viện chuẩn và cú pháp hoạt động tốt trên những phiên bản Python 3 hiện đại. Khi một chủ đề cần gói ngoài, sách sẽ nói rõ lý do, cách tạo môi trường riêng và cách ghi lại phụ thuộc. Mục tiêu không phải là chạy theo mọi thư viện đang thịnh hành, mà là xây một nền móng đủ vững để khi công cụ đổi tên, bạn vẫn hiểu bài toán không hề đổi.

Bạn có thể học theo nhịp ba buổi cho mỗi chương. Buổi đầu đọc và chạy ví dụ. Buổi hai đóng sách, tự viết lại phần cốt lõi rồi đối chiếu. Buổi ba giải bài tập và ghi một nhật ký lỗi ngắn: lỗi gì đã xảy ra, bạn từng đoán nguyên nhân là gì, bằng chứng nào giúp tìm ra nguyên nhân thật. Sau vài tháng, cuốn nhật ký ấy thường có giá trị hơn một tập hợp mã nguồn hoàn hảo nhưng không còn nhớ mình đã viết bằng cách nào.

Cuối hành trình là hai dự án. Dự án thứ nhất là một ứng dụng dòng lệnh nhỏ nhưng có

cấu trúc tử tế. Dự án thứ hai là hệ thống quản lý việc học có tệp cấu hình, lưu dữ liệu, báo cáo, kiểm thử và khả năng mở rộng. Chúng không được chọn để gây ấn tượng bằng số lượng tính năng. Chúng được chọn vì đủ gần với đời sống để bạn thực sự dùng, đủ rộng để mọi chương trước đó có một chỗ đứng, và đủ chưa hoàn hảo để bạn tiếp tục phát triển sau khi gấp sách lại.

Tôi vẫn tin rằng lập trình không bắt đầu từ việc thuộc cú pháp. Nó bắt đầu ở khoảnh khắc một người dám nhìn vào điều mình chưa hiểu, chia nó thành những câu hỏi nhỏ hơn và kiên nhẫn kiểm chứng từng câu trả lời. Tập 2 được viết để đồng hành với khoảnh khắc ấy.

Nguyễn Huỳnh Đăng Nhựt
Tác giả

Cách sử dụng cuốn sách

Bốn nguyên tắc học

1. **Gõ lại thay vì chỉ sao chép.** Cử động nhỏ của việc gõ khiến bạn buộc phải nhìn dấu ngoặc, thụt lề và tên biến. Đó là lúc mắt bắt đầu học cách đọc mã.
2. **Dự đoán trước khi chạy.** Hãy viết ra giấy kết quả mà bạn nghĩ sẽ xuất hiện. Nếu dự đoán sai, khoảng cách giữa hai kết quả chính là nơi có kiến thức mới.
3. **Thay một thứ mỗi lần.** Khi thử nghiệm, chỉ đổi một dữ kiện, một điều kiện hoặc một cấu trúc. Nếu đổi năm thứ cùng lúc, bạn không còn biết điều gì tạo ra kết quả.
4. **Giải thích bằng lời của mình.** Sau mỗi chương, thử kể lại cho một người chưa học. Chỗ nào bạn phải dùng những từ mơ hồ như "nó tự chạy" thường là chỗ bạn chưa hiểu bản chất.

Quy ước trong sách

- Hộp **Ghi nhớ nhanh** giữ một nguyên tắc nên mang sang chương sau.
- Hộp **Cảnh báo lỗi thường gặp** mô tả lỗi, dấu hiệu và cách kiểm tra; không chỉ đưa một câu lệnh sửa nhanh.
- Hộp **Dừng lại để nghĩ** yêu cầu dự đoán hoặc so sánh trước khi đọc tiếp.
- Hộp **Phòng thực hành** là một thí nghiệm nhỏ có đầu vào, đầu ra và tiêu chí hoàn thành.
- Hộp **Xưởng dự án** nối kiến thức của chương với sản phẩm đang được xây dần.

Một phiên học 75 phút

Bạn có thể chia một phiên học thành 10 phút đọc bối cảnh, 20 phút chạy ví dụ, 15 phút thay đổi ví dụ, 25 phút làm bài và 5 phút ghi nhật ký lỗi. Nếu hôm đó chỉ có 25 phút, hãy chọn một ví dụ và làm trọn chu trình dự đoán–chạy–giải thích. Học ít nhưng khép kín tốt hơn đọc dở ba chương.

Dấu hiệu bạn đã học xong một chương

Bạn không cần nhớ mọi tên phương thức. Bạn cần nhận ra dạng bài toán, biết tìm đúng công cụ, giải thích được vì sao chọn công cụ đó và tự kiểm tra kết quả bằng một trường hợp nhỏ.

Mục lục

Lời nói đầu **iii**

Cách sử dụng cuốn sách **v**

I Học cách giữ nhiều dữ liệu **1**

1 Danh sách – Khi một chiếc hộp không còn đủ **3**
1.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 4
1.1.1 Bước 1: khoanh ranh giới của bài toán . . . 4
1.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 4
1.1.3 Bước 3: xác định bất biến . . . 5
1.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 5
1.3 Một ca làm mẫu đi đủ các bước . . . 6
1.3.1 Bước 1: phát biểu lại yêu cầu . . . 6
1.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 6
1.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 6
1.3.4 Bước 4: viết thuật toán bằng động từ . . . 7
1.3.5 Bước 5: chứng minh đủ dùng . . . 7
1.4 Kiểm thử là thí nghiệm của mô hình . . . 7
1.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 8
1.6 Vận dụng và mở rộng mô hình . . . 8
1.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 8
1.8 Tạo một danh sách và đọc từng phần tử . . . 9
1.9 Sửa đổi: danh sách là một vật có thể thay đổi . . . 9
1.10 Cắt lát – lấy một cửa sổ trên dãy . . . 10
1.11 Duyệt danh sách: giá trị, vị trí và cặp chỉ số–giá trị . . . 10
1.12 Tìm kiếm, đếm và sắp xếp . . . 11
1.13 Cạm bẫy sao chép và tên gọi . . . 11
1.14 Một bài toán hoàn chỉnh: theo dõi thời gian học . . . 12
1.15 Bài tập luyện tập . . . 12
1.16 Tóm tắt cuối chương . . . 13

2 Tuple và set – Khi thứ tự, thay đổi và trùng lặp trở thành lựa chọn **15**
2.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 15

2.1.1 Bước 1: khoanh ranh giới của bài toán . . . 16
2.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 16
2.1.3 Bước 3: xác định bất biến . . . 17
2.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 17
2.3 Một ca làm mẫu đi đủ các bước . . . 18
2.3.1 Bước 1: phát biểu lại yêu cầu . . . 18
2.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 18
2.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 18
2.3.4 Bước 4: viết thuật toán bằng động từ . . . 19
2.3.5 Bước 5: chứng minh đủ dùng . . . 19
2.4 Kiểm thử là thí nghiệm của mô hình . . . 19
2.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 19
2.6 Vận dụng và mở rộng mô hình . . . 20
2.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 20
2.8 Tuple – một cam kết nhỏ trong mã nguồn . . . 20
2.9 Bất biến không có nghĩa mọi thứ bên trong đều bất biến . . . 21
2.10 Set – quan tâm ai có mặt, không quan tâm đứng thứ mấy . . . 22
2.11 Các phép toán tập hợp . . . 22
2.12 Vì sao set tìm kiếm nhanh? . . . 23
2.13 Chọn list, tuple hay set . . . 23
2.14 Bài toán hoàn chỉnh: so sánh hai lần điểm danh . . . 23
2.15 Bài tập luyện tập . . . 24
2.16 Tóm tắt cuối chương . . . 24

3 Dictionary – Gọi dữ liệu bằng tên thay vì nhớ vị trí 27
3.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 27
3.1.1 Bước 1: khoanh ranh giới của bài toán . . . 28
3.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 28
3.1.3 Bước 3: xác định bất biến . . . 28
3.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 29
3.3 Một ca làm mẫu đi đủ các bước . . . 29
3.3.1 Bước 1: phát biểu lại yêu cầu . . . 30
3.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 30
3.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 30
3.3.4 Bước 4: viết thuật toán bằng động từ . . . 30
3.3.5 Bước 5: chứng minh đủ dùng . . . 31
3.4 Kiểm thử là thí nghiệm của mô hình . . . 31
3.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 31
3.6 Vận dụng và mở rộng mô hình . . . 32
3.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 32
3.8 Cặp khóa–giá trị . . . 32
3.9 Ngoặc vuông hay get? . . . 33
3.10 Duyệt dictionary . . . 33
3.11 Dictionary như bảng tần suất . . . 33

3.12 Dictionary như bảng tra cứu . . . 34
3.13 Khóa nào được phép dùng? . . . 34
3.14 Các phương thức hữu ích . . . 35
3.15 Chuẩn hóa và kiểm tra dữ liệu . . . 35
3.16 Bài tập luyện tập . . . 35
3.17 Tóm tắt cuối chương . . . 36

4 Chuỗi nâng cao – Dữ liệu văn bản không chỉ là những chữ đứng cạnh nhau 39
4.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 40
4.1.1 Bước 1: khoanh ranh giới của bài toán . . . 40
4.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 40
4.1.3 Bước 3: xác định bất biến . . . 41
4.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 41
4.3 Một ca làm mẫu đi đủ các bước . . . 42
4.3.1 Bước 1: phát biểu lại yêu cầu . . . 42
4.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 42
4.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 42
4.3.4 Bước 4: viết thuật toán bằng động từ . . . 43
4.3.5 Bước 5: chứng minh đủ dùng . . . 43
4.4 Kiểm thử là thí nghiệm của mô hình . . . 43
4.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 44
4.6 Vận dụng và mở rộng mô hình . . . 44
4.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 44
4.8 Chuỗi là dãy bất biến . . . 45
4.9 Kiểm tra trước khi chuyển đổi . . . 45
4.10 Tách, phân vùng và ghép . . . 45
4.11 Tìm, thay và đếm có kiểm soát . . . 46
4.12 F-string như một công cụ trình bày . . . 46
4.13 Unicode và encoding . . . 46
4.14 Biểu thức chính quy: một chiếc kính chuyên dụng . . . 47
4.15 Bài toán hoàn chỉnh: chuẩn hóa nhật ký học . . . 47
4.16 Bài tập luyện tập . . . 48
4.17 Tóm tắt cuối chương . . . 48

5 Cấu trúc lồng nhau và comprehension – Dữ liệu bắt đầu có hình dạng 51
5.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 51
5.1.1 Bước 1: khoanh ranh giới của bài toán . . . 52
5.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 52
5.1.3 Bước 3: xác định bất biến . . . 53
5.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 53
5.3 Một ca làm mẫu đi đủ các bước . . . 54
5.3.1 Bước 1: phát biểu lại yêu cầu . . . 54
5.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 54
5.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 54

5.3.4 Bước 4: viết thuật toán bằng động từ . . . 55
5.3.5 Bước 5: chứng minh đủ dùng . . . 55
5.4 Kiểm thử là thí nghiệm của mô hình . . . 55
5.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 55
5.6 Vận dụng và mở rộng mô hình . . . 56
5.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 56
5.8 Danh sách các dictionary . . . 56
5.9 Ma trận và lỗi nhân bản hàng . . . 57
5.10 Bản sao nông và bản sao sâu . . . 57
5.11 List comprehension – một phép biến đổi có thể đọc thành câu . . . 58
5.12 Biểu thức điều kiện nằm ở đâu? . . . 58
5.13 Set và dictionary comprehension . . . 58
5.14 Comprehension lồng nhau . . . 59
5.15 Một đường ống xử lý dữ liệu nhỏ . . . 59
5.16 Bài tập luyện tập . . . 60
5.17 Tóm tắt cuối chương . . . 60

II Làm cho chương trình sống lâu hơn một lần chạy 61

6 Tệp và đường dẫn – Cho chương trình một trí nhớ 63
6.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 63
6.1.1 Bước 1: khoanh ranh giới của bài toán . . . 64
6.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 64
6.1.3 Bước 3: xác định bất biến . . . 64
6.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 65
6.3 Một ca làm mẫu đi đủ các bước . . . 65
6.3.1 Bước 1: phát biểu lại yêu cầu . . . 66
6.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 66
6.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 66
6.3.4 Bước 4: viết thuật toán bằng động từ . . . 66
6.3.5 Bước 5: chứng minh đủ dùng . . . 67
6.4 Kiểm thử là thí nghiệm của mô hình . . . 67
6.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 67
6.6 Vận dụng và mở rộng mô hình . . . 68
6.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 68
6.8 Đường dẫn không phải chính tệp . . . 68
6.9 Đọc tệp với with . . . 69
6.10 Ghi mới, ghi đè và nối tiếp . . . 69
6.11 Thao tác thư mục bằng pathlib . . . 70
6.12 Ghi an toàn bằng tệp tạm . . . 70
6.13 Văn bản và nhị phân . . . 71
6.14 Bài tập luyện tập . . . 71
6.15 Tóm tắt cuối chương . . . 71

7 JSON và CSV – Khi tệp cần một cấu trúc chung **73**
7.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 73
7.1.1 Bước 1: khoanh ranh giới của bài toán . . . 74
7.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 74
7.1.3 Bước 3: xác định bất biến . . . 74
7.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 75
7.3 Một ca làm mẫu đi đủ các bước . . . 75
7.3.1 Bước 1: phát biểu lại yêu cầu . . . 76
7.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 76
7.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 76
7.3.4 Bước 4: viết thuật toán bằng động từ . . . 76
7.3.5 Bước 5: chứng minh đủ dùng . . . 77
7.4 Kiểm thử là thí nghiệm của mô hình . . . 77
7.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 77
7.6 Vận dụng và mở rộng mô hình . . . 78
7.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 78
7.8 CSV không đơn giản chỉ là split bằng dấu phẩy . . . 78
7.9 JSON như cây dữ liệu . . . 79
7.10 Những kiểu JSON không biết . . . 79
7.11 Schema – hợp đồng nằm ngoài cú pháp . . . 80
7.12 Phiên bản hóa dữ liệu . . . 80
7.13 Kho lưu JSON nhỏ nhưng tử tế . . . 81
7.14 CSV hay JSON? . . . 82
7.15 Bài tập luyện tập . . . 82
7.16 Tóm tắt cuối chương . . . 82

8 Lỗi và ngoại lệ – Không giấu điều sai, hãy cho nó một đường đi **85**
8.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 85
8.1.1 Bước 1: khoanh ranh giới của bài toán . . . 86
8.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 86
8.1.3 Bước 3: xác định bất biến . . . 86
8.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 87
8.3 Một ca làm mẫu đi đủ các bước . . . 87
8.3.1 Bước 1: phát biểu lại yêu cầu . . . 88
8.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 88
8.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 88
8.3.4 Bước 4: viết thuật toán bằng động từ . . . 88
8.3.5 Bước 5: chứng minh đủ dùng . . . 89
8.4 Kiểm thử là thí nghiệm của mô hình . . . 89
8.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 89
8.6 Vận dụng và mở rộng mô hình . . . 90
8.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 90
8.8 Ba nhóm vấn đề . . . 90
8.9 Cấu trúc try đầy đủ . . . 90

8.10 Bắt đúng thứ mình xử lý được . . . 91
8.11 raise và hợp đồng hàm . . . 91
8.12 Giữ nguyên nhân khi dịch lỗi . . . 92
8.13 Ngoại lệ riêng có dữ liệu . . . 92
8.14 LBYL và EAFP . . . 92
8.15 Vòng nhập có khả năng phục hồi . . . 93
8.16 Bài tập luyện tập . . . 93
8.17 Tóm tắt cuối chương . . . 94

9 Module, package và môi trường – Chia một chương trình thành những căn phòng 95
9.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 95
9.1.1 Bước 1: khoanh ranh giới của bài toán . . . 96
9.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 96
9.1.3 Bước 3: xác định bất biến . . . 97
9.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 97
9.3 Một ca làm mẫu đi đủ các bước . . . 98
9.3.1 Bước 1: phát biểu lại yêu cầu . . . 98
9.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 98
9.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 98
9.3.4 Bước 4: viết thuật toán bằng động từ . . . 99
9.3.5 Bước 5: chứng minh đủ dùng . . . 99
9.4 Kiểm thử là thí nghiệm của mô hình . . . 99
9.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 99
9.6 Vận dụng và mở rộng mô hình . . . 100
9.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 100
9.8 Một module là một tệp Python . . . 100
9.9 Mã ở mức module được chạy khi import . . . 101
9.10 Python tìm module ở đâu? . . . 101
9.11 Tổ chức một package . . . 102
9.12 Môi trường ảo . . . 102
9.13 Ghi phụ thuộc có ý nghĩa . . . 103
9.14 Tách lõi khỏi giao diện . . . 103
9.15 Bài tập luyện tập . . . 104
9.16 Tóm tắt cuối chương . . . 104

10 Gỡ lỗi, logging và kiểm thử – Biến niềm tin thành bằng chứng 105
10.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 105
10.1.1 Bước 1: khoanh ranh giới của bài toán . . . 106
10.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 106
10.1.3 Bước 3: xác định bất biến . . . 107
10.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 107
10.3 Một ca làm mẫu đi đủ các bước . . . 108
10.3.1 Bước 1: phát biểu lại yêu cầu . . . 108

10.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 108
10.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 108
10.3.4 Bước 4: viết thuật toán bằng động từ . . . 109
10.3.5 Bước 5: chứng minh đủ dùng . . . 109
10.4 Kiểm thử là thí nghiệm của mô hình . . . 109
10.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 109
10.6 Vận dụng và mở rộng mô hình . . . 110
10.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 110
10.8 Tái hiện trước khi sửa . . . 110
10.9 Breakpoint và bước chạy . . . 111
10.10 Logging thay cho print trong ứng dụng . . . 111
10.11 Một test nhỏ . . . 112
10.12 Kiểm tra ngoại lệ và biên . . . 112
10.13 Test double và tiêm phụ thuộc . . . 113
10.14 Regression test – giữ lại bài học của lỗi cũ . . . 113
10.15 Bài tập luyện tập . . . 113
10.16 Tóm tắt cuối chương . . . 114

III Tư duy thuật toán và cấu trúc dữ liệu 115

11 Độ phức tạp – Khi chương trình đúng vẫn có thể chưa đủ tốt 117
11.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 117
11.1.1 Bước 1: khoanh ranh giới của bài toán . . . 118
11.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 118
11.1.3 Bước 3: xác định bất biến . . . 118
11.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 119
11.3 Một ca làm mẫu đi đủ các bước . . . 119
11.3.1 Bước 1: phát biểu lại yêu cầu . . . 120
11.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 120
11.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 120
11.3.4 Bước 4: viết thuật toán bằng động từ . . . 120
11.3.5 Bước 5: chứng minh đủ dùng . . . 121
11.4 Kiểm thử là thí nghiệm của mô hình . . . 121
11.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 121
11.6 Vận dụng và mở rộng mô hình . . . 122
11.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 122
11.8 Ta đang đếm điều gì? . . . 122
11.9 Bỏ hằng số và bậc thấp . . . 123
11.10 Vòng lặp nối tiếp và lồng nhau . . . 123
11.11 Chi phí ẩn trong một dòng . . . 123
11.12 Không gian và dữ liệu trung gian . . . 124
11.13 Phân tích chia đôi . . . 124
11.14 Đo bằng timeit . . . 124

11.15 Tối ưu đúng chỗ . . . 124
11.16 Bài tập luyện tập . . . 125
11.17 Tóm tắt cuối chương . . . 125

12 Tìm kiếm và sắp xếp – Đặt dữ liệu vào một trật tự có ích 127
12.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 127
12.1.1 Bước 1: khoanh ranh giới của bài toán . . . 128
12.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 128
12.1.3 Bước 3: xác định bất biến . . . 128
12.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 129
12.3 Một ca làm mẫu đi đủ các bước . . . 129
12.3.1 Bước 1: phát biểu lại yêu cầu . . . 130
12.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 130
12.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 130
12.3.4 Bước 4: viết thuật toán bằng động từ . . . 130
12.3.5 Bước 5: chứng minh đủ dùng . . . 131
12.4 Kiểm thử là thí nghiệm của mô hình . . . 131
12.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 131
12.6 Vận dụng và mở rộng mô hình . . . 132
12.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 132
12.8 Tìm tuyến tính . . . 132
12.9 Tìm nhị phân và bất biến khoảng . . . 132
12.10 Selection sort – chọn nhỏ nhất từng lượt . . . 133
12.11 Insertion sort – chèn vào đoạn đã có trật tự . . . 133
12.12 Merge sort – chia, giải, ghép . . . 134
12.13 Sắp xếp thực tế bằng key . . . 134
12.14 Có cần sắp xếp không? . . . 135
12.15 Bài tập luyện tập . . . 135
12.16 Tóm tắt cuối chương . . . 136

13 Stack, queue và heap – Thứ tự phục vụ cũng là một thuật toán 137
13.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 137
13.1.1 Bước 1: khoanh ranh giới của bài toán . . . 138
13.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 138
13.1.3 Bước 3: xác định bất biến . . . 138
13.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 139
13.3 Một ca làm mẫu đi đủ các bước . . . 139
13.3.1 Bước 1: phát biểu lại yêu cầu . . . 140
13.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 140
13.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 140
13.3.4 Bước 4: viết thuật toán bằng động từ . . . 140
13.3.5 Bước 5: chứng minh đủ dùng . . . 141
13.4 Kiểm thử là thí nghiệm của mô hình . . . 141
13.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 141

13.6 Vận dụng và mở rộng mô hình . . . 142
13.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 142
13.8 Stack – vào sau, ra trước . . . 142
13.9 Kiểm tra ngoặc cân bằng . . . 143
13.10 Queue – vào trước, ra trước . . . 143
13.11 Mô phỏng quầy phục vụ . . . 143
13.12 Priority queue bằng heap . . . 144
13.13 Hai stack cho undo/redo . . . 144
13.14 Chọn cấu trúc theo quy tắc lấy ra . . . 145
13.15 Bài tập luyện tập . . . 145
13.16 Tóm tắt cuối chương . . . 146

14 Đệ quy và quay lui – Giải bài toán bằng phiên bản nhỏ hơn của chính nó 147
14.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 147
14.1.1 Bước 1: khoanh ranh giới của bài toán . . . 148
14.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 148
14.1.3 Bước 3: xác định bất biến . . . 148
14.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 149
14.3 Một ca làm mẫu đi đủ các bước . . . 149
14.3.1 Bước 1: phát biểu lại yêu cầu . . . 150
14.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 150
14.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 150
14.3.4 Bước 4: viết thuật toán bằng động từ . . . 150
14.3.5 Bước 5: chứng minh đủ dùng . . . 151
14.4 Kiểm thử là thí nghiệm của mô hình . . . 151
14.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 151
14.6 Vận dụng và mở rộng mô hình . . . 152
14.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 152
14.8 Hai điều kiện để đệ quy kết thúc . . . 152
14.9 Call stack và giới hạn . . . 153
14.10 Fibonacci và bài toán con lặp lại . . . 153
14.11 Quay lui: chọn, đi tiếp, hoàn tác . . . 153
14.12 Hoán vị và tập lựa chọn . . . 154
14.13 Cắt tỉa trên bài toán tổng . . . 154
14.14 Đệ quy trên cấu trúc lồng nhau . . . 155
14.15 Bài tập luyện tập . . . 155
14.16 Tóm tắt cuối chương . . . 156

15 Cây và đồ thị – Khi dữ liệu trở thành những mối quan hệ 157
15.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 157
15.1.1 Bước 1: khoanh ranh giới của bài toán . . . 158
15.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 158
15.1.3 Bước 3: xác định bất biến . . . 158
15.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 159

15.3 Một ca làm mẫu đi đủ các bước . . . 159
15.3.1 Bước 1: phát biểu lại yêu cầu . . . 160
15.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 160
15.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 160
15.3.4 Bước 4: viết thuật toán bằng động từ . . . 160
15.3.5 Bước 5: chứng minh đủ dùng . . . 161
15.4 Kiểm thử là thí nghiệm của mô hình . . . 161
15.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 161
15.6 Vận dụng và mở rộng mô hình . . . 162
15.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 162
15.8 Cây: mỗi nút nhìn xuống các cây con . . . 162
15.9 Biểu diễn đồ thị bằng danh sách kề . . . 163
15.10 DFS bằng stack . . . 163
15.11 BFS bằng queue và đường ngắn nhất không trọng số . . . 163
15.12 Thành phần liên thông . . . 164
15.13 Đồ thị có trọng số và Dijkstra . . . 165
15.14 Phụ thuộc và sắp xếp topo . . . 165
15.15 Bài tập luyện tập . . . 166
15.16 Tóm tắt cuối chương . . . 167

IV Nhìn chương trình như một thế giới đối tượng 169

16 Lớp và đối tượng – Gom trạng thái cùng hành vi 171
16.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 171
16.1.1 Bước 1: khoanh ranh giới của bài toán . . . 172
16.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 172
16.1.3 Bước 3: xác định bất biến . . . 172
16.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 173
16.3 Một ca làm mẫu đi đủ các bước . . . 173
16.3.1 Bước 1: phát biểu lại yêu cầu . . . 174
16.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 174
16.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 174
16.3.4 Bước 4: viết thuật toán bằng động từ . . . 174
16.3.5 Bước 5: chứng minh đủ dùng . . . 175
16.4 Kiểm thử là thí nghiệm của mô hình . . . 175
16.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 175
16.6 Vận dụng và mở rộng mô hình . . . 176
16.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 176
16.8 Lớp là bản mô tả, đối tượng là một trường hợp cụ thể . . . 176
16.9 Trạng thái hợp lệ ngay từ khi sinh ra . . . 177
16.10 Phương thức instance, class và static . . . 177
16.11 Thuộc tính lớp và cạm bẫy dữ liệu mutable . . . 178
16.12 Đối tượng hợp tác với nhau . . . 178

16.13 Danh tính và bằng nhau . . . 179
16.14 Khi nào không cần lớp? . . . 179
16.15 Bài tập luyện tập . . . 179
16.16 Tóm tắt cuối chương . . . 180

17 Đóng gói, property và bất biến – Bảo vệ trạng thái khỏi những thay đổi vô tình 181
17.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 181
17.1.1 Bước 1: khoanh ranh giới của bài toán . . . 182
17.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 182
17.1.3 Bước 3: xác định bất biến . . . 183
17.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 183
17.3 Một ca làm mẫu đi đủ các bước . . . 184
17.3.1 Bước 1: phát biểu lại yêu cầu . . . 184
17.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 184
17.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 184
17.3.4 Bước 4: viết thuật toán bằng động từ . . . 185
17.3.5 Bước 5: chứng minh đủ dùng . . . 185
17.4 Kiểm thử là thí nghiệm của mô hình . . . 185
17.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 185
17.6 Vận dụng và mở rộng mô hình . . . 186
17.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 186
17.8 Dấu gạch dưới là một lời nhắc . . . 186
17.9 Property giữ giao diện đơn giản . . . 187
17.10 Bất biến phải đúng sau mọi phương thức công khai . . . 188
17.11 Không để lộ danh sách nội bộ . . . 188
17.12 Đối tượng bất biến . . . 188
17.13 Máy trạng thái nhỏ . . . 189
17.14 Getter/setter không phải mục tiêu . . . 189
17.15 Bài tập luyện tập . . . 190
17.16 Tóm tắt cuối chương . . . 190

18 Kế thừa, hợp thành và đa hình – Tái sử dụng mà không trói chặt 191
18.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 191
18.1.1 Bước 1: khoanh ranh giới của bài toán . . . 192
18.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 192
18.1.3 Bước 3: xác định bất biến . . . 192
18.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 193
18.3 Một ca làm mẫu đi đủ các bước . . . 193
18.3.1 Bước 1: phát biểu lại yêu cầu . . . 194
18.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 194
18.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 194
18.3.4 Bước 4: viết thuật toán bằng động từ . . . 194
18.3.5 Bước 5: chứng minh đủ dùng . . . 195

18.4 Kiểm thử là thí nghiệm của mô hình . . . 195
18.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 195
18.6 Vận dụng và mở rộng mô hình . . . 196
18.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 196
18.8 Một ví dụ kế thừa hợp lý . . . 196
18.9 Ghi đè và super . . . 197
18.10 Nguyên tắc thay thế . . . 197
18.11 Hợp thành: có một thay vì là một . . . 197
18.12 Duck typing và Protocol . . . 198
18.13 Strategy thay cho if dài . . . 198
18.14 Tránh kiểm tra kiểu lan rộng . . . 199
18.15 Bài tập luyện tập . . . 199
18.16 Tóm tắt cuối chương . . . 199

19 Dataclass và các phương thức đặc biệt – Làm kiểu dữ liệu hòa vào Python 201
19.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 201
19.1.1 Bước 1: khoanh ranh giới của bài toán . . . 202
19.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 202
19.1.3 Bước 3: xác định bất biến . . . 202
19.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 203
19.3 Một ca làm mẫu đi đủ các bước . . . 203
19.3.1 Bước 1: phát biểu lại yêu cầu . . . 204
19.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 204
19.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 204
19.3.4 Bước 4: viết thuật toán bằng động từ . . . 204
19.3.5 Bước 5: chứng minh đủ dùng . . . 205
19.4 Kiểm thử là thí nghiệm của mô hình . . . 205
19.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 205
19.6 Vận dụng và mở rộng mô hình . . . 206
19.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 206
19.8 Dataclass đầu tiên . . . 206
19.9 repr và str . . . 207
19.10 Frozen và hash . . . 207
19.11 slots . . . 207
19.12 Làm một collection giống Python . . . 208
19.13 Toán tử chỉ khi ngữ nghĩa tự nhiên . . . 208
19.14 Serialization không tự nhiên hoàn toàn . . . 209
19.15 Bài tập luyện tập . . . 209
19.16 Tóm tắt cuối chương . . . 210

V Viết Python theo cách của Python 211

20 Iterator và generator – Xử lý dữ liệu từng phần 213
20.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 213

20.1.1 Bước 1: khoanh ranh giới của bài toán . . . 214
20.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 214
20.1.3 Bước 3: xác định bất biến . . . 214
20.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 215
20.3 Một ca làm mẫu đi đủ các bước . . . 215
20.3.1 Bước 1: phát biểu lại yêu cầu . . . 216
20.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 216
20.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 216
20.3.4 Bước 4: viết thuật toán bằng động từ . . . 216
20.3.5 Bước 5: chứng minh đủ dùng . . . 217
20.4 Kiểm thử là thí nghiệm của mô hình . . . 217
20.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 217
20.6 Vận dụng và mở rộng mô hình . . . 218
20.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 218
20.8 Giao thức lặp . . . 218
20.9 Generator function . . . 219
20.10 Đọc bản ghi từng dòng . . . 219
20.11 Pipeline lười . . . 219
20.12 yield from . . . 220
20.13 Iterator lớp tùy chỉnh . . . 220
20.14 itertools . . . 220
20.15 Dãy vô hạn và hàng rào . . . 221
20.16 Bài tập luyện tập . . . 221
20.17 Tóm tắt cuối chương . . . 222

21 Hàm bậc cao, closure và decorator – Truyền cả cách xử lý 223
21.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 223
21.1.1 Bước 1: khoanh ranh giới của bài toán . . . 224
21.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 224
21.1.3 Bước 3: xác định bất biến . . . 224
21.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 225
21.3 Một ca làm mẫu đi đủ các bước . . . 225
21.3.1 Bước 1: phát biểu lại yêu cầu . . . 225
21.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 226
21.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 226
21.3.4 Bước 4: viết thuật toán bằng động từ . . . 226
21.3.5 Bước 5: chứng minh đủ dùng . . . 226
21.4 Kiểm thử là thí nghiệm của mô hình . . . 227
21.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 227
21.6 Vận dụng và mở rộng mô hình . . . 227
21.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 228
21.8 Hàm là giá trị . . . 228
21.9 Hàm nhận hàm . . . 228
21.10 Closure . . . 229

21.11 Cạm bẫy late binding . . . 229
21.12 Decorator đầu tiên . . . 229
21.13 Decorator có tham số . . . 230
21.14 Thứ tự decorator . . . 230
21.15 Khi context manager rõ hơn . . . 231
21.16 Bài tập luyện tập . . . 231
21.17 Tóm tắt cuối chương . . . 232

22 Context manager và type hints – Quản lý vòng đời, ghi rõ hợp đồng 233
22.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 233
22.1.1 Bước 1: khoanh ranh giới của bài toán . . . 234
22.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 234
22.1.3 Bước 3: xác định bất biến . . . 234
22.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 235
22.3 Một ca làm mẫu đi đủ các bước . . . 235
22.3.1 Bước 1: phát biểu lại yêu cầu . . . 236
22.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 236
22.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 236
22.3.4 Bước 4: viết thuật toán bằng động từ . . . 236
22.3.5 Bước 5: chứng minh đủ dùng . . . 237
22.4 Kiểm thử là thí nghiệm của mô hình . . . 237
22.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 237
22.6 Vận dụng và mở rộng mô hình . . . 238
22.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 238
22.8 Giao thức context manager . . . 238
22.9 contextmanager bằng generator . . . 239
22.10 Type hints cơ bản . . . 239
22.11 Callable, TypeAlias và Literal . . . 240
22.12 TypedDict cho dữ liệu dạng dictionary . . . 240
22.13 Generic đơn giản . . . 241
22.14 Type checker, validation và test . . . 241
22.15 Bài tập luyện tập . . . 241
22.16 Tóm tắt cuối chương . . . 242

23 Bất đồng bộ và đồng thời – Làm việc khác trong lúc chờ 243
23.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 243
23.1.1 Bước 1: khoanh ranh giới của bài toán . . . 244
23.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 244
23.1.3 Bước 3: xác định bất biến . . . 245
23.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 245
23.3 Một ca làm mẫu đi đủ các bước . . . 246
23.3.1 Bước 1: phát biểu lại yêu cầu . . . 246
23.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 246
23.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 246

23.3.4 Bước 4: viết thuật toán bằng động từ . . . 247
23.3.5 Bước 5: chứng minh đủ dùng . . . 247
23.4 Kiểm thử là thí nghiệm của mô hình . . . 247
23.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 247
23.6 Vận dụng và mở rộng mô hình . . . 248
23.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 248
23.8 Chờ I/O và tính CPU . . . 248
23.9 Coroutine đầu tiên . . . 249
23.10 Chạy chồng thời gian . . . 249
23.11 Giới hạn đồng thời . . . 250
23.12 Timeout, hủy và dọn dẹp . . . 250
23.13 Race condition vẫn có trong một luồng . . . 250
23.14 Đưa hàm blocking sang thread . . . 251
23.15 Bài tập luyện tập . . . 251
23.16 Tóm tắt cuối chương . . . 251

VI Mang kiến thức ra khỏi trang giấy 253

24 Tự động hóa – Để máy tính nhận những việc lặp lại 255
24.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 255
24.1.1 Bước 1: khoanh ranh giới của bài toán . . . 256
24.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 256
24.1.3 Bước 3: xác định bất biến . . . 256
24.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 257
24.3 Một ca làm mẫu đi đủ các bước . . . 257
24.3.1 Bước 1: phát biểu lại yêu cầu . . . 258
24.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 258
24.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 258
24.3.4 Bước 4: viết thuật toán bằng động từ . . . 258
24.3.5 Bước 5: chứng minh đủ dùng . . . 259
24.4 Kiểm thử là thí nghiệm của mô hình . . . 259
24.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 259
24.6 Vận dụng và mở rộng mô hình . . . 260
24.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 260
24.8 Một quy trình trước khi viết code . . . 260
24.9 Lập kế hoạch trước khi thay đổi . . . 260
24.10 Đổi tên hai pha . . . 261
24.11 Idempotency . . . 262
24.12 argparse cho CLI . . . 262
24.13 Tự động hóa bảng dữ liệu . . . 262
24.14 Bảo mật trong script . . . 263
24.15 Bài tập luyện tập . . . 263
24.16 Tóm tắt cuối chương . . . 264

25 API và SQLite – Giao tiếp với dịch vụ, lưu dữ liệu có truy vấn 265
25.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 265
25.1.1 Bước 1: khoanh ranh giới của bài toán . . . 266
25.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 266
25.1.3 Bước 3: xác định bất biến . . . 266
25.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 267
25.3 Một ca làm mẫu đi đủ các bước . . . 267
25.3.1 Bước 1: phát biểu lại yêu cầu . . . 268
25.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 268
25.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 268
25.3.4 Bước 4: viết thuật toán bằng động từ . . . 268
25.3.5 Bước 5: chứng minh đủ dùng . . . 269
25.4 Kiểm thử là thí nghiệm của mô hình . . . 269
25.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 269
25.6 Vận dụng và mở rộng mô hình . . . 270
25.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 270
25.8 Một yêu cầu HTTP gồm gì? . . . 270
25.9 Không đặt bí mật trong mã . . . 271
25.10 Phân trang, rate limit và retry . . . 271
25.11 SQLite trong thư viện chuẩn . . . 271
25.12 Truy vấn có tham số . . . 272
25.13 CRUD và ánh xạ . . . 272
25.14 Transaction . . . 273
25.15 Migration schema . . . 273
25.16 Bài tập luyện tập . . . 274
25.17 Tóm tắt cuối chương . . . 274

26 Dự án 1: Trình quản lý công việc dòng lệnh 275
26.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 275
26.1.1 Bước 1: khoanh ranh giới của bài toán . . . 276
26.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 276
26.1.3 Bước 3: xác định bất biến . . . 276
26.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 277
26.3 Một ca làm mẫu đi đủ các bước . . . 277
26.3.1 Bước 1: phát biểu lại yêu cầu . . . 278
26.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 278
26.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 278
26.3.4 Bước 4: viết thuật toán bằng động từ . . . 278
26.3.5 Bước 5: chứng minh đủ dùng . . . 279
26.4 Kiểm thử là thí nghiệm của mô hình . . . 279
26.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 279
26.6 Vận dụng và mở rộng mô hình . . . 280
26.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 280
26.8 Yêu cầu và điều không làm . . . 280

26.9 Cấu trúc . . . 281
26.10 Domain model . . . 281
26.11 Repository contract và memory fake . . . 282
26.12 Service use cases . . . 282
26.13 JSON repository . . . 283
26.14 CLI là adapter . . . 284
26.15 Test theo tầng . . . 284
26.16 Các mốc triển khai . . . 285
26.17 Bài tập mở rộng . . . 285
26.18 Tóm tắt cuối chương . . . 285

27 Dự án 2: Hệ thống quản lý việc học cá nhân 287
27.1 Từ một tình huống thật đến mô hình có thể kiểm tra . . . 287
27.1.1 Bước 1: khoanh ranh giới của bài toán . . . 288
27.1.2 Bước 2: viết quan hệ trước khi viết cú pháp . . . 288
27.1.3 Bước 3: xác định bất biến . . . 289
27.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi . . . 289
27.3 Một ca làm mẫu đi đủ các bước . . . 290
27.3.1 Bước 1: phát biểu lại yêu cầu . . . 290
27.3.2 Bước 2: dựng ví dụ nhỏ bằng tay . . . 290
27.3.3 Bước 3: chọn biểu diễn dữ liệu . . . 290
27.3.4 Bước 4: viết thuật toán bằng động từ . . . 291
27.3.5 Bước 5: chứng minh đủ dùng . . . 291
27.4 Kiểm thử là thí nghiệm của mô hình . . . 291
27.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm . . . 291
27.6 Vận dụng và mở rộng mô hình . . . 292
27.7 Bài tập ba tầng: từ hiểu đến tự thiết kế . . . 292
27.8 Khám phá vấn đề trước tính năng . . . 292
27.9 Mô hình miền . . . 293
27.10 Schema SQLite . . . 293
27.11 Use case ghi phiên học nguyên tử . . . 294
27.12 Báo cáo tuần bằng truy vấn và hàm thuần . . . 294
27.13 Gợi ý kế hoạch vừa sức . . . 295
27.14 Đồ thị prerequisites . . . 295
27.15 Kiến trúc và ranh giới . . . 296
27.16 Kế hoạch test . . . 296
27.17 Backup, export và quyền riêng tư . . . 296
27.18 Lộ trình tám mốc . . . 296
27.19 Bài tập phát triển sau sách . . . 297
27.20 Tóm tắt cuối chương . . . 297

Lời kết 299

A Gợi ý và lời giải chọn lọc 301

B Bảng tra cứu nhanh **317**

C Từ điển thuật ngữ **321**

D Tài liệu tham khảo và đường học tiếp **325**

Phần I

Học cách giữ nhiều dữ liệu

Chương 1

Danh sách – Khi một chiếc hộp không còn đủ

Giới thiệu ngắn

Trong Tập 1, một biến thường giữ một giá trị: tên của một người, nhiệt độ của một ngày hoặc điểm của một môn. Nhưng hãy thử quản lý điểm của bốn mươi học sinh. Nếu tạo bốn mươi biến mang tên `diem1`, `diem2`, …, ta không chỉ phải viết dài mà còn tự đẩy mình vào một căn phòng đầy hộp nhưng không có kệ. Danh sách (*list*) chính là chiếc kệ đầu tiên của Python: nhiều giá trị được đặt theo thứ tự và cùng được gọi bằng một tên.

Mục tiêu học tập

- Tạo, đọc, sửa và xóa phần tử trong danh sách.
- Hiểu chỉ số, chỉ số âm và phép cắt lát.
- Phân biệt thay đổi tại chỗ với tạo danh sách mới.
- Duyệt danh sách bằng giá trị hoặc bằng vị trí.
- Chọn đúng phương thức khi thêm, tìm, sắp xếp và sao chép dữ liệu.

Hành trang trước khi học

Bạn đã biết biến, vòng lặp `for`, câu lệnh điều kiện và hàm. Hãy nhớ rằng danh sách không thay thế các kiến thức ấy; nó cho phép chúng làm việc với nhiều dữ liệu cùng lúc.

1.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một lớp có bốn mươi điểm kiểm tra. Giáo viên cần giữ thứ tự nhập, sửa một điểm sai, tính trung bình và tìm nhóm cần hỗ trợ. Bốn mươi biến rời rạc không thể hiện được rằng đây là một dãy cùng ý nghĩa. Danh sách gom dữ liệu thành một đối tượng, nhưng đổi lại người viết phải quản lý chỉ số, độ dài, tính thay đổi và quan hệ giữa nhiều danh sách.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **danh sách như một dãy hữu hạn có thứ tự và có trạng thái**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

1.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: độ dài n, chỉ số i, giá trị x_i, thứ tự, điều kiện hợp lệ và thao tác thêm–xóa–sửa.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

1.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$L = [x_0, x_1, \ldots, x_{n-1}], \qquad \bar{x} = \frac{1}{n}\sum_{i=0}^{n-1} x_i$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể

thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: dãy phần tử hiện tại, vị trí đang duyệt và phần dữ liệu đã xử lý?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

1.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Mọi chỉ số truy cập phải thuộc đoạn từ 0 đến $n-1$; các phần tử phải cùng vai trò dữ liệu và thứ tự chỉ thay đổi khi yêu cầu cho phép.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

1.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **danh sách như một dãy hữu hạn có thứ tự và có trạng thái** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng

có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

1.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Quản lý điểm kiểm tra theo đúng thứ tự danh sách lớp. Ta truy vết thao tác sửa điểm thứ 12, lọc các điểm dưới 5, tính trung bình và đối chiếu tổng số phần tử trước–sau để chắc rằng việc lọc không vô tình làm mất dữ liệu gốc.

1.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: “Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?” Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

1.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

1.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay

đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

1.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

1.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

1.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định “kết quả ấy có vẻ được”, ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là “thuật toán giữ bất biến và tạo hậu điều kiện”. Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

1.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Dùng hai danh sách song song rồi để chúng lệch độ dài.
- Sửa danh sách trong lúc duyệt làm bỏ qua phần tử.
- Nhầm sao chép tham chiếu với tạo bản sao độc lập.
- Đưa nhiều loại ý nghĩa vào cùng một danh sách chỉ vì Python cho phép.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

1.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

1.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Truy vết mọi chỉ số của một danh sách năm phần tử, gồm chỉ số âm.
2. Thiết kế bộ tính trung bình bỏ qua dữ liệu thiếu nhưng vẫn báo số mẫu thực.
3. So sánh sửa tại chỗ với tạo danh sách mới về trạng thái trước–sau.
4. Xây dựng mô hình hàng điểm và phát hiện hai danh sách song song bị lệch.
5. Thiết kế năm ca thử cho thao tác xóa theo giá trị và theo vị trí.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ “gõ cho chạy” thành luyện tư duy thiết kế.

1.8 Tạo một danh sách và đọc từng phần tử

Danh sách được viết bằng cặp ngoặc vuông. Các phần tử ngăn cách bởi dấu phẩy và có thể thuộc nhiều kiểu dữ liệu, dù trong phần lớn bài toán thực tế ta nên giữ chúng cùng một ý nghĩa.

Một tuần tự học

```
mon_hoc = ["Giai tich", "Python", "Tieng Anh"]
so_phut = [45, 60, 30]

print(mon_hoc[0])      # Giai tich
print(mon_hoc[-1])     # Tieng Anh
print(len(mon_hoc))    # 3
```

Python đánh số vị trí từ 0. Phần tử đầu tiên có chỉ số 0, phần tử thứ hai có chỉ số 1. Cách đánh số này thoạt đầu thiếu tự nhiên, nhưng có thể hiểu chỉ số là *khoảng dịch chuyển* tính từ điểm bắt đầu: phần tử đầu cách đầu danh sách 0 bước. Chỉ số âm đi ngược từ cuối; `-1` là phần tử cuối, `-2` là phần tử áp chót.

Nếu truy cập một vị trí không tồn tại, Python phát sinh `IndexError`. Đây không phải lời phàn nàn vô nghĩa. Nó nói rằng chương trình đã đưa ra một giả định sai về độ dài dữ liệu.

Chỉ số cuối không phải là độ dài

Danh sách có n phần tử thì chỉ số hợp lệ chạy từ 0 đến $n-1$. Vì thế, phần tử cuối có thể viết là `ds[len(ds) - 1]` hoặc gọn hơn là `ds[-1]`, chứ không phải `ds[len(ds)]`.

1.9 Sửa đổi: danh sách là một vật có thể thay đổi

List là kiểu *mutable*: sau khi tạo, ta có thể thay một phần tử mà không cần tạo lại toàn bộ danh sách.

```
cong_viec = ["doc sach", "tap code", "di bo"]
cong_viec[1] = "lam bai Python"
cong_viec.append("ghi nhat ky")
cong_viec.insert(1, "on bai cu")

da_xong = cong_viec.pop(0)
print(da_xong)
print(cong_viec)
```

`append(x)` đặt một phần tử ở cuối. `insert(i, x)` chèn tại vị trí i. `pop(i)` vừa xóa vừa trả về phần tử ở vị trí i; nếu không truyền chỉ số, nó lấy phần tử cuối. `remove(x)` xóa lần

xuất hiện đầu tiên của giá trị x, còn `del ds[i]` xóa theo vị trí.

Sự khác nhau giữa xóa theo giá trị và xóa theo vị trí rất quan trọng. Khi người dùng bấm "xóa công việc có mã 17", ta nên tìm theo mã. Khi thuật toán hàng đợi lấy phần tử đầu, vị trí lại là điều cần quan tâm.

Hai câu lệnh trông giống nhưng không giống

Điều gì xảy ra với `ds.append([3, 4])` và `ds.extend([3, 4])` nếu ban đầu `ds = [1, 2]`? Hãy dự đoán độ dài của hai kết quả trước khi chạy. `append` thêm *một phần tử*; `extend` lần lượt nối các phần tử từ một đối tượng có thể lặp.

1.10 Cắt lát – lấy một cửa sổ trên dãy

Cú pháp `ds[start:stop:step]` tạo một danh sách mới từ vị trí `start` đến trước `stop`. Việc không lấy `stop` làm các đoạn liền nhau ghép lại mà không trùng: `ds[:3]` và `ds[3:]`. Bỏ `start` nghĩa là bắt đầu từ đầu; bỏ `stop` nghĩa là đi đến hết.

```
diem = [7.5, 8.0, 6.5, 9.0, 8.5, 7.0]

ba_bai_dau = diem[:3]
ba_bai_cuoi = diem[-3:]
cach_mot_bai = diem[::2]
dao_nguoc = diem[::-1]
```

Cắt lát thường không báo lỗi khi biên vượt độ dài; Python lấy phần còn tồn tại. Điều này tiện khi chia dữ liệu thành lô, nhưng cũng có thể che một giả định sai. Vì thế, trong mã quan trọng, hãy kiểm tra độ dài khi số phần tử là một điều kiện bắt buộc.

1.11 Duyệt danh sách: giá trị, vị trí và cặp chỉ số–giá trị

Nếu chỉ cần từng giá trị, hãy duyệt trực tiếp:

```
for mon in mon_hoc:
    print(mon)
```

Nếu cần cả vị trí, `enumerate()` diễn đạt ý định rõ hơn so với `range(len(...))`:

```
for stt, mon in enumerate(mon_hoc, start=1):
    print(f"{stt}. {mon}")
```

Khi hai danh sách có quan hệ theo vị trí, `zip()` ghép các phần tử tương ứng:

```
for mon, phut in zip(mon_hoc, so_phut):
    print(f"{mon}: {phut} phut")
```

`zip()` dừng ở danh sách ngắn nhất. Nếu việc thiếu dữ liệu là lỗi, hãy kiểm tra hai độ dài trước hoặc sử dụng chế độ nghiêm ngặt trên phiên bản Python hỗ trợ: `zip(a, b, strict=True)`.

1.12 Tìm kiếm, đếm và sắp xếp

Toán tử `in` trả lời câu hỏi có/không. Phương thức `index(x)` trả về vị trí đầu tiên nhưng báo lỗi nếu không tìm thấy. `count(x)` đếm số lần xuất hiện.

```
if "Python" in mon_hoc:
    vi_tri = mon_hoc.index("Python")
    print(f"Python nam o vi tri {vi_tri}")

print([8, 7, 8, 9].count(8))
```

Hai cách sắp xếp thường bị nhầm:

- `ds.sort()` thay đổi chính danh sách và trả về `None`.
- `sorted(ds)` tạo danh sách mới, giữ nguyên dữ liệu ban đầu.

```
hoc_sinh = [
    ("An", 8.5),
    ("Binh", 7.0),
    ("Chi", 9.2),
]

theo_diem = sorted(hoc_sinh, key=lambda ban: ban[1], reverse=True)
print(theo_diem)
```

Tham số `key` không bảo Python "so sánh thế nào" mà nói "hãy lấy đại diện nào của mỗi phần tử để so sánh". Trong ví dụ, mỗi học sinh được đại diện bởi điểm ở vị trí 1.

1.13 Cạm bẫy sao chép và tên gọi

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)  # [1, 2, 3, 4]
```

`b = a` không tạo danh sách thứ hai. Nó đặt thêm một nhãn lên cùng một đối tượng. Muốn sao chép lớp ngoài, dùng `a.copy()` hoặc `a[:]`. Với danh sách lồng nhau, bản sao nông vẫn chia sẻ các danh sách bên trong; chương 5 sẽ quay lại vấn đề này.

Tên biến không phải chiếc hộp vật lý

Một cách hình dung chính xác hơn là tên biến giống nhãn dán trỏ đến một đối tượng. Nhiều nhãn có thể trỏ cùng một danh sách, nên thay đổi qua nhãn này sẽ được nhìn thấy từ nhãn kia.

1.14 Một bài toán hoàn chỉnh: theo dõi thời gian học

Tổng hợp một tuần

```
def thong_ke(phut_moi_ngay):
    if not phut_moi_ngay:
        return {"tong": 0, "trung_binh": 0, "ngay_tot_nhat": None}

    tong = sum(phut_moi_ngay)
    vi_tri_max = phut_moi_ngay.index(max(phut_moi_ngay))
    return {
        "tong": tong,
        "trung_binh": tong / len(phut_moi_ngay),
        "ngay_tot_nhat": vi_tri_max + 1,
    }

du_lieu = [45, 60, 0, 90, 35, 120, 50]
ket_qua = thong_ke(du_lieu)
print(ket_qua)
```

Điều kiện `if not phut_moi_ngay` bảo vệ phép chia khỏi danh sách rỗng. Hàm không in ngay mà trả dữ liệu để phần khác quyết định cách hiển thị. Đây là bước chuyển từ "viết cho chạy" sang "viết để dùng lại".

1.15 Bài tập luyện tập

Tầng 1 – làm quen công cụ

1. Tạo danh sách bảy nhiệt độ; in nhiệt độ đầu, cuối, lớn nhất và trung bình.
2. Viết hàm nhận danh sách số và trả về danh sách chỉ gồm số dương.
3. Không dùng `reverse()`, hãy tạo bản đảo ngược bằng cắt lát.
4. Cho danh sách có phần tử trùng; đếm số lần một giá trị do người dùng nhập xuất hiện.

Tầng 2 – chọn cách làm

5. Chuẩn hóa danh sách tên bằng cách bỏ khoảng trắng hai đầu và viết hoa chữ cái đầu.
6. Ghép hai danh sách tên môn và điểm; in bảng xếp theo điểm giảm dần nhưng không làm thay đổi dữ liệu gốc.
7. Viết hàm `chia_lo(ds, kich_thuoc)` chia danh sách thành các lô nhỏ. Xử lý trường hợp kích thước không hợp lệ.
8. Xóa mọi số âm khỏi danh sách. So sánh cách tạo danh sách mới với cách xóa ngay trong lúc duyệt.

Tầng 3 – thiết kế lời giải

9. Xây dựng chương trình quản lý hàng đợi ở phòng tư vấn: thêm người, gọi người đầu, xem danh sách và thoát.
10. Một chuỗi số đo có thể chứa giá trị đột biến. Hãy loại giá trị nhỏ hơn tứ phân vị dưới hoặc lớn hơn tứ phân vị trên theo một ngưỡng tự chọn; giải thích lựa chọn.

Tự kiểm tra

Không chạy chương trình, hãy cho biết kết quả và giải thích đối tượng nào đang được chia sẻ:

```
x = [1, 2]
y = x
z = x.copy()
y += [3]
z.append(4)
print(x, y, z)
```

1.16 Tóm tắt cuối chương

Bản đồ kiến thức

- List lưu một dãy có thứ tự và có thể thay đổi.
- Chỉ số đọc một phần tử; cắt lát tạo một dãy con mới.
- `append`, `extend`, `insert`, `pop`, `remove` mang các ý nghĩa khác nhau.
- Duyệt trực tiếp khi cần giá trị; dùng `enumerate` khi cần vị trí; dùng `zip` khi ghép dãy.
- Phép gán không đồng nghĩa với sao chép đối tượng.

Góc nhìn cá nhân

Khi mới học, tôi từng nghĩ biết nhiều phương thức là dấu hiệu tiến bộ. Sau đó tôi nhận ra câu hỏi quan trọng hơn là: dữ liệu này đại diện cho điều gì, thứ tự có quan trọng không, có được phép trùng không và ai có quyền thay đổi nó? Trả lời bốn câu hỏi ấy thường dẫn ta đến cấu trúc đúng trước cả khi nhớ tên phương thức.

Chương 2

Tuple và set – Khi thứ tự, thay đổi và trùng lặp trở thành lựa chọn

Giới thiệu ngắn

Nếu list làm được rất nhiều việc, tại sao Python còn cần tuple và set? Câu trả lời nằm ở ý nghĩa chứ không chỉ ở khả năng. Tọa độ của một điểm không nên bị thêm một phần tử tùy tiện. Danh sách mã sinh viên đã check-in không cần giữ cùng một mã năm lần. Chọn cấu trúc dữ liệu là cách người lập trình viết một phần của quy tắc bài toán ngay vào chương trình.

Mục tiêu học tập

- Hiểu tuple là dãy có thứ tự nhưng không thay đổi được.
- Đóng gói và tháo gói nhiều giá trị an toàn.
- Dùng set để loại trùng và thực hiện phép toán tập hợp.
- Nhận biết khi nào thứ tự, tính duy nhất hoặc khả năng thay đổi là yêu cầu chính.
- Tránh các cạm bẫy về tuple một phần tử, thứ tự set và phần tử không băm được.

2.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Tọa độ một điểm (x, y) có hai thành phần với vai trò cố định; danh sách mã học sinh đã điểm danh lại cần loại mọi bản ghi trùng. Hai bài toán đều chứa nhiều giá trị nhưng mang hai lời hứa khác nhau. Tuple nói rằng cấu trúc không đổi; set nói rằng mỗi giá trị chỉ xuất hiện một lần và thứ tự không phải hợp đồng.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu

bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **tuple và set như hai tuyên bố về tính cố định và tính duy nhất**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

2.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: vị trí thành phần, khả năng thay đổi, tính duy nhất, quan hệ tập hợp và thứ tự có hay không có ý nghĩa.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

2.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$P = (x, y), \qquad A \cup B, \quad A \cap B, \quad A \setminus B$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: các phần tử thuộc tập, các phần tử đã xử lý và cấu trúc tuple không đổi?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

2.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Tuple giữ số lượng/vai trò thành phần; set không chứa hai phần tử bằng nhau và mọi phép tập hợp phải giữ đúng định nghĩa Toán học.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

2.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **tuple và set như hai tuyên bố về tính cố định và tính duy nhất** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

2.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

So sánh danh sách đăng ký với danh sách tham dự. Ta chuyển mã sinh viên sang set, tính giao để biết người có mặt, hiệu để biết người vắng và hợp để kiểm tra toàn bộ mã liên quan; tên và điểm chi tiết vẫn được giữ ở cấu trúc khác.

2.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

2.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

2.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

2.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

2.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

2.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

2.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Dùng set khi cần giữ thứ tự hiển thị.
- Cho phần tử có thể thay đổi vào set rồi ngạc nhiên vì không băm được.
- Nhầm tuple không đổi với mọi đối tượng bên trong tuple đều bất biến.

- Dùng set để loại trùng nhưng không giải thích quy tắc xác định hai bản ghi giống nhau.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

2.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

2.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Mô hình hóa điểm và vector 2D bằng tuple, chỉ rõ vai trò từng vị trí.
2. Giải bài toán hai câu lạc bộ bằng giao, hợp và hiệu tập hợp.
3. Đo sự khác nhau giữa kiểm tra thuộc list và set trên dữ liệu lớn.
4. Thiết kế quy tắc chuẩn hóa email trước khi loại trùng.
5. Tìm một phản ví dụ cho việc dựa vào thứ tự lặp của set.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

2.8 Tuple – một cam kết nhỏ trong mã nguồn

Tuple thường được viết bằng dấu ngoặc tròn, nhưng chính dấu phẩy mới tạo ra tuple.

```
toa_do = (10.5, 21.0)
mau_rgb = (245, 180, 30)
mot_phan_tu = (42,)
khong_phai_tuple = (42)
```

Ta vẫn đọc tuple bằng chỉ số, cắt lát và duyệt như list. Điểm khác biệt là không thể gán lại một vị trí: `toa_do[0] = 3` gây `TypeError`. Tính bất biến không khiến dữ liệu "an toàn tuyệt đối", nhưng ngăn một lớp thay đổi vô tình và truyền tải ý rằng nhóm giá trị này là một bản ghi cố định.

Một hàm trả về nhiều kết quả

```
def min_max(ds):
    if not ds:
        raise ValueError("Danh sach khong duoc rong")
    return min(ds), max(ds)

thap_nhat, cao_nhat = min_max([7, 2, 9, 4])
print(thap_nhat, cao_nhat)
```

Hàm thực chất trả về một tuple. Phép gán bên trái tháo gói hai phần tử vào hai biến. Số biến phải phù hợp số phần tử, trừ khi dùng dấu sao để nhận phần còn lại.

```
dau, *giua, cuoi = [1, 2, 3, 4, 5]
print(dau)   # 1
print(giua)  # [2, 3, 4]
print(cuoi)  # 5
```

Tháo gói làm mã dễ đọc khi mỗi vị trí có một vai trò rõ ràng. Tuy nhiên, nếu một bản ghi có quá nhiều vị trí, ta dễ quên `ban[4]` là email hay số điện thoại. Khi đó, dictionary hoặc lớp sẽ diễn đạt tốt hơn.

2.9 Bất biến không có nghĩa mọi thứ bên trong đều bất biến

```
ho_so = ("An", [8.0, 9.0])
ho_so[1].append(10.0)
print(ho_so)
```

Tuple không cho thay tham chiếu ở vị trí 1 bằng danh sách khác, nhưng danh sách đang nằm tại đó vẫn có thể tự thay đổi. Đây là khác biệt giữa "không đổi cấu trúc tuple" và "đóng băng sâu mọi đối tượng". Khi cần khóa dữ liệu thật sự, thiết kế phải kiểm soát cả các thành phần bên trong.

2.10 Set – quan tâm ai có mặt, không quan tâm đứng thứ mấy

Set là tập hợp các phần tử duy nhất. Tạo set bằng ngoặc nhọn hoặc `set()`. Riêng tập rỗng phải viết `set()` vì `{}` là dictionary rỗng.

```
mon_da_hoc = {"Python", "Toan", "Python", "Vat ly"}
print(mon_da_hoc)  # Python chi xuat hien mot lan

mon_da_hoc.add("Tieng Anh")
mon_da_hoc.discard("Hoa")
```

Không dựa vào thứ tự khi in set. Cách Python sắp xếp nội bộ có thể thay đổi giữa lần chạy. Nếu đầu ra cần ổn định, dùng `sorted(tap_hop)` trước khi hiển thị.

remove và discard

`remove(x)` báo `KeyError` nếu x không tồn tại. `discard(x)` im lặng. Chọn theo ý nghĩa: nếu thiếu phần tử chứng tỏ trạng thái chương trình bị sai, lỗi của `remove` là tín hiệu hữu ích; nếu thao tác "xóa nếu có", `discard` tự nhiên hơn.

2.11 Các phép toán tập hợp

Giả sử hai câu lạc bộ có danh sách thành viên:

```
ai_club = {"An", "Binh", "Chi", "Dung"}
robot_club = {"Chi", "Dung", "Ha"}

ca_hai = ai_club & robot_club
it_nhat_mot = ai_club | robot_club
chi_ai = ai_club - robot_club
chi_mot_clb = ai_club ^ robot_club
```

Giao `&` giữ người có trong cả hai. Hợp `|` giữ người có ở ít nhất một tập. Hiệu `-` trả phần chỉ thuộc tập bên trái. Hiệu đối xứng `^` giữ phần chỉ xuất hiện ở một trong hai. Những phép toán này không chỉ ngắn hơn vòng lặp; chúng nói đúng ngôn ngữ của bài toán.

Quan hệ tập con dùng `<=`, tập con thực sự dùng `<`. Ví dụ, kiểm tra người học đã hoàn thành đủ học phần bắt buộc:

```
bat_buoc = {"Python", "DSA", "Git"}
da_hoc = {"Python", "DSA", "Git", "SQL"}

if bat_buoc <= da_hoc:
    print("Du dieu kien lam du an")
```

2.12 Vì sao set tìm kiếm nhanh?

List thường phải lần lượt so sánh từ đầu cho đến khi gặp giá trị. Set dùng một mã băm để định vị vùng lưu trữ, nên phép kiểm tra thành viên trung bình thường gần như không phụ thuộc vào số phần tử. Đó là lý do đổi danh sách lớn thành set có thể làm bước tra cứu lặp lại nhanh hơn rõ rệt.

Đổi lại, phần tử của set phải *hashable*: giá trị băm phải ổn định trong suốt thời gian nằm trong tập. Số, chuỗi và tuple chứa phần tử bất biến thường dùng được; list không dùng được vì nó có thể thay đổi.

Không có cấu trúc "tốt nhất" tuyệt đối

Nếu cần giữ đúng thứ tự người xếp hàng, set là lựa chọn sai dù tra cứu nhanh. Nếu cần loại một triệu mã trùng, list lại làm ta phải viết thêm logic và có thể chậm. Hiệu năng chỉ có ý nghĩa sau khi ý nghĩa dữ liệu đã đúng.

2.13 Chọn list, tuple hay set

Đặc điểm	List	Tuple	Set
Có thứ tự	Có	Có	Không nên dựa vào
Cho phép trùng	Có	Có	Không
Thay đổi trực tiếp	Có	Không	Có
Truy cập bằng chỉ số	Có	Có	Không
Tình huống điển hình	Dãy công việc	Tọa độ, bản ghi cố định	Thành viên duy nhất, phép tập hợp

2.14 Bài toán hoàn chỉnh: so sánh hai lần điểm danh

```
def bao_cao_diem_danh(dang_ky, co_mat):
    ds_dang_ky = set(dang_ky)
    ds_co_mat = set(co_mat)
    return {
        "vang": sorted(ds_dang_ky - ds_co_mat),
        "khong_dang_ky": sorted(ds_co_mat - ds_dang_ky),
        "hop_le": sorted(ds_dang_ky & ds_co_mat),
    }

ket_qua = bao_cao_diem_danh(
    ["SV01", "SV02", "SV03", "SV03"],
    ["SV01", "SV03", "SV09"],
)
print(ket_qua)
```

Việc chuyển sang set đồng thời loại bản ghi trùng và làm rõ ba nhóm cần tìm. Sau đó ta sắp xếp chỉ ở biên hiển thị để kết quả ổn định.

2.15 Bài tập luyện tập

Tầng 1

1. Tạo tuple chứa ngày, tháng, năm rồi tháo gói vào ba biến.
2. Từ một câu, tạo set các từ khác nhau sau khi chuyển về chữ thường.
3. Cho hai set môn học, tìm giao, hợp và phần chỉ có ở set thứ nhất.
4. Viết hàm nhận dãy và trả tuple gồm tổng, trung bình, giá trị nhỏ nhất, lớn nhất.

Tầng 2

5. Tìm phần tử xuất hiện đúng một lần trong hai danh sách bằng phép tập hợp, rồi bàn về trường hợp mỗi danh sách tự có phần tử trùng.
6. Kiểm tra một câu có phải pangram tiếng Anh, tức chứa đủ 26 chữ cái hay không.
7. Một khóa học có tập điều kiện tiên quyết. Viết hàm trả về những môn người học còn thiếu.
8. Dùng tuple làm khóa dictionary để đếm số lần các tọa độ lưới được ghé qua.

Tầng 3

9. Thiết kế hệ thống gợi ý bạn học dựa trên số môn học chung. Mỗi người có một set sở thích; hãy xếp hạng ứng viên và giải thích tiêu chí khi hòa điểm.
10. Phát hiện tài khoản dùng chung: từ log các cặp `(tai_khoan, thiet_bi)`, tìm tài khoản xuất hiện trên hơn ba thiết bị và thiết bị phục vụ quá nhiều tài khoản.

2.16 Tóm tắt cuối chương

Bản đồ kiến thức

Tuple biểu đạt một dãy có cấu trúc ổn định; set biểu đạt sự có mặt duy nhất. Tháo gói giúp đặt tên cho vai trò của từng vị trí. Các phép giao, hợp, hiệu và hiệu đối xứng thường biến nhiều vòng lặp thành một biểu thức gần với ngôn ngữ bài toán. Phần tử của set phải băm được, và thứ tự của set không phải hợp đồng để chương trình dựa vào.

Góc nhìn cá nhân

Một cấu trúc dữ liệu tốt giống như cách đặt tên tốt: nó giảm số câu giải thích mà người đọc phải tự mang trong đầu. Khi viết `tap_ma_da_xu_ly`, tôi đang nói rằng trùng lặp không có ý nghĩa. Khi trả về tuple `(min, max)`, tôi đang nói rằng hai giá trị cùng tạo thành một kết quả. Chương trình bắt đầu rõ ràng từ những cam kết nhỏ như thế.

Chương 3

Dictionary – Gọi dữ liệu bằng tên thay vì nhớ vị trí

Giới thiệu ngắn

Một hồ sơ gồm họ tên, mã số, điểm và email. Nếu lưu bằng list, người đọc phải nhớ vị trí 0 là tên, vị trí 1 là mã. Chỉ cần ai đó chèn thêm ngày sinh vào giữa, mọi chỉ số phía sau thay đổi. Dictionary (từ điển) cho phép gắn mỗi giá trị với một khóa có ý nghĩa. Ta không hỏi "phần tử thứ ba là gì" mà hỏi "điểm của hồ sơ này là gì".

Mục tiêu học tập

- Tạo, đọc, cập nhật và xóa cặp khóa–giá trị.
- Phân biệt truy cập bằng ngoặc vuông và phương thức `get`.
- Duyệt khóa, giá trị và cặp dữ liệu.
- Mô hình hóa bản ghi và bảng tra cứu bằng dictionary.
- Hiểu khóa phải băm được và xử lý dictionary lồng nhau.

3.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Trong một trường học, mã sinh viên là cách gọi ổn định hơn vị trí trong danh sách. Khi một người nghỉ học, vị trí của những người sau thay đổi nhưng mã định danh không đổi. Dictionary mô hình hóa phép tra cứu từ khóa sang hồ sơ và buộc ta suy nghĩ về tính duy nhất của khóa, dữ liệu thiếu và cập nhật đồng thời nhiều trường.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng

nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **dictionary như ánh xạ từ khóa có nghĩa đến giá trị**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

3.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: miền khóa K, miền giá trị V, tính duy nhất, quy tắc khi thiếu khóa và cấu trúc của một hồ sơ.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

3.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$D : K \to V, \qquad D(k) = v$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: tập khóa hiện có, giá trị gắn với từng khóa và thay đổi đang thực hiện?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

3.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Mỗi khóa xuất hiện tối đa một lần và luôn đại diện cùng một thực thể; cập nhật phải giữ các trường bắt buộc của hồ sơ.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

3.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **dictionary như ánh xạ từ khóa có nghĩa đến giá trị** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

3.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Xây sổ điểm theo mã sinh viên. Ta phân biệt thêm hồ sơ mới với cập nhật hồ sơ cũ, kiểm tra mã chưa tồn tại khi đăng ký, xử lý tra cứu thiếu và lập thống kê mà không phụ thuộc thứ tự nhập.

3.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

3.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

3.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

3.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

3.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

3.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

3.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Dùng tên người làm khóa dù tên có thể trùng hoặc thay đổi.
- Truy cập bằng ngoặc vuông khi thiếu khóa là tình huống bình thường.
- Lồng dictionary sâu nhưng không định nghĩa schema.
- Sửa dictionary trong lúc lặp trên chính tập khóa của nó.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

3.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

3.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Thiết kế schema hồ sơ sinh viên và nêu khóa bắt buộc/tùy chọn.
2. Xây bảng tần suất từ một đoạn văn và kiểm chứng tổng tần suất.
3. So sánh `get`, `setdefault` và `defaultdict` theo hợp đồng.
4. Thiết kế ca thử khi thêm trùng mã sinh viên.
5. Chuyển hai danh sách song song thành một ánh xạ có ý nghĩa.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ “gõ cho chạy” thành luyện tư duy thiết kế.

3.8 Cặp khóa–giá trị

```
sinh_vien = {
    "ma": "SV001",
    "ho_ten": "Nguyen An",
    "diem": 8.7,
    "dang_hoc": True,
}

print(sinh_vien["ho_ten"])
sinh_vien["diem"] = 9.0
sinh_vien["email"] = "an@example.com"
```

Mỗi khóa là duy nhất. Gán vào khóa đã tồn tại cập nhật giá trị; gán vào khóa mới thêm một cặp. Từ Python 3.7 trở đi, thứ tự chèn được bảo toàn như một phần hành vi ngôn ngữ, nhưng ý nghĩa chính của dictionary vẫn là tra cứu bằng khóa chứ không phải truy cập theo

chỉ số.

3.9 Ngoặc vuông hay get?

`sinh_vien["lop"]` gây `KeyError` nếu khóa không tồn tại. `sinh_vien.get("lop")` trả `None`; có thể truyền giá trị mặc định: `get("lop", "Chưa xếp lớp")`.

Chọn cách nào phụ thuộc vào hợp đồng dữ liệu. Nếu mọi hồ sơ bắt buộc phải có mã, dùng ngoặc vuông để lỗi xuất hiện sớm. Nếu biệt danh là tùy chọn, `get` phản ánh điều đó tốt hơn.

Giá trị None và khóa không tồn tại

Nếu một khóa thực sự có giá trị `None`, kết quả của `get` không phân biệt được với khóa bị thiếu. Khi cần phân biệt, dùng `if khoa in tu_dien` hoặc một giá trị lính gác riêng.

```
KHONG_CO = object()
gia_tri = sinh_vien.get("so_dien_thoai", KHONG_CO)
if gia_tri is KHONG_CO:
    print("Khoa chua ton tai")
```

3.10 Duyệt dictionary

Mặc định, vòng lặp trên dictionary đi qua khóa. `values()` cho giá trị, `items()` cho từng cặp.

```
for khoa in sinh_vien:
    print(khoa)

for khoa, gia_tri in sinh_vien.items():
    print(f"{khoa}: {gia_tri}")
```

Không nên thêm hoặc xóa khóa trong lúc đang duyệt trực tiếp vì kích thước dictionary thay đổi. Nếu cần xóa theo điều kiện, tạo danh sách khóa cần xóa trước, hoặc dựng dictionary mới.

3.11 Dictionary như bảng tần suất

Một mẫu bài toán xuất hiện rất nhiều là đếm số lần mỗi giá trị xuất hiện.

Đếm từ

```
def dem_tu(cau):
    tan_suat = {}
    for tu in cau.lower().split():
        tu = tu.strip(".,!?;:")
        tan_suat[tu] = tan_suat.get(tu, 0) + 1
    return tan_suat

print(dem_tu("Hoc Python, hoc cach tu duy."))
```

Dòng cập nhật có thể đọc thành lời: lấy số đếm cũ của từ, nếu chưa có coi là 0, rồi cộng 1. Thư viện chuẩn còn có `collections.Counter`, nhưng tự viết một lần giúp ta hiểu cơ chế trước khi dùng công cụ gọn hơn.

3.12 Dictionary như bảng tra cứu

Khi thường xuyên tìm một bản ghi theo mã, danh sách buộc ta quét nhiều phần tử. Dictionary đặt mã làm khóa:

```
theo_ma = {
    "SV001": {"ten": "An", "diem": 8.7},
    "SV002": {"ten": "Binh", "diem": 7.9},
}

ma_can_tim = "SV002"
if ma_can_tim in theo_ma:
    print(theo_ma[ma_can_tim]["ten"])
```

Đây là dictionary lồng nhau. Cách tổ chức giúp tra cứu nhanh nhưng biểu thức nhiều ngoặc dễ dài. Hãy tách bước và đặt tên trung gian khi cần:

```
ho_so = theo_ma[ma_can_tim]
ten = ho_so["ten"]
print(ten)
```

3.13 Khóa nào được phép dùng?

Khóa phải hashable. Chuỗi, số và tuple bất biến là lựa chọn phổ biến. List và dictionary không thể làm khóa vì chúng thay đổi được. Một khóa tốt cần ổn định và đại diện duy nhất cho bản ghi. Dùng tên người làm khóa thường nguy hiểm vì có thể trùng hoặc thay đổi; mã sinh viên phù hợp hơn.

Khóa là quyết định mô hình hóa

Đừng chọn khóa chỉ vì dữ liệu đó đang có sẵn. Hãy hỏi: nó có duy nhất không, có ổn định không, có dễ chuẩn hóa không? Một khóa sai khiến dữ liệu bị ghi đè mà chương trình vẫn có vẻ chạy bình thường.

3.14 Các phương thức hữu ích

- `pop(k)` xóa khóa và trả giá trị; có thể truyền mặc định.
- `setdefault(k, d)` trả giá trị hiện có, hoặc thêm $k : d$ nếu thiếu.
- `update(other)` cập nhật nhiều cặp; khóa trùng bị ghi đè.
- Toán tử `|` trên Python hiện đại tạo dictionary hợp nhất, phía bên phải thắng khi trùng khóa.

```
nhom = {}
for ten, lop in [("An", "AI1"), ("Binh", "AI2"), ("Chi", "AI1")]:
    nhom.setdefault(lop, []).append(ten)
```

`setdefault` tiện cho việc nhóm, nhưng một biểu thức quá dày có thể khó đọc. `defaultdict(list)` là lựa chọn khác khi mẫu này lặp lại nhiều.

3.15 Chuẩn hóa và kiểm tra dữ liệu

Dictionary linh hoạt nên cũng dễ chứa hồ sơ thiếu hoặc sai kiểu. Ta cần một cổng kiểm tra trước khi dữ liệu đi sâu vào chương trình.

```
def kiem_tra_ho_so(ho_so):
    loi = []
    bat_buoc = {"ma", "ho_ten", "diem"}
    thieu = bat_buoc - ho_so.keys()
    if thieu:
        loi.append(f"Thieu khoa: {sorted(thieu)}")

    if "diem" in ho_so:
        diem = ho_so["diem"]
        if not isinstance(diem, (int, float)) or not 0 <= diem <= 10:
            loi.append("Diem phai la so tu 0 den 10")
    return loi
```

Ta không dừng ở lỗi đầu tiên mà gom các lỗi để người dùng sửa một lần. Tuy nhiên, với dữ liệu cực lớn hoặc thao tác nguy hiểm, dừng sớm có thể hợp lý hơn. Thiết kế xử lý lỗi luôn gắn với trải nghiệm và rủi ro thực tế.

3.16 Bài tập luyện tập

Tầng 1

1. Tạo dictionary biểu diễn một cuốn sách; thêm năm xuất bản, cập nhật số trang, xóa trường ghi chú.
2. Viết hàm nhận câu và trả ký tự xuất hiện nhiều nhất, bỏ qua khoảng trắng.
3. Đảo một dictionary có các giá trị duy nhất: giá trị cũ thành khóa mới.
4. Gộp hai bảng điểm; nếu trùng mã, giữ điểm lớn hơn.

Tầng 2

5. Nhóm danh sách từ theo chữ cái đầu.
6. Từ danh sách giao dịch `(loai, so_tien)`, tính tổng theo từng loại.
7. Xây chỉ mục ngược: mỗi từ khóa ánh xạ đến set các tài liệu có chứa nó.
8. So sánh hai hồ sơ và trả dictionary các trường đã thay đổi gồm giá trị cũ, mới.

Tầng 3

9. Xây danh bạ cho phép thêm, sửa, tìm theo số và tìm gần đúng theo tên. Tách giao diện khỏi hàm xử lý dữ liệu.
10. Thiết kế biểu diễn lịch học một tuần bằng dictionary lồng nhau. Viết chức năng phát hiện hai tiết bị trùng thời gian.

Mảnh ghép cho dự án cuối sách

Hãy thống nhất cấu trúc một công việc học tập gồm `id`, `title`, `course`, `due`, `priority`, `done`. Viết hàm kiểm tra một dictionary có đúng hợp đồng đó không. Sau này cùng cấu trúc sẽ được lưu JSON, đưa vào lớp và ghi xuống SQLite.

3.17 Tóm tắt cuối chương

Bản đồ kiến thức

Dictionary ánh xạ khóa đến giá trị. Ngoặc vuông phù hợp với khóa bắt buộc; `get` phù hợp với dữ liệu tùy chọn. `items` giúp duyệt cặp, bảng tần suất và bảng tra cứu là hai mẫu sử dụng quan trọng. Khóa cần duy nhất, ổn định và băm được. Dictionary linh hoạt nên phải đi cùng kiểm tra dữ liệu rõ ràng.

Góc nhìn cá nhân

Nhiều lỗi dữ liệu không đến từ thuật toán khó, mà từ việc chúng ta chưa đặt tên cho ý nghĩa. Khi thay chỉ số `[2]` bằng khóa `"diem"`, mã nguồn dài hơn vài ký tự nhưng bớt đi một giả định thầm lặng. Tôi luôn sẵn sàng đổi vài ký tự lấy một chương trình có thể tự giải thích.

Chương 4

Chuỗi nâng cao – Dữ liệu văn bản không chỉ là những chữ đứng cạnh nhau

Giới thiệu ngắn

Chuỗi đã xuất hiện từ chương đầu Tập 1, nhưng xử lý văn bản thực tế phức tạp hơn việc nối lời chào. Tên có khoảng trắng thừa, mã nhập chữ thường, tệp dùng ký tự xuống dòng, dữ liệu có dấu câu, và một biểu mẫu có thể chứa emoji hoặc chữ tiếng Việt. Chương này nhìn chuỗi như một dãy bất biến, đồng thời học cách biến văn bản lộn xộn thành dữ liệu có cấu trúc.

Mục tiêu học tập

- Hiểu tính bất biến, chỉ số và phép cắt lát của chuỗi.
- Chuẩn hóa văn bản bằng các phương thức phù hợp.
- Tách và ghép dữ liệu bằng `split` và `join`.
- Định dạng dữ liệu có chủ đích bằng f-string.
- Bước đầu sử dụng biểu thức chính quy mà không lạm dụng.
- Nhận biết Unicode, encoding và lý do văn bản có thể bị lỗi dấu.

4.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một dòng nhật ký có thời gian, mức lỗi và thông điệp. Với người đọc, đó là một câu; với chương trình, nó là dãy ký tự cần tách theo quy tắc. Chỉ một dấu cách thừa, ký tự Unicode hoặc khác biệt chữ hoa đã có thể làm phép so sánh thất bại. Xử lý chuỗi vì thế là bài toán về cấu trúc và chuẩn hóa, không chỉ là gọi `split`.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **chuỗi như dãy ký tự, mã hóa và dữ liệu cần phân tích**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

4.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: ký tự, mã Unicode, độ dài, delimiter, token, quy tắc chuẩn hóa và schema sau khi phân tích.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

4.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$s = c_0 c_1 \cdots c_{n-1}, \qquad P(s) = \big(t, \ell, m\big)$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: vị trí đang quét, token đang xây và các trường đã nhận diện?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

4.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Phân tích thành công phải bảo toàn thông tin cần thiết; chuẩn hóa không được làm hai dữ liệu có ý nghĩa khác nhau trở thành giống nhau.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

4.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **chuỗi như dãy ký tự, mã hóa và dữ liệu cần phân tích** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích

nhiều lỗi tưởng như bí ẩn.

4.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Phân tích dòng `2026-09-05 | ERROR | Sensor timeout`. Ta mô tả delimiter, giới hạn số lần tách, loại khoảng trắng ở biên, kiểm tra đủ ba trường và giữ nguyên thông điệp nếu bên trong nó cũng có dấu phân cách.

4.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

4.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

4.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay

đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

4.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

4.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

4.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định “kết quả ấy có vẻ được”, ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là “thuật toán giữ bất biến và tạo hậu điều kiện”. Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

4.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Gọi `split()` không đối số dù khoảng trắng bên trong trường có ý nghĩa.
- So sánh chuỗi chưa chuẩn hóa rồi tạo nhiều nhánh vá lỗi.
- Cắt chuỗi bằng chỉ số cố định khi định dạng có thể đổi.
- Cho rằng một ký tự hiển thị luôn tương ứng đúng một code point.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

4.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

4.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Thiết kế bộ phân tích tên tệp theo quy ước ngày–loại–số thứ tự.
2. Chuẩn hóa họ tên mà không làm mất dấu tiếng Việt.
3. Viết đặc tả cho parser dòng log và năm trường hợp sai.
4. So sánh tìm kiếm chính xác, tiền tố và biểu thức chính quy.
5. Giải thích vì sao định dạng dữ liệu ổn định quan trọng hơn chuỗi đẹp mắt.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ “gõ cho chạy” thành luyện tư duy thiết kế.

4.8 Chuỗi là dãy bất biến

Ta có thể đọc `s[0]`, `s[-1]` và `s[2:5]` giống list, nhưng không thể gán `s[0] = "A"`. Mọi phương thức như `lower()`, `replace()` tạo chuỗi mới.

```
ten_goc = "  nGUYEN van an  "
ten_sach = " ".join(ten_goc.split()).title()

print(repr(ten_goc))
print(ten_sach)
```

`split()` không đối số tách theo mọi cụm khoảng trắng và bỏ phần rỗng. `" ".join(...)` ghép lại bằng đúng một dấu cách. Đây là một quy trình chuẩn hóa, không chỉ là một lời gọi phương thức ngẫu nhiên.

4.9 Kiểm tra trước khi chuyển đổi

Các phương thức `isalpha`, `isdigit`, `isalnum`, `isspace` trả giá trị logic. Tuy vậy, thế giới Unicode rộng hơn các ký tự ASCII. `"²".isdigit()` có thể là đúng nhưng `int("²")` không chuyển được. Vì thế, cách đáng tin cậy nhất để đọc số là thử chuyển đổi và xử lý ngoại lệ; chương 8 sẽ đi sâu hơn.

```
def doc_so_nguyen(van_ban):
    try:
        return int(van_ban.strip())
    except ValueError:
        return None
```

4.10 Tách, phân vùng và ghép

`split(sep)` tách tất cả; `partition(sep)` luôn trả tuple ba phần: trước dấu, dấu, sau dấu. Khi chỉ cần tách lần đầu, `partition` làm ý định rõ.

```
dong = "email=nhut@example.com"
khoa, dau, gia_tri = dong.partition("=")
if dau:
    print(khoa, gia_tri)
```

`join` thuộc về chuỗi phân cách:

```
tu = ["hoc", "de", "hieu"]
cau = " ".join(tu)
csv_don_gian = ",".join(tu)
```

Không tự ghép CSV bằng dấu phẩy nếu dữ liệu có thể chứa dấu phẩy hoặc dấu ngoặc kép; chương 7 sẽ dùng mô-đun `csv` để xử lý quy tắc thoát ký tự đúng.

4.11 Tìm, thay và đếm có kiểm soát

`find` trả `-1` nếu không có; `index` báo lỗi. `startswith` và `endswith` diễn đạt việc kiểm tra tiền tố, hậu tố tốt hơn cắt lát thủ công. `replace(cu, moi, so_lan)` cho phép giới hạn số lần thay.

Che một phần số điện thoại

```
def che_so_dien_thoai(so):
    sach = "".join(ch for ch in so if ch.isdigit())
    if len(sach) < 7:
        raise ValueError("So dien thoai qua ngan")
    return sach[:3] + "*" * (len(sach) - 6) + sach[-3:]

print(che_so_dien_thoai("090-123-4567"))
```

Ta tạo một chuỗi mới chỉ gồm chữ số, kiểm tra hợp đồng tối thiểu rồi mới che. Nếu che trước khi chuẩn hóa, dấu gạch ngang có thể làm lệch vị trí.

4.12 F-string như một công cụ trình bày

F-string không chỉ chèn biến; nó định dạng số, độ rộng, căn lề và ngày tháng.

```
ten = "Python"
diem = 8.756
ti_le = 0.873

print(f"{ten:<12} | {diem:>6.2f} | {ti_le:>7.1%}")
print(f"{1234567:,}")
```

Thông số sau dấu hai chấm là “mini-language” định dạng. Hãy giữ dữ liệu thô để tính toán và chỉ định dạng ở lớp hiển thị. Chuỗi `"8.76"` không còn thuận tiện cho việc cộng trung bình như số `8.756`.

4.13 Unicode và encoding

Trong Python 3, `str` lưu văn bản Unicode; `bytes` lưu các byte. Encoding là quy tắc biến ký tự thành byte. UTF-8 là lựa chọn phổ biến, nhưng tệp cũ có thể dùng bảng mã khác.

```
van_ban = "Chao Viet Nam"
du_lieu = van_ban.encode("utf-8")
```

```
khoi_phuc = du_lieu.decode("utf-8")
```

Nếu mã hóa bằng một quy tắc và giải mã bằng quy tắc khác, chữ có thể biến dạng hoặc phát sinh `UnicodeDecodeError`. Vì vậy, khi mở tệp văn bản, hãy chỉ rõ `encoding="utf-8"`; đừng dựa vào mặc định của hệ điều hành.

Ký tự, byte và thứ mắt nhìn thấy

Một ký tự Unicode không nhất thiết bằng một byte, và một "hình" người dùng nhìn thấy đôi khi được tạo bởi nhiều code point. Với hầu hết bài học, thao tác trên `str` là đủ; nhưng đừng dùng `len(text)` để suy ra dung lượng tệp theo byte.

4.14 Biểu thức chính quy: một chiếc kính chuyên dụng

Mô-đun `re` mô tả mẫu văn bản. Nó hữu ích khi quy tắc gồm nhiều khả năng, nhưng khó đọc nếu dùng cho việc mà `split` hoặc `startswith` đã giải quyết rõ ràng.

```
import re

MA_SV = re.compile(r"^[A-Z]{2}\d{6}$")

def ma_hop_le(ma):
    return MA_SV.fullmatch(ma.strip().upper()) is not None

print(ma_hop_le("ai123456"))
```

Trong mẫu, `[A-Z]{2}` là đúng hai chữ hoa, `\d{6}` là sáu chữ số. Dùng raw string `r"..."` để dấu gạch chéo được chuyển nguyên cho bộ máy regex.

Regex không phải bộ hiểu ngôn ngữ

Một biểu thức khớp hình thức email không chứng minh hộp thư tồn tại. Một mẫu tên không thể bao quát mọi nền văn hóa. Hãy phân biệt kiểm tra cú pháp, kiểm tra nghiệp vụ và xác minh ngoài đời.

4.15 Bài toán hoàn chỉnh: chuẩn hóa nhật ký học

```
import re

def tach_nhat_ky(dong):
    # Mau: 2026-09-04 | Python | 75 phut
    mau = re.compile(
        r"^(\d{4}-\d{2}-\d{2})\s*\|\s*([^|]+)\s*\|\s*(\d+)\s*phut$",
        re.IGNORECASE,
    )
```

```
    khop = mau.fullmatch(dong.strip())
    if not khop:
        return None

    ngay, mon, phut = khop.groups()
    return {
        "ngay": ngay,
        "mon": " ".join(mon.split()).title(),
        "phut": int(phut),
    }
```

Hàm không cố kiểm tra ngày 2026-99-99 có tồn tại; regex chỉ tách hình thức. Chương về thư viện chuẩn sẽ dùng `datetime` để xác minh ý nghĩa ngày.

4.16 Bài tập luyện tập

1. Chuẩn hóa họ tên có nhiều khoảng trắng nhưng không làm sai các từ nối bạn tự quy định.
2. Viết hàm tạo slug: chữ thường, khoảng trắng thành gạch ngang, loại ký tự không cần thiết. Nêu giới hạn với tiếng Việt có dấu.
3. Đếm số câu, số từ và số ký tự không tính khoảng trắng trong một đoạn.
4. Kiểm tra mật khẩu theo các quy tắc độ dài, chữ hoa, chữ thường, số; trả danh sách quy tắc chưa đạt.
5. Tách URL thành giao thức, tên miền và đường dẫn ở mức đơn giản; sau đó so sánh với `urllib.parse`.
6. Dùng regex trích mọi số đo như `23.5 C`, `70%` trong một log.
7. Viết bộ che email giữ hai ký tự đầu tên hộp thư và giữ nguyên tên miền.
8. Thiết kế một định dạng dòng cho nhật ký chi tiêu và hàm phân tích ngược về dictionary.

4.17 Tóm tắt cuối chương

Bản đồ kiến thức

Chuỗi là dãy bất biến; mọi phép biến đổi tạo giá trị mới. Chuẩn hóa nên diễn ra theo một quy trình có thứ tự. `split`/`join` xử lý cấu trúc đơn giản, f-string xử lý trình bày, regex phù hợp mẫu phức tạp vừa phải. `str` và `bytes` không phải một; encoding là chiếc cầu giữa hai thế giới.

Góc nhìn cá nhân

Văn bản thường khiến ta chủ quan vì mắt người có thể hiểu một cái tên dù thừa hai dấu cách. Máy tính không có sự rộng lượng đó. Viết một hàm chuẩn hóa tốt là lúc ta biến những điều "ai cũng hiểu" thành các quyết định chính xác, có thể kiểm tra và có thể giải thích.

Chương 5

Cấu trúc lồng nhau và comprehension – Dữ liệu bắt đầu có hình dạng

Giới thiệu ngắn

Một danh sách số là một đường thẳng. Một bảng điểm là danh sách các hồ sơ. Một bàn cờ là danh sách các hàng. Khi cấu trúc lồng nhau, dữ liệu bắt đầu có hình dạng, nhưng mỗi lớp ngoặc cũng làm tăng gánh nặng đọc. Chương này học cách đi qua cấu trúc nhiều tầng, sao chép đúng và dùng comprehension vừa đủ để mã ngắn mà vẫn sáng.

Mục tiêu học tập

- Mô hình hóa bảng, ma trận và danh sách bản ghi.
- Duyệt an toàn qua nhiều tầng dữ liệu.
- Hiểu bản sao nông, bản sao sâu và lỗi nhân cùng một hàng.
- Viết list, set và dictionary comprehension.
- Nhận biết lúc nên quay về vòng lặp thông thường.

5.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Bảng điểm có nhiều học sinh, mỗi học sinh có nhiều môn. Đây không còn là một dãy phẳng mà là ma trận hoặc danh sách hồ sơ. Cấu trúc lồng nhau phải phản ánh câu hỏi ta thường đặt: truy cập theo hàng, theo cột, theo mã hay theo thời gian. Comprehension chỉ nên dùng sau khi phép biến đổi đã rõ bằng lời.

Một chương trình tốt không bắt đầu bằng việc hỏi “phải dùng câu lệnh nào?”. Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng

nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **dữ liệu lồng nhau và comprehension như phép biến đổi có cấu trúc**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

5.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: số hàng, số cột, ý nghĩa mỗi trục, khóa hồ sơ, điều kiện lọc và giá trị được biến đổi.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

5.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$A = [a_{ij}]_{m \times n}, \qquad r_i = \frac{1}{n} \sum_{j=1}^{n} a_{ij}$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: phần tử hiện tại, cấp lồng đang duyệt và kết quả đã xây?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

5.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Mọi hàng phải tuân schema đã chọn; phép biến đổi phải giữ quan hệ đúng giữa học sinh và môn học.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

5.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **dữ liệu lồng nhau và comprehension như phép biến đổi có cấu trúc** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

5.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Từ bảng điểm ba học sinh và bốn môn, ta tính trung bình từng học sinh, lấy danh sách môn dưới ngưỡng và dựng báo cáo mới mà không sửa bảng gốc. Trước khi dùng comprehension, ta viết vòng lặp thường và bảng truy vết hai cấp.

5.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

5.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

5.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

5.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

5.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

5.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

5.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Lồng comprehension đến mức không còn đọc được thứ tự xử lý.
- Tạo ma trận bằng phép nhân danh sách làm các hàng dùng chung tham chiếu.
- Giả định mọi hàng có cùng độ dài mà không kiểm tra schema.

- Trộn lọc, biến đổi và tác dụng phụ trong một biểu thức.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

5.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

5.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Vẽ chỉ số hàng/cột cho một ma trận 3×4.
2. Chuyển vòng lặp hai cấp thành comprehension rồi so sánh độ dễ đọc.
3. Tìm lỗi tham chiếu chung trong cách tạo ma trận.
4. Thiết kế cấu trúc dữ liệu cho thời khóa biểu theo ngày và tiết.
5. Viết kiểm thử cho dữ liệu lồng thiếu một trường bắt buộc.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

5.8 Danh sách các dictionary

Một mô hình phổ biến cho dữ liệu nhỏ là list giữ nhiều dictionary cùng cấu trúc.

```
lop_hoc = [
    {"ma": "SV01", "ten": "An", "diem": [8.0, 9.0]},
    {"ma": "SV02", "ten": "Binh", "diem": [7.0, 8.5]},
    {"ma": "SV03", "ten": "Chi", "diem": []},
]
```

```
6
7  for ban in lop_hoc:
8      ds_diem = ban["diem"]
9      trung_binh = sum(ds_diem) / len(ds_diem) if ds_diem else None
10     print(ban["ten"], trung_binh)
```

Hãy đặt tên biến ở mỗi tầng. Biểu thức `lop_hoc[i]["diem"][j]` không sai, nhưng người đọc phải giữ quá nhiều vị trí trong đầu. Tên `ban` và `ds_diem` biến đường đi qua dữ liệu thành câu chuyện dễ theo dõi.

5.9 Ma trận và lỗi nhân bản hàng

```
1  # Sai: ba vi tri cung tro den mot danh sach con
2  bang_sai = [[0] * 4] * 3
3  bang_sai[0][0] = 1
4  print(bang_sai)
5
6  # Dung: moi lan lap tao mot hang moi
7  bang = [[0] * 4 for _ in range(3)]
8  bang[0][0] = 1
```

Toán tử nhân danh sách lặp lại tham chiếu, không gọi phép tạo đối tượng mới cho từng hàng. Với các phần tử bất biến như số 0 ở một danh sách phẳng, điều đó không gây vấn đề. Với danh sách con có thể thay đổi, mọi hàng chia sẻ cùng đối tượng.

Vẽ mũi tên thay vì chỉ nhìn ngoặc

Hãy vẽ ba ô của `bang_sai` và các mũi tên từ chúng. Bạn sẽ thấy cả ba trỏ đến một hàng. Với `bang`, comprehension thực hiện biểu thức tạo hàng ba lần, sinh ba đối tượng khác nhau.

5.10 Bản sao nông và bản sao sâu

```
1  import copy
2
3  goc = [[1, 2], [3, 4]]
4  nong = goc.copy()
5  sau = copy.deepcopy(goc)
6
7  nong[0].append(99)
8  print(goc)  # hang dau cung thay doi
9  print(sau)  # doc lap
```

Bản sao nông tạo lớp chứa ngoài mới, còn phần tử bên trong vẫn được chia sẻ. Bản sao

sâu đi đệ quy và sao chép các thành phần. Tuy nhiên, `deepcopy` không phải câu thần chú dùng ở mọi nơi: nó có thể tốn bộ nhớ, khó xử lý tài nguyên đặc biệt và che một thiết kế sở hữu dữ liệu chưa rõ. Trước tiên hãy quyết định phần nào được chia sẻ, phần nào cần độc lập.

5.11 List comprehension – một phép biến đổi có thể đọc thành câu

```
binh_phuong_chan = [x * x for x in range(20) if x % 2 == 0]
```

Đọc từ trái sang phải theo ý nghĩa: lấy x^2, với mỗi x trong dãy, nếu x chẵn. Cùng ý tưởng bằng vòng lặp:

```
binh_phuong_chan = []
for x in range(20):
    if x % 2 == 0:
        binh_phuong_chan.append(x * x)
```

Comprehension phù hợp khi có một phép ánh xạ, có thể kèm một điều kiện lọc đơn giản. Nếu có nhiều nhánh, xử lý lỗi, ghi log hoặc biến trạng thái, vòng lặp thường rõ hơn.

5.12 Biểu thức điều kiện nằm ở đâu?

Hai cú pháp sau trả lời hai câu hỏi khác nhau:

```
# Loc: chi giu so duong
duong = [x for x in ds if x > 0]

# Bien doi: giu du so phan tu, thay so am bang 0
khong_am = [x if x >= 0 else 0 for x in ds]
```

`if` ở cuối quyết định phần tử có được đưa vào kết quả. `if...else` trước `for` quyết định mỗi phần tử được biến thành gì.

5.13 Set và dictionary comprehension

```
tu = ["Python", "python", "AI", "Data", "data"]
tu_chuan = {x.lower() for x in tu}

do_dai = {x: len(x) for x in sorted(tu_chuan)}
```

Set comprehension tự loại trùng. Dictionary comprehension cần sinh cặp `khoa: gia_tri`. Nếu nhiều phần tử sinh cùng khóa, giá trị đến sau ghi đè giá trị trước; hãy chắc đó là điều

mong muốn.

5.14 Comprehension lồng nhau

Làm phẳng một ma trận:

```
ma_tran = [[1, 2, 3], [4, 5, 6]]
phang = [gia_tri for hang in ma_tran for gia_tri in hang]
```

Thứ tự các `for` giống thứ tự vòng lặp từ ngoài vào trong. Dù vậy, khi có hơn hai tầng hoặc nhiều điều kiện, hãy viết vòng lặp thường. Mã nguồn được đọc nhiều lần hơn số lần nó được viết.

Chuyển vị ma trận:

```
chuyen_vi = [list(cot) for cot in zip(*ma_tran)]
```

Dấu sao tháo danh sách hàng thành các đối số cho `zip`. Đây là cú pháp đẹp sau khi đã hiểu, nhưng không nên ném vào một bài giải mà không giải thích. Một phiên bản vòng lặp hai tầng có thể phù hợp hơn cho người mới.

5.15 Một đường ống xử lý dữ liệu nhỏ

Lọc và xếp hạng hồ sơ

```
def diem_trung_binh(ho_so):
    diem = ho_so.get("diem", [])
    return sum(diem) / len(diem) if diem else None

def xep_hang(lop_hoc, nguong=5.0):
    ket_qua = []
    for ho_so in lop_hoc:
        tb = diem_trung_binh(ho_so)
        if tb is None or tb < nguong:
            continue
        ket_qua.append({
            "ma": ho_so["ma"],
            "ten": ho_so["ten"],
            "trung_binh": round(tb, 2),
        })
    return sorted(ket_qua, key=lambda x: x["trung_binh"], reverse=True)
```

Ta cố ý không ép toàn bộ vào một comprehension. Việc tính giá trị trung gian, bỏ hai trường hợp và dựng bản ghi mới được trình bày theo từng bước. “Pythonic” không có nghĩa là càng ít dòng càng tốt; nó là dùng đặc trưng Python để ý định rõ và hành vi ít bất ngờ.

5.16 Bài tập luyện tập

1. Tạo bảng 5×5 có đường chéo chính bằng 1, vị trí khác bằng 0.
2. Từ danh sách câu, tạo set mọi từ khác nhau dài ít nhất năm ký tự.
3. Tạo dictionary ánh xạ số từ 1 đến 20 sang "chẵn" hoặc "lẻ".
4. Làm phẳng danh sách lồng nhau đúng hai tầng, nhưng bỏ giá trị `None`.
5. Tính tổng từng hàng và từng cột của ma trận chữ nhật; kiểm tra các hàng có cùng độ dài.
6. Từ dữ liệu đơn hàng lồng nhau, tạo báo cáo tổng tiền theo khách hàng.
7. Viết hàm sao chép một cấu trúc và minh họa khác biệt giữa bản sao nông, sâu bằng ít nhất hai phép thay đổi.
8. Viết lại một comprehension phức tạp thành vòng lặp dễ đọc, rồi giải thích phiên bản nào bạn chọn trong dự án thật.

Bảng theo dõi thói quen

Biểu diễn 30 ngày, mỗi ngày là dictionary gồm ngày, số phút học, số trang đọc và trạng thái vận động. Viết các hàm tính chuỗi ngày học liên tiếp dài nhất, ngày có tổng hoạt động cao nhất và báo cáo theo tuần. Không giả định dữ liệu luôn đủ 30 ngày.

5.17 Tóm tắt cuối chương

Bản đồ kiến thức

Cấu trúc lồng nhau mô tả dữ liệu có nhiều tầng nhưng cần tên trung gian và hợp đồng rõ. Phép nhân danh sách con có thể tạo nhiều tham chiếu đến cùng đối tượng. Bản sao nông chỉ tách lớp ngoài; bản sao sâu tách tiếp các thành phần. Comprehension phù hợp với phép biến đổi và lọc ngắn, không phải cuộc thi nén mã.

Góc nhìn cá nhân

Khi một dòng mã khiến tôi thấy "thông minh", tôi thường đọc lại nó vào ngày hôm sau. Nếu phải giải mã chính thứ mình viết, sự thông minh ấy đã vay thời gian của tương lai. Cấu trúc tốt không làm người đọc kinh ngạc; nó làm họ bình tĩnh vì biết dữ liệu đang đi đâu.

Phần II

Làm cho chương trình sống lâu hơn một lần chạy

Chương 6

Tệp và đường dẫn – Cho chương trình một trí nhớ

Giới thiệu ngắn

Khi chương trình kết thúc, các biến trong bộ nhớ biến mất. Nếu một ứng dụng quản lý công việc quên sạch mọi thứ sau mỗi lần đóng, nó chỉ là một bản trình diễn. Tệp là bước đầu để chương trình có trí nhớ lâu dài. Nhưng đọc và ghi tệp không chỉ là gọi `open`; ta phải hiểu đường dẫn, encoding, chế độ mở, vòng đời tài nguyên và nguy cơ ghi đè dữ liệu.

Mục tiêu học tập

- Đọc, ghi và nối tiếp tệp văn bản bằng context manager.
- Phân biệt đường dẫn tương đối, tuyệt đối và thư mục làm việc.
- Dùng `pathlib.Path` để thao tác đường dẫn đa nền tảng.
- Xử lý tệp lớn theo dòng và ghi dữ liệu an toàn hơn.
- Nhận biết khác biệt giữa văn bản và dữ liệu nhị phân.

6.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một chương trình ghi nhật ký học tập phải tìm đúng thư mục, không ghi đè nhầm tệp và vẫn đóng tài nguyên khi lỗi xảy ra. Đường dẫn nhìn giống chuỗi nhưng mang cấu trúc của hệ thống tệp. Dữ liệu đã ghi còn tồn tại sau khi chương trình dừng, vì vậy sai lầm ở I/O có hậu quả lâu hơn sai lầm trong RAM.

Một chương trình tốt không bắt đầu bằng việc hỏi “phải dùng câu lệnh nào?”. Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng

nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **tệp và đường dẫn như bộ nhớ bền có ngữ cảnh**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

6.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: đường dẫn gốc, tên tệp, chế độ mở, encoding, vị trí đọc/ghi và chính sách khi tệp đã tồn tại.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

6.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$\text{path} = \text{root/directory/filename}, \qquad \text{new} = \text{old} \oplus \text{record}$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: đường dẫn đã phân giải, handle đang mở và dữ liệu đã ghi bền?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

6.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Chương trình chỉ đọc/ghi trong vùng được phép, luôn đóng tệp và không công bố thành công trước khi dữ liệu đã được ghi đầy đủ.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

6.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **tệp và đường dẫn như bộ nhớ bền có ngữ cảnh** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

6.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Lưu một mục nhật ký theo ngày. Ta dựng đường dẫn bằng `pathlib`, tạo thư mục cha có chủ đích, ghi UTF-8 vào tệp tạm, kiểm tra, rồi mới thay thế tệp đích để tránh bản ghi nửa chừng.

6.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

6.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

6.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

6.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

6.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

6.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

6.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Ghép đường dẫn bằng dấu gạch chéo thủ công.
- Dùng chế độ `w` rồi vô tình xóa nội dung cũ.
- Phụ thuộc thư mục chạy hiện tại mà không ghi rõ.
- Đọc toàn bộ tệp lớn vào bộ nhớ dù chỉ cần từng dòng.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

6.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

6.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Vẽ cây thư mục cho một dự án và chỉ rõ đường dẫn tương đối/tuyệt đối.
2. Thiết kế cách ghi an toàn không để lại tệp hỏng khi chương trình dừng.
3. Đọc log từng dòng và thống kê mà không tải toàn bộ.
4. Tạo ca thử cho tệp thiếu, không đủ quyền và sai encoding.
5. Giải thích vòng đời tài nguyên trong `with`.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

6.8 Đường dẫn không phải chính tệp

Một đường dẫn là chỉ dẫn để hệ điều hành tìm tài nguyên. Đường dẫn tương đối được hiểu từ *thư mục làm việc hiện tại*, không nhất thiết từ nơi tệp mã nguồn nằm. Đây là nguyên nhân của câu chuyện quen thuộc: chương trình chạy trong IDE nhưng lỗi khi chạy từ Terminal khác.

```
from pathlib import Path

print(Path.cwd())
duong_dan = Path("data") / "nhat_ky.txt"
print(duong_dan.resolve())
```

Toán tử `/` của `Path` ghép thành phần theo quy tắc hệ điều hành. Không cần tự nối bằng `"/"` hoặc `"\"`.

Muốn đường dẫn dựa trên vị trí tệp chương trình:

```
THU_MUC_GOC = Path(__file__).resolve().parent
TEP_DU_LIEU = THU_MUC_GOC / "data" / "nhat_ky.txt"
```

Trong notebook hoặc trình thông dịch, `__file__` có thể không tồn tại; khi đó hãy chọn một thư mục gốc rõ ràng từ cấu hình hoặc `Path.cwd()`.

6.9 Đọc tệp với with

```
duong_dan = Path("data/nhat_ky.txt")

with duong_dan.open("r", encoding="utf-8") as tep:
    noi_dung = tep.read()

print(noi_dung)
```

Khối `with` bảo đảm tệp được đóng ngay cả khi bên trong phát sinh lỗi. Đây là context manager: nó quản lý bước chuẩn bị và dọn dẹp tài nguyên. Đóng tệp giải phóng mô tả tệp và bảo đảm dữ liệu đệm được đẩy xuống hệ thống.

Các cách đọc:

- `read()` lấy toàn bộ nội dung, phù hợp tệp nhỏ.
- `readline()` lấy một dòng.
- Duyệt `for dong in tep` đọc lần lượt, phù hợp tệp lớn.
- `read_text(encoding="utf-8")` của `Path` là cách gọn cho tệp nhỏ.

Đếm dòng hợp lệ mà không giữ cả tệp

```
def dem_dong_du_lieu(duong_dan):
    so_dong = 0
    with Path(duong_dan).open(encoding="utf-8") as tep:
        for dong in tep:
            if dong.strip() and not dong.lstrip().startswith("#"):
                so_dong += 1
    return so_dong
```

6.10 Ghi mới, ghi đè và nối tiếp

Chế độ `"w"` tạo mới hoặc xóa nội dung cũ rồi ghi. `"a"` nối vào cuối. `"x"` chỉ tạo khi tệp chưa tồn tại và báo lỗi nếu đã có. `"r"` chỉ đọc.

```
with Path("bao_cao.txt").open("w", encoding="utf-8") as tep:
```

```
    tep.write("BAO CAO TUAN\n")
    tep.write("Tong thoi gian: 420 phut\n")

with Path("bao_cao.txt").open("a", encoding="utf-8") as tep:
    tep.write("Ghi chu: duy tri deu dan\n")
```

`write` không tự thêm ký tự xuống dòng. Trên mọi hệ điều hành, viết `"\n"`; chế độ văn bản sẽ chuyển đổi phù hợp khi cần.

Một chữ w có thể xóa cả tệp

Trước khi mở đường dẫn do người dùng cung cấp bằng `"w"`, hãy xác nhận đích, tạo bản sao nếu dữ liệu quan trọng và cân nhắc `"x"`. Nhiều lỗi mất dữ liệu không đến từ thuật toán mà từ một chế độ mở sai.

6.11 Thao tác thư mục bằng pathlib

```
thu_muc = Path("exports")
thu_muc.mkdir(parents=True, exist_ok=True)

for tep in thu_muc.glob("*.txt"):
    print(tep.name, tep.stat().st_size)
```

`exists`, `is_file`, `is_dir` kiểm tra loại đích. `suffix` lấy phần mở rộng, `stem` lấy tên không phần mở rộng, `with_suffix` tạo đường dẫn với đuôi khác. `rglob("*.py")` tìm đệ quy.

Kiểm tra tồn tại rồi mới thao tác không hoàn toàn loại được lỗi vì tệp có thể bị thay đổi giữa hai bước. Vẫn phải xử lý `OSError` tại thao tác thật.

6.12 Ghi an toàn bằng tệp tạm

Nếu chương trình đang ghi mà mất điện, tệp có thể chỉ còn một phần. Một mẫu an toàn hơn là ghi vào tệp tạm trong cùng thư mục, sau đó thay thế đích bằng thao tác đổi tên.

```
from pathlib import Path

def ghi_an_toan(duong_dan, noi_dung):
    dich = Path(duong_dan)
    dich.parent.mkdir(parents=True, exist_ok=True)
    tam = dich.with_suffix(dich.suffix + ".tmp")
    tam.write_text(noi_dung, encoding="utf-8")
    tam.replace(dich)
```

Việc thay thế thường nguyên tử trên cùng hệ thống tệp, nhưng chi tiết phụ thuộc nền

tảng. Với dữ liệu quan trọng, cần chiến lược sao lưu, đồng bộ xuống đĩa và cơ sở dữ liệu có giao dịch. Mẫu này vẫn tốt hơn ghi đè trực tiếp cho ứng dụng học tập nhỏ.

6.13 Văn bản và nhị phân

Ảnh, PDF, tệp nén phải đọc ở chế độ byte: `"rb"`, `"wb"`. Không giải mã tùy tiện như UTF-8.

```
nguon = Path("anh.png")
dich = Path("anh_sao.png")

with nguon.open("rb") as vao, dich.open("wb") as ra:
    while khoi := vao.read(64 * 1024):
        ra.write(khoi)
```

Biểu thức gán `:=` vừa đọc khối vừa dùng làm điều kiện. Khi trả về `b''`, vòng lặp dừng. Nếu cú pháp này còn lạ, tách thành vòng `while True` với lệnh đọc và `break`; rõ ràng quan trọng hơn ngắn.

6.14 Bài tập luyện tập

1. Đọc tệp văn bản và thống kê số dòng, từ, ký tự.
2. Viết hàm nối một dòng log có thời gian hiện tại vào tệp; không xóa log cũ.
3. Quét một thư mục và nhóm tệp theo phần mở rộng.
4. Tạo bản sao tệp chỉ khi đích chưa tồn tại; báo lỗi dễ hiểu.
5. Tìm mười dòng dài nhất trong tệp lớn mà không dùng `read()` toàn bộ.
6. Viết công cụ đổi tên hàng loạt ở chế độ xem trước: in kế hoạch, chỉ đổi khi người dùng xác nhận.
7. Thiết kế định dạng văn bản đơn giản cho danh sách công việc và hàm đọc ngược. Nêu hạn chế khi tiêu đề chứa ký tự phân cách.
8. Cải tiến `ghi_an_toan` để giữ một bản `.bak` và phục hồi khi tệp chính lỗi.

6.15 Tóm tắt cuối chương

Bản đồ kiến thức

Tệp cho chương trình bộ nhớ bền, còn đường dẫn là chỉ dẫn tìm tệp. `Path` diễn đạt đường dẫn rõ và đa nền tảng. Dùng `with` để quản lý vòng đời, chỉ rõ UTF-8 cho văn bản, đọc theo dòng khi dữ liệu lớn. Chế độ `w` ghi đè; mẫu tệp tạm giảm nguy cơ để lại dữ liệu dở dang.

Góc nhìn cá nhân

Lúc chương trình chỉ sống trong một lần chạy, lỗi thường biến mất cùng dữ liệu. Khi nó bắt đầu ghi ra đĩa, mỗi quyết định trở nên có hậu quả dài hơn. Tôi xem chương về tệp là lúc người học bắt đầu chịu trách nhiệm không chỉ với câu lệnh, mà với ký ức mà chương trình giữ hộ người dùng.

Chương 7

JSON và CSV – Khi tệp cần một cấu trúc chung

Giới thiệu ngắn

Tệp văn bản tự nghĩ ra dấu phân cách hoạt động cho đến ngày tiêu đề chứa chính dấu đó. Các định dạng chuẩn giúp chương trình của ta trao đổi dữ liệu với bảng tính, dịch vụ web và ngôn ngữ khác. CSV gần với bảng; JSON gần với list và dictionary. Học định dạng không chỉ là gọi hàm đọc–ghi, mà còn là hiểu dữ liệu nào được giữ, dữ liệu nào bị mất và cách kiểm tra biên đầu vào.

Mục tiêu học tập

- Đọc, ghi CSV đúng quy tắc bằng mô-đun `csv`.
- Chuyển đổi giữa đối tượng Python và JSON.
- Hiểu giới hạn kiểu dữ liệu, encoding và schema.
- Kiểm tra, phiên bản hóa và di chuyển dữ liệu lưu trữ.
- Viết thao tác lưu tải có lỗi rõ ràng và ghi an toàn.

7.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một ứng dụng xuất bảng điểm cho cả con người dùng bảng tính và chương trình web. CSV phù hợp bảng phẳng; JSON giữ được cấu trúc lồng. Chọn định dạng không chỉ dựa vào đuôi tệp mà dựa vào schema, kiểu dữ liệu, cách xử lý giá trị thiếu và bên sẽ đọc dữ liệu.

Một chương trình tốt không bắt đầu bằng việc hỏi “phải dùng câu lệnh nào?”. Nó bắt

đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **JSON và CSV như hợp đồng trao đổi dữ liệu**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

7.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: tên trường, kiểu, trường bắt buộc, giá trị thiếu, delimiter, encoding và phiên bản schema.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

7.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$\text{object} \xleftrightarrow[\text{decode}]{\text{encode}} \text{bytes}, \qquad \text{schema}(d) = \text{valid}$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: bản ghi đang đọc, lỗi schema và tập bản ghi đã xác nhận?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

7.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không

còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Dữ liệu sau vòng encode–decode phải giữ nguyên ý nghĩa; mọi bản ghi được chấp nhận phải thỏa cùng schema đã công bố.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

7.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **JSON và CSV như hợp đồng trao đổi dữ liệu** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

7.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Xuất danh sách việc học sang JSON để ứng dụng đọc lại và sang CSV để mở trong bảng tính. Ta quy định ngày theo ISO, thời lượng là số phút, trạng thái là tập giá trị hữu hạn và kiểm tra từng bản ghi trước khi ghi.

7.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: “Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?” Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

7.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

7.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

7.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

7.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

7.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

7.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Xem JSON như nơi lưu mọi đối tượng Python trực tiếp.
- Tự nối chuỗi CSV và bỏ qua dấu phẩy hoặc xuống dòng trong dữ liệu.
- Không có phiên bản schema nên tệp cũ không đọc được sau cập nhật.
- Tin rằng parse thành công đồng nghĩa dữ liệu có ý nghĩa hợp lệ.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

7.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

7.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Thiết kế schema JSON cho một phiên học.
2. Tạo bản ghi CSV chứa dấu phẩy và giải thích cơ chế quoting.
3. Viết bộ kiểm tra trường bắt buộc trước khi lưu.
4. Lập kế hoạch chuyển schema từ phiên bản 1 sang 2.
5. So sánh round-trip JSON với CSV cho dữ liệu lồng.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ “gõ cho chạy” thành luyện tư duy thiết kế.

7.8 CSV không đơn giản chỉ là split bằng dấu phẩy

Một ô có thể chứa dấu phẩy, dấu ngoặc kép hoặc xuống dòng. Mô-đun `csv` biết cách bao và thoát chúng.

```
import csv
from pathlib import Path

du_lieu = [
    {"ma": "SV01", "ten": "Nguyen An", "diem": 8.5},
    {"ma": "SV02", "ten": "Tran Binh, Jr.", "diem": 9.0},
]

with Path("diem.csv").open("w", encoding="utf-8", newline="") as tep:
    cot = ["ma", "ten", "diem"]
    writer = csv.DictWriter(tep, fieldnames=cot)
    writer.writeheader()
```

```
    writer.writerows(du_lieu)
```

Tham số `newline=""` để mô-đun `csv` tự quản lý xuống dòng, tránh dòng trống thừa trên một số nền tảng. Khi đọc, mọi ô ban đầu là chuỗi; ta phải chuyển điểm về số và kiểm tra miền giá trị.

```
def doc_diem(duong_dan):
    ket_qua = []
    with Path(duong_dan).open(encoding="utf-8", newline="") as tep:
        for so_dong, hang in enumerate(csv.DictReader(tep), start=2):
            try:
                diem = float(hang["diem"])
            except (KeyError, ValueError) as loi:
                raise ValueError(f"Dong {so_dong} co diem khong hop le") from
    loi
            ket_qua.append({**hang, "diem": diem})
    return ket_qua
```

Số dòng bắt đầu từ 2 vì dòng đầu là tiêu đề. Thông báo có vị trí giúp người dùng sửa dữ liệu, thay vì chỉ nhận “could not convert string to float”.

7.9 JSON như cây dữ liệu

JSON có object, array, string, number, boolean và null. Chúng tương ứng gần với dictionary, list, str, int/float, bool và `None`.

```
import json

cau_hinh = {
    "theme": "dark",
    "minutes_per_session": 50,
    "notifications": True,
    "courses": ["Python", "Calculus"],
}

with open("config.json", "w", encoding="utf-8") as tep:
    json.dump(cau_hinh, tep, ensure_ascii=False, indent=2)
```

`ensure_ascii=False` giữ chữ Unicode dễ đọc; tệp vẫn mã hóa UTF-8. `indent=2` đẹp cho con người nhưng lớn hơn định dạng nén. `dump/load` làm việc với tệp; `dumps/loads` làm việc với chuỗi.

7.10 Những kiểu JSON không biết

JSON không có set, tuple, `datetime`, `Decimal` hay đối tượng lớp tùy chỉnh. Tuple khi ghi thường thành array và đọc lại thành list, tức kiểu gốc bị mất. Ta phải chọn cách biểu diễn.

```
from datetime import datetime

phien_hoc = {
    "bat_dau": datetime.now().isoformat(timespec="seconds"),
    "tags": sorted({"python", "evening"}),
}
```

Chuỗi ISO 8601 dễ trao đổi, nhưng cần quy ước múi giờ. Với tiền, số thực có thể gây sai số biểu diễn; lưu số đơn vị nhỏ nhất bằng integer hoặc chuỗi decimal là lựa chọn an toàn hơn tùy nghiệp vụ.

7.11 Schema – hợp đồng nằm ngoài cú pháp

Một JSON hợp lệ cú pháp vẫn có thể sai nghiệp vụ: thiếu `id`, `priority` là 99, ngày không đúng. Ta cần hàm kiểm tra hoặc thư viện schema.

```
def kiem_tra_task(task):
    loi = []
    khoa = {"id", "title", "done", "priority"}
    thieu = khoa - task.keys()
    if thieu:
        loi.append(f"Thieu: {', '.join(sorted(thieu))}")
    if "priority" in task and task["priority"] not in {1, 2, 3}:
        loi.append("priority phai la 1, 2 hoac 3")
    if "done" in task and not isinstance(task["done"], bool):
        loi.append("done phai la boolean")
    return loi
```

Kiểm tra ở biên ngay sau khi đọc. Phần lõi chương trình được phép giả định dữ liệu đã hợp lệ, nhờ đó bớt các điều kiện rải rác.

7.12 Phiên bản hóa dữ liệu

Hôm nay mỗi công việc có `title`; ngày mai thêm `due`. Nếu mở tệp cũ, chương trình không nên vỡ.

```
def migrate(data):
    version = data.get("version", 1)

    if version == 1:
```

```
        for task in data.get("tasks", []):
            task.setdefault("due", None)
        data["version"] = 2
        version = 2

    if version != 2:
        raise ValueError(f"Khong ho tro version {version}")
    return data
```

Mỗi bước di chuyển phải nhỏ, có kiểm thử và không xóa thông tin ngoài ý muốn. Giữ trường `version` biến thay đổi ngầm thành một quá trình có tên.

7.13 Kho lưu JSON nhỏ nhưng tử tế

```
import json
from pathlib import Path

class JsonStore:
    def __init__(self, path):
        self.path = Path(path)

    def load(self):
        if not self.path.exists():
            return {"version": 2, "tasks": []}
        try:
            text = self.path.read_text(encoding="utf-8")
            return migrate(json.loads(text))
        except json.JSONDecodeError as loi:
            raise ValueError(
                f"JSON loi tai dong {loi.lineno}, cot {loi.colno}"
            ) from loi

    def save(self, data):
        text = json.dumps(data, ensure_ascii=False, indent=2)
        tmp = self.path.with_suffix(self.path.suffix + ".tmp")
        self.path.parent.mkdir(parents=True, exist_ok=True)
        tmp.write_text(text, encoding="utf-8")
        tmp.replace(self.path)
```

Lớp sẽ được giải thích sâu ở Phần 4. Ngay lúc này, hãy nhìn nó như một chiếc hộp gom hai thao tác và đường dẫn. Phần còn lại không cần biết JSON được mở ra sao.

7.14 CSV hay JSON?

Tiêu chí	CSV	JSON
Hình dạng	Bảng hàng–cột	Cây lồng nhau
Kiểu dữ liệu	Thường đọc thành chuỗi	Giữ số, boolean, null
Trao đổi với bảng tính	Rất thuận tiện	Cần chuyển đổi
Dữ liệu lồng nhau	Khó, phải làm phẳng	Tự nhiên
Đọc từng dòng lớn	Thuận tiện	Một tệp JSON lớn khó stream đơn giản

7.15 Bài tập luyện tập

1. Xuất bảng điểm CSV, rồi đọc lại và tính trung bình. Tạo một tên có dấu phẩy để kiểm tra.
2. Chuyển CSV sang JSON, giữ kiểu số đúng.
3. Viết bộ kiểm tra tiêu đề CSV và báo cột thiếu, cột dư.
4. Thiết kế schema JSON cho thư viện cá nhân có sách và tác giả.
5. Thêm phiên bản 3 cho kho công việc, đổi `done` thành trạng thái `todo/doing/done`; viết hàm di chuyển.
6. Viết chế độ nhập dữ liệu bỏ qua dòng lỗi nhưng ghi lỗi ra tệp riêng; cân nhắc khi nào không nên bỏ qua.
7. So sánh kích thước JSON đẹp và JSON nén trên 10.000 bản ghi.
8. Tạo bản sao lưu có dấu thời gian trước mỗi lần lưu và chỉ giữ năm bản gần nhất.

7.16 Tóm tắt cuối chương

Bản đồ kiến thức

CSV phù hợp dữ liệu dạng bảng; JSON phù hợp cấu trúc cây. Dùng mô-đun chuẩn thay vì tự tách dấu phẩy. Việc đọc không kết thúc ở cú pháp: phải chuyển kiểu và kiểm tra schema. Khi cấu trúc thay đổi, trường phiên bản cùng các bước migrate giúp tệp cũ tiếp tục sống. Ghi qua tệp tạm giảm rủi ro dữ liệu dở dang.

Góc nhìn cá nhân

Định dạng tệp là một lời hứa với phiên bản tương lai của chương trình. Khi viết dữ liệu xuống đĩa, tôi cố tưởng tượng chính mình sáu tháng sau đang mở lại nó. Một trường `version`,

một thông báo có số dòng và một bản sao lưu nhỏ thường là sự tử tế mà tương lai sẽ nhận ra.

Chương 8

Lỗi và ngoại lệ – Không giấu điều sai, hãy cho nó một đường đi

Giới thiệu ngắn

Chương trình thật luôn gặp thứ nằm ngoài kịch bản đẹp: người dùng nhập chữ thay vì số, tệp bị xóa, mạng ngắt, dữ liệu thiếu trường. Mục tiêu không phải làm chương trình "không bao giờ lỗi"; đó là điều bất khả thi. Mục tiêu là phân biệt lỗi nào có thể phục hồi, lỗi nào phải dừng, giữ nguyên nhân đủ rõ và không làm hỏng trạng thái đang có.

Mục tiêu học tập

- Đọc traceback để tìm vị trí và loại ngoại lệ.
- Sử dụng `try/except/else/finally` đúng vai trò.
- Bắt ngoại lệ cụ thể, giữ chuỗi nguyên nhân bằng `raise ... from`.
- Chủ động phát sinh ngoại lệ và thiết kế ngoại lệ riêng.
- Phân biệt kiểm tra trước với thử thao tác rồi xử lý lỗi.

8.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một chương trình đọc điểm từ tệp có thể gặp tệp không tồn tại, dòng sai định dạng hoặc điểm ngoài thang. Ba lỗi này không giống nhau: một lỗi thuộc môi trường, một lỗi thuộc cú pháp dữ liệu và một lỗi thuộc nghiệp vụ. Nếu bắt tất cả bằng một `except`, chương trình mất khả năng giải thích và phục hồi đúng.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng

nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **ngoại lệ như đường đi có chủ đích của thất bại**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

8.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: tiền điều kiện, loại thất bại, mức có thể phục hồi, thông điệp và tài nguyên cần dọn dẹp.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

8.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$\text{operation} : \text{input} \to \text{result} \cup \text{failure}$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: bước đang thực hiện, ngoại lệ đang lan truyền và tài nguyên đang sở hữu?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

8.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Chỉ bắt lỗi tại nơi có đủ ngữ cảnh để xử lý; mọi tài nguyên phải được giải phóng dù thành công hay thất bại.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

8.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **ngoại lệ như đường đi có chủ đích của thất bại** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

8.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Nạp cấu hình ứng dụng từ JSON. Ta tách lỗi không có tệp, JSON hỏng và thiếu khóa bắt buộc; dùng giá trị mặc định chỉ cho trường được phép, còn lỗi làm thay đổi ý nghĩa phải được báo rõ.

8.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: “Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?” Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

8.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

8.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

8.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

8.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

8.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định “kết quả ấy có vẻ được”, ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là “thuật toán giữ bất biến và tạo hậu điều kiện”. Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

8.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Bắt `Exception` rồi bỏ qua.
- Dùng ngoại lệ thay cho luồng điều khiển bình thường không cần thiết.
- Báo lỗi quá thấp nên mất ngữ cảnh nghiệp vụ.
- Dọn tài nguyên chỉ ở nhánh thành công.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

8.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

8.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Vẽ cây lỗi cho quy trình đọc và kiểm tra tệp cấu hình.
2. Phân loại lỗi nào nên thử lại, lỗi nào cần người dùng sửa.
3. Thiết kế thông điệp lỗi chứa đủ ngữ cảnh nhưng không lộ dữ liệu nhạy cảm.
4. So sánh `else` và `finally` trong một giao dịch.
5. Tạo ngoại lệ nghiệp vụ cho điểm ngoài miền.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ “gõ cho chạy” thành luyện tư duy thiết kế.

8.8 Ba nhóm vấn đề

Lỗi cú pháp khiến Python không phân tích được chương trình. Ngoại lệ xảy ra khi cú pháp hợp lệ nhưng thao tác không thực hiện được, như chia 0 hay mở tệp thiếu. Lỗi logic nguy hiểm hơn: chương trình chạy và cho kết quả sai.

Traceback nên đọc từ dòng cuối: tên ngoại lệ và thông điệp. Sau đó đi ngược các khung gọi để tìm dòng mã của mình. Đừng chỉ sao chép dòng cuối lên công cụ tìm kiếm; thông tin đường đi mới cho biết dữ liệu đến lỗi bằng cách nào.

8.9 Cấu trúc try đầy đủ

```
try:
    van_ban = input("Nhap diem: ")
    diem = float(van_ban)
except ValueError:
```

```
    print("Diem phai la mot con so.")
else:
    print(f"Da nhan diem {diem}")
finally:
    print("Ket thuc thao tac nhap.")
```

`except` chạy khi ngoại lệ phù hợp phát sinh trong `try`. `else` chỉ chạy nếu không lỗi, giúp vùng `try` ngắn. `finally` luôn được cố gắng thực hiện, phù hợp dọn tài nguyên; context manager thường là cách tốt hơn cho tệp.

8.10 Bắt đúng thứ mình xử lý được

```
try:
    data = json.loads(path.read_text(encoding="utf-8"))
except FileNotFoundError:
    data = {"tasks": []}
except json.JSONDecodeError as loi:
    print(f"JSON sai tai dong {loi.lineno}")
    raise
```

Không dùng `except Exception: pass`. Nó có thể nuốt lỗi lập trình như gõ sai khóa và khiến dữ liệu biến mất không dấu vết. Chỉ bắt ngoại lệ khi tầng hiện tại có thể phục hồi, thêm ngữ cảnh hoặc dọn dẹp; nếu không, hãy để nó đi lên.

Không đặt cả chương trình trong một try khổng lồ

Vùng `try` càng rộng, bạn càng khó biết thao tác nào tạo ngoại lệ. Hãy bao đúng câu lệnh có thể thất bại và đặt xử lý thành công ở `else` khi thích hợp.

8.11 raise và hợp đồng hàm

```
def tinh_trung_binh(ds):
    if not ds:
        raise ValueError("Can it nhat mot gia tri")
    if not all(isinstance(x, (int, float)) for x in ds):
        raise TypeError("Moi phan tu phai la so")
    return sum(ds) / len(ds)
```

Ngoại lệ là một phần giao diện hàm. `ValueError` nói kiểu dữ liệu đúng nhưng giá trị không phù hợp; `TypeError` nói loại đối tượng sai. Thông điệp nên mô tả điều kiện vi phạm và, khi an toàn, giá trị liên quan.

8.12 Giữ nguyên nhân khi dịch lỗi

Một tầng lưu trữ có thể không muốn để lỗi JSON lan ra giao diện. Ta tạo ngoại lệ có ý nghĩa miền bài toán nhưng giữ nguyên nhân:

```
class DataStoreError(Exception):
    """Khong the doc hoac ghi kho du lieu."""

def tai_cong_viec(path):
    try:
        return json.loads(Path(path).read_text(encoding="utf-8"))
    except (OSError, json.JSONDecodeError) as loi:
        raise DataStoreError(f"Khong the tai {path}") from loi
```

`from loi` tạo chuỗi nguyên nhân trong traceback: lỗi cấp cao dễ hiểu và lỗi gốc vẫn phục vụ chẩn đoán.

8.13 Ngoại lệ riêng có dữ liệu

```
class ValidationError(Exception):
    def __init__(self, errors):
        self.errors = list(errors)
        super().__init__("; ".join(self.errors))

def validate_task(task):
    errors = []
    if not task.get("title", "").strip():
        errors.append("Tieu de khong duoc rong")
    if task.get("priority") not in {1, 2, 3}:
        errors.append("Do uu tien khong hop le")
    if errors:
        raise ValidationError(errors)
```

Tầng giao diện có thể hiển thị từng lỗi. Đừng tạo một lớp ngoại lệ cho mọi câu chữ; tạo khi người gọi cần phân biệt loại thất bại hoặc lấy thêm dữ liệu có cấu trúc.

8.14 LBYL và EAFP

LBYL (*look before you leap*) kiểm tra trước. EAFP (*easier to ask forgiveness than permission*) thử thao tác rồi xử lý ngoại lệ. Python thường khuyến khích EAFP khi thao tác là nguồn sự thật.

```
# Kiem tra truoc
if "email" in profile:
    email = profile["email"]
```

```
4
5 # Thu va xu ly
6 try:
7     email = profile["email"]
8 except KeyError:
9     email = None
```

Với tệp, kiểm tra `exists()` rồi mở vẫn có khoảng thời gian tệp bị xóa. Bắt ngoại lệ "không tìm thấy tệp" ngay ở thao tác mở đáng tin cậy hơn. Tên ngoại lệ Python là `FileNotFoundError`. Tuy nhiên, nếu điều kiện là một nhánh nghiệp vụ bình thường, `if` có thể dễ đọc hơn ngoại lệ.

8.15 Vòng nhập có khả năng phục hồi

```
1 def nhap_diem():
2     while True:
3         try:
4             diem = float(input("Diem [0-10]: "))
5             if not 0 <= diem <= 10:
6                 raise ValueError("Diem nam ngoai khoang")
7         except ValueError as loi:
8             print(f"Du lieu chua dung: {loi}")
9         else:
10            return diem
```

Vòng lặp chỉ bắt lỗi dữ liệu dự kiến và trả về khi hợp lệ. `KeyboardInterrupt` không bị nuốt, nên người dùng vẫn có thể dừng chương trình.

8.16 Bài tập luyện tập

1. Viết hàm nhập số nguyên có giới hạn số lần thử; sau đó phát sinh ngoại lệ riêng khi hết lượt.
2. Đọc tệp cấu hình và phân biệt thiếu tệp, không có quyền, JSON hỏng, schema sai.
3. Sửa một đoạn mã `except Exception: return []` để không che lỗi.
4. Thiết kế `InsufficientBalanceError` giữ số dư và số tiền yêu cầu.
5. Viết context manager nhỏ ghi thời gian chạy, bảo đảm vẫn ghi khi khối lệnh lỗi.
6. Tạo ba tầng hàm đọc–kiểm tra–tính báo cáo; dùng `raise from` để traceback giữ đủ bối cảnh.
7. Lập bảng những lỗi có thể phục hồi và không thể phục hồi trong ứng dụng quản lý học tập.

8. Viết kiểm thử chứng minh một hàm phát sinh đúng loại ngoại lệ và thông điệp.

8.17 Tóm tắt cuối chương

Bản đồ kiến thức

Traceback là bản đồ, không phải bức tường chữ. Bắt ngoại lệ cụ thể ở nơi có thể xử lý. `else` tách đường thành công, `finally` dành cho dọn dẹp. `raise` biểu đạt hợp đồng; `raise ... from` giữ chuỗi nguyên nhân. Ngoại lệ không nên thay mọi câu lệnh điều kiện và tuyệt đối không nên bị im lặng vô cớ.

Góc nhìn cá nhân

Một chương trình trưởng thành không phải chương trình luôn mỉm cười và nói "thành công". Nó biết thừa nhận điều không thể làm, nói chính xác mình vấp ở đâu và rời đi mà không làm hỏng những thứ đã có. Cách ta xử lý lỗi thường cho thấy chất lượng thiết kế rõ hơn con đường mọi thứ đều thuận lợi.

Chương 9

Module, package và môi trường – Chia một chương trình thành những căn phòng

Giới thiệu ngắn

Một tệp vài trăm dòng có thể chạy, nhưng mỗi lần sửa giống như tìm đồ trong căn phòng không có ngăn. Module cho phép đặt các định nghĩa liên quan vào tệp riêng; package nhóm nhiều module; môi trường ảo giữ phụ thuộc của từng dự án không giẫm lên nhau. Đây là bước chuyển từ bài tập đơn lẻ sang dự án có cấu trúc.

Mục tiêu học tập

- Tạo và nhập module mà không gây tác dụng phụ bất ngờ.
- Hiểu `__name__`, phạm vi tên và đường tìm module.
- Tổ chức package nhỏ với giao diện công khai rõ.
- Tạo môi trường ảo, cài và ghi phụ thuộc.
- Phân biệt mã ứng dụng, mã thư viện và điểm khởi chạy.

9.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một dự án ban đầu có một tệp, sau đó thêm đọc dữ liệu, báo cáo và giao diện. Nếu mọi hàm nằm chung, thay đổi nhỏ gây hiệu ứng dây chuyền. Module và package tạo ranh giới trách nhiệm; môi trường ảo tạo ranh giới phiên bản. Cả hai đều trả lời câu hỏi: thành phần này phụ thuộc vào điều gì và ai được phép dùng nó?

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **module, package và môi trường như ranh giới phụ thuộc**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

9.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: module, API công khai, hướng phụ thuộc, phiên bản gói, điểm vào chương trình và cấu hình môi trường.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

9.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$G = (V, E), \qquad (u, v) \in E \iff u \text{ phụ thuộc } v$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: module đã nạp, namespace và tập phụ thuộc đã khóa phiên bản?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

9.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Đồ thị phụ thuộc không được tạo vòng không chủ ý; API công khai phải ổn định hơn chi tiết cài đặt bên trong.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

9.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **module, package và môi trường như ranh giới phụ thuộc** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

9.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Chia ứng dụng quản lý học thành `domain`, `storage`, `report` và `cli`. Miền không phụ thuộc giao diện; storage thực hiện một hợp đồng; CLI ghép các thành phần tại điểm vào.

9.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

9.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

9.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

9.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

9.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

9.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định “kết quả ấy có vẻ được”, ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là “thuật toán giữ bất biến và tạo hậu điều kiện”. Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

9.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Tách tệp theo kích thước thay vì trách nhiệm.
- Đặt mã chạy ngay ở cấp module làm import có tác dụng phụ.
- Không khóa phiên bản nên dự án thay đổi theo máy cài.
- Dùng import vòng như giải pháp thay vì sửa kiến trúc.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

9.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

9.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Vẽ đồ thị import của một dự án bốn module.
2. Tìm và phá một chu trình phụ thuộc.
3. Thiết kế API công khai tối thiểu cho module lưu trữ.
4. Tạo môi trường mới và ghi lại quy trình tái tạo.
5. Giải thích vai trò của `if __name__ == '__main__'`.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

9.8 Một module là một tệp Python

Giả sử `stats.py` chứa:

```
def mean(values):
    if not values:
        raise ValueError("Empty values")
    return sum(values) / len(values)

PI = 3.141592653589793
```

Tệp khác có thể dùng:

```
import stats

print(stats.mean([7, 8, 9]))
print(stats.PI)
```

Viết `stats.mean` cho người đọc biết tên đến từ đâu. `from stats import mean` ngắn hơn nhưng có thể va chạm khi nhiều module cùng định nghĩa `mean`. Tránh `from stats import *` vì không rõ những tên nào được đưa vào phạm vi.

9.9 Mã ở mức module được chạy khi import

Python thực thi phần thân module lần đầu được nhập trong tiến trình. Vì thế, không đặt vòng hỏi người dùng, khởi động máy chủ hoặc xóa tệp ở mức trên cùng.

```
def main():
    print("Chay ung dung")

if __name__ == "__main__":
    main()
```

Khi chạy trực tiếp, `__name__` là `"__main__"`; khi import, nó là tên module. Mẫu này cho phép cùng tệp vừa được chạy, vừa được nhập để kiểm thử mà không tự bật giao diện.

9.10 Python tìm module ở đâu?

Trình thông dịch tìm trong các vị trí thuộc `sys.path`: thư mục điểm chạy, thư viện chuẩn, thư mục gói đã cài và các đường dẫn cấu hình. Đặt một tệp `json.py` cạnh chương trình có thể che mô-đun chuẩn `json`. Khi lỗi import kỳ lạ, in `module.__file__` để xem Python đã lấy tệp nào.

Không sửa sys.path như một bản vá đầu tiên

Thêm đường dẫn thủ công có thể làm chương trình chạy trên máy bạn nhưng thất bại nơi khác. Hãy chạy từ gốc dự án, dùng package đúng cấu trúc hoặc cài dự án ở chế độ phát triển. Đường tìm kiếm là một phần triển khai, không nên phụ thuộc vào may

mắn.

9.11 Tổ chức một package

Một cấu trúc nhỏ:

```
study_tracker/
    pyproject.toml
    src/
        study_tracker/
            __init__.py
            models.py
            service.py
            storage.py
            cli.py
    tests/
        test_service.py
```

`models` giữ kiểu dữ liệu, `storage` giữ việc lưu, `service` giữ quy tắc nghiệp vụ, `cli` giao tiếp người dùng. Việc chia theo trách nhiệm giúp thay JSON bằng SQLite mà không viết lại cách tính báo cáo.

Import tương đối bên trong package:

```
from .models import Task
from .storage import JsonStore
```

Tệp `__init__.py` có thể rỗng hoặc công bố một giao diện nhỏ. Đừng nhập mọi thứ vào đó chỉ để đường dẫn ngắn; giao diện công khai nên có chủ đích.

9.12 Môi trường ảo

Hai dự án có thể cần hai phiên bản thư viện khác nhau. Môi trường ảo tạo vùng cài đặt riêng.

```
python -m venv .venv

# Linux/macOS
source .venv/bin/activate

# Windows PowerShell
.venv\Scripts\Activate.ps1
```

Sau khi kích hoạt, `python -m pip install ten-goi` cài vào môi trường. Dùng `python -m pip` bảo đảm pip thuộc đúng trình thông dịch. Không đưa thư mục `.venv` vào Git; hãy ghi cách tái tạo.

9.13 Ghi phụ thuộc có ý nghĩa

`pip freeze > requirements.txt` ghi toàn bộ môi trường, tiện cho ứng dụng nhỏ nhưng gồm cả phụ thuộc gián tiếp. Với dự án đóng gói, `pyproject.toml` mô tả metadata và phụ thuộc trực tiếp.

```
[project]
name = "study-tracker"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = []

[project.optional-dependencies]
dev = ["pytest", "ruff"]
```

Khóa phiên bản tuyệt đối giúp tái lập nhưng có thể giữ lỗi cũ; khoảng phiên bản linh hoạt nhận bản vá nhưng cần kiểm thử cập nhật. Không có một đáp án cho mọi dự án. Điều quan trọng là có quy trình nâng cấp, chạy test và ghi lại môi trường đã phát hành.

9.14 Tách lõi khỏi giao diện

```
# service.py
def complete_task(tasks, task_id):
    for task in tasks:
        if task["id"] == task_id:
            task["done"] = True
            return task
    raise KeyError(task_id)

# cli.py
from .service import complete_task

def handle_complete(tasks):
    task_id = input("ID: ").strip()
    try:
        task = complete_task(tasks, task_id)
    except KeyError:
        print("Khong tim thay cong viec")
    else:
        print(f"Da hoan thanh: {task['title']}")
```

Hàm lõi nhận và trả dữ liệu, không gọi `input`. Nó dễ kiểm thử và có thể được dùng lại cho web. CLI quyết định câu chữ và cách hỏi người dùng.

9.15 Bài tập luyện tập

1. Tách chương trình tính hình học thành module công thức và tệp giao diện.
2. Tạo module có tác dụng phụ khi import, quan sát, rồi sửa bằng `main guard`.
3. Dùng `__file__` xác định nơi một module chuẩn được tải.
4. Thiết kế package `gradebook` gồm model, service, storage, cli.
5. Tạo môi trường ảo mới và ghi tệp hướng dẫn tái tạo từ đầu.
6. Viết `pyproject.toml` tối thiểu cho một package có lệnh dòng lệnh.
7. Tìm một vòng import giữa hai module và sửa bằng cách di chuyển trách nhiệm chung.
8. Vẽ sơ đồ phụ thuộc: module tầng giao diện được phép nhập tầng nào? Tầng lưu trữ có nên nhập CLI không?

9.16 Tóm tắt cuối chương

Bản đồ kiến thức

Module là tệp chứa định nghĩa; package nhóm module thành không gian tên. Mã mức module chạy lúc import nên phải ít tác dụng phụ. `main guard` tách điểm chạy. Môi trường ảo cô lập phụ thuộc, còn cấu trúc dự án tách model, nghiệp vụ, lưu trữ và giao diện để mỗi phần thay đổi với ít ảnh hưởng.

Góc nhìn cá nhân

Chia tệp không tự động tạo thiết kế tốt. Nếu mỗi tệp đều biết mọi thứ, ta chỉ biến một mớ rối lớn thành nhiều mớ rối nhỏ. Tôi chỉ tách một module khi có thể nói một câu rõ ràng về trách nhiệm của nó và hướng phụ thuộc mà mình muốn bảo vệ.

Chương 10

Gỡ lỗi, logging và kiểm thử – Biến niềm tin thành bằng chứng

Giới thiệu ngắn

Khi chương trình sai, phản xạ đầu tiên thường là thêm nhiều `print` rồi thay ngẫu nhiên vài dòng. Cách ấy đôi lúc may mắn, nhưng không tạo kỹ năng có thể lặp lại. Gỡ lỗi là quá trình đặt giả thuyết, thu thập bằng chứng và thu hẹp vùng nghi ngờ. Logging lưu bằng chứng khi chương trình chạy thật; kiểm thử biến một hành vi mong đợi thành phép kiểm tra có thể chạy lại.

Mục tiêu học tập

- Thực hiện quy trình tái hiện–thu hẹp–giả thuyết–kiểm chứng.
- Dùng debugger, breakpoint và biểu thức theo dõi.
- Ghi log theo mức độ, không làm lộ dữ liệu nhạy cảm.
- Viết unit test theo Arrange–Act–Assert.
- Kiểm tra trường hợp biên, ngoại lệ và lỗi từng được sửa.

10.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một lỗi chỉ xuất hiện với dữ liệu ngày cuối tháng. Thêm nhiều `print` có thể làm log rối nhưng chưa chắc cho biết giả định nào sai. Gỡ lỗi có phương pháp bắt đầu bằng tái tạo, thu hẹp, lập giả thuyết, đo và kiểm chứng. Logging ghi bối cảnh; kiểm thử biến lỗi đã sửa thành điều không được phép quay lại.

Một chương trình tốt không bắt đầu bằng việc hỏi “phải dùng câu lệnh nào?”. Nó bắt

đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **gỡ lỗi, logging và kiểm thử như một hệ thống tạo bằng chứng**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

10.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: đầu vào tái tạo, kết quả mong đợi, kết quả quan sát, giả thuyết, điểm đo và mức log.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

10.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$\text{failure} = \text{expected} - \text{observed}, \qquad H_i \xrightarrow{\text{experiment}} \{\text{keep}, \text{reject}\}$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: bước thực thi, dữ liệu liên quan và giả thuyết còn phù hợp?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

10.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Mỗi lần sửa phải đi kèm ca kiểm thử tái hiện lỗi; log phải đủ bối cảnh để nối sự kiện mà không thay đổi hành vi chương trình.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

10.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **gỡ lỗi, logging và kiểm thử như một hệ thống tạo bằng chứng** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

10.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Điều tra báo cáo tuần bị thiếu ngày Chủ nhật. Ta tạo dữ liệu tối thiểu, viết expected trước, đặt điểm dừng tại bước lọc ngày, so sánh trạng thái và phát hiện biên trên bị loại; sau đó thêm regression test.

10.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

10.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

10.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

10.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

10.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

10.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

10.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Sửa dòng đầu tiên có vẻ khả nghi mà chưa tái tạo lỗi.
- Ghi mọi thứ ở mức INFO khiến tín hiệu chìm trong nhiễu.
- Kiểm thử chi tiết cài đặt thay vì hành vi công khai.

- Chỉ kiểm thử đường đi thành công.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

10.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

10.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Viết báo cáo lỗi có bước tái tạo tối thiểu.
2. Thiết kế ma trận kiểm thử cho một hàm có ba điều kiện biên.
3. Chọn mức DEBUG, INFO, WARNING và ERROR cho các sự kiện.
4. Dùng chia đôi để thu hẹp dữ liệu gây lỗi.
5. Biến một lỗi đã gặp thành regression test.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ “gõ cho chạy” thành luyện tư duy thiết kế.

10.8 Tái hiện trước khi sửa

Một báo cáo “thỉnh thoảng bị sai” chưa đủ. Hãy tìm đầu vào nhỏ nhất làm lỗi lặp lại, ghi phiên bản Python, hệ điều hành, câu lệnh chạy và kết quả mong đợi. Nếu không tái hiện, mọi chỉnh sửa chỉ là phỏng đoán.

Quy trình thực dụng:

1. Đóng băng triệu chứng bằng một ví dụ hoặc test thất bại.

2. Xác định lần cuối dữ liệu còn đúng và lần đầu nó sai.
3. Nêu một giả thuyết cụ thể.
4. Thay một yếu tố hoặc quan sát một biến để bác bỏ/ủng hộ.
5. Sửa nguyên nhân, chạy lại test lỗi và toàn bộ test liên quan.

Đừng hỏi "tại sao Python sai?" quá sớm

Hãy hỏi "giả định nào của tôi không đúng?". Phần lớn lỗi là khoảng cách giữa mô hình trong đầu và hành vi thật: danh sách bị chia sẻ, chuỗi chưa chuyển kiểu, múi giờ khác hoặc hàm thay đổi dữ liệu tại chỗ.

10.9 Breakpoint và bước chạy

Hầu hết IDE cho phép đặt breakpoint. Khi chương trình dừng, ta xem call stack, biến cục bộ, bước vào hàm (step into), đi qua dòng (step over) hoặc chạy ra khỏi hàm (step out). Python cũng có `breakpoint()`:

```
def discount(total, rate):
    breakpoint()
    return total * (1 - rate)
```

Không để breakpoint trong bản phát hành. Debugger đặc biệt hữu ích khi trạng thái thay đổi qua vòng lặp; hãy dừng theo điều kiện thay vì bước qua hàng nghìn lần.

10.10 Logging thay cho print trong ứng dụng

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(name)s: %(message)s",
)
logger = logging.getLogger(__name__)

def load_tasks(store):
    logger.info("Dang tai cong viec")
    try:
        tasks = store.load()
    except OSError:
        logger.exception("Tai cong viec that bai")
        raise
    logger.debug("Da tai %d cong viec", len(tasks))
    return tasks
```

Các mức thường dùng: DEBUG cho chi tiết chẩn đoán, INFO cho sự kiện bình thường quan trọng, WARNING cho bất thường có thể tiếp tục, ERROR cho thao tác thất bại, CRITICAL cho hệ thống không thể tiếp tục. `logger.exception` trong khối `except` kèm traceback.

Dùng tham số định dạng của logger thay vì f-string khi log có thể bị tắt; việc nội suy chỉ diễn ra khi cần. Không ghi mật khẩu, token, toàn bộ hồ sơ cá nhân. Log cũng là dữ liệu phải được bảo vệ, xoay vòng và xóa theo chính sách.

10.11 Một test nhỏ

Với `unittest` trong thư viện chuẩn:

```
import unittest

def normalize_title(text):
    return " ".join(text.split()).strip()

class TestNormalizeTitle(unittest.TestCase):
    def test_removes_extra_spaces(self):
        result = normalize_title("  Hoc   Python  ")
        self.assertEqual(result, "Hoc Python")

    def test_empty_text(self):
        self.assertEqual(normalize_title("   "), "")

if __name__ == "__main__":
    unittest.main()
```

Một test có ba phần: Arrange chuẩn bị, Act thực hiện, Assert kiểm tra. Tên test nên mô tả hành vi. Test không chứng minh chương trình không có lỗi; nó cung cấp bằng chứng cho các trường hợp đã được chọn.

10.12 Kiểm tra ngoại lệ và biên

```
class TestMean(unittest.TestCase):
    def test_empty_list_raises(self):
        with self.assertRaisesRegex(ValueError, "it nhat"):
            tinh_trung_binh([])

    def test_one_value(self):
        self.assertEqual(tinh_trung_binh([8]), 8)
```

Các biên thường đáng thử: rỗng, một phần tử, giá trị nhỏ/lớn nhất, trùng lặp, Unicode, ngày chuyển tháng, tệp thiếu, quyền bị từ chối. Đừng chỉ kiểm tra “đường vui”.

10.13 Test double và tiêm phụ thuộc

Hàm gọi thời gian hiện tại hoặc mạng sẽ khó kiểm thử ổn định. Thay vì để hàm tự lấy mọi phụ thuộc, truyền chúng vào.

```
from datetime import date

def overdue(task, today=None):
    today = today or date.today()
    due = date.fromisoformat(task["due"])
    return not task["done"] and due < today

def test_overdue():
    task = {"due": "2026-09-01", "done": False}
    assert overdue(task, today=date(2026, 9, 4)) is True
```

Test không phụ thuộc ngày đang chạy. Cùng nguyên tắc áp dụng kho dữ liệu: truyền một kho giả trong bộ nhớ thay vì ghi tệp thật cho mọi unit test.

10.14 Regression test – giữ lại bài học của lỗi cũ

Khi sửa một lỗi, trước hết viết test tái hiện nó. Test thất bại ở bản cũ, thành công sau sửa và ở lại trong bộ test. Nhờ đó lỗi không lặng lẽ quay lại khi tái cấu trúc.

Test cũng có thể sai

Một test chỉ lặp lại đúng thuật toán của hàm có thể cùng sai. Ưu tiên ví dụ có kết quả biết trước, tính chất bất biến, hoặc một cách tính độc lập. Test phải dễ đọc hơn phần được kiểm tra.

10.15 Bài tập luyện tập

1. Chọn một lỗi cũ của bạn và viết báo cáo tái hiện tối thiểu.
2. Dùng debugger theo dõi một vòng lặp tìm max bị sai với toàn số âm.
3. Thay các `print` chẩn đoán trong ứng dụng bằng logger có mức phù hợp.
4. Viết test cho hàm chuẩn hóa tên gồm Unicode, chuỗi rỗng và khoảng trắng đặc biệt.
5. Viết test cho kho JSON bằng thư mục tạm; không chạm dữ liệu thật.
6. Thiết kế kho trong bộ nhớ để test service quản lý công việc.
7. Tạo một mutation nhỏ, chẳng hạn đổi `<` thành `<=`; bộ test có phát hiện không?
8. Lập ma trận test cho chức năng đăng ký môn: điều kiện tiên quyết, trùng lịch, đủ chỗ,

giới hạn tín chỉ.

Một phiên gỡ lỗi có nhật ký

Chọn chương trình ít nhất 50 dòng. Cố tình cài ba lỗi thuộc ba nhóm: sai kiểu, chia sẻ đối tượng và điều kiện biên. Đổi bài với một người khác hoặc để một ngày sau mới sửa. Ghi từng giả thuyết, bằng chứng và thời điểm thu hẹp được lỗi. Mục tiêu không phải sửa nhanh nhất mà là tạo quy trình có thể kể lại.

10.16 Tóm tắt cuối chương

Bản đồ kiến thức

Gỡ lỗi bắt đầu bằng tái hiện và giả thuyết có thể bác bỏ. Debugger quan sát trạng thái; logging giữ dấu vết lúc chạy thật; test kiểm tra hành vi lặp lại. Bắt đầu với trường hợp biên và lỗi từng xảy ra. Tiêm thời gian, kho dữ liệu hoặc dịch vụ qua tham số giúp kiểm thử ổn định.

Góc nhìn cá nhân

Tôi từng xem lỗi là dấu hiệu mình chưa đủ giỏi. Sau này, tôi xem mỗi lỗi là một thí nghiệm mà chương trình đã làm thay mình: nó đưa ra một kết quả trái dự đoán và yêu cầu mình cập nhật mô hình. Sự khác biệt không nằm ở việc có mắc lỗi hay không, mà ở việc ta có để lại một bằng chứng giúp mình không mắc lại cùng cách hay không.

Phần III

Tư duy thuật toán và cấu trúc dữ liệu

Chương 11

Độ phức tạp – Khi chương trình đúng vẫn có thể chưa đủ tốt

Giới thiệu ngắn

Hai chương trình cùng trả kết quả đúng với mười phần tử. Khi dữ liệu tăng lên một triệu, một chương trình kết thúc trong nháy mắt, chương trình kia cần hàng giờ. Độ phức tạp không đo chính xác số giây; nó mô tả cách lượng công việc tăng khi kích thước đầu vào tăng. Đây là chiếc la bàn giúp ta so sánh thuật toán trước khi mua máy mạnh hơn.

Mục tiêu học tập

- Xác định kích thước đầu vào và phép toán chủ đạo.
- Hiểu trực giác của Big-O, Omega và Theta ở mức thực hành.
- Phân tích vòng lặp nối tiếp, lồng nhau và chia đôi dữ liệu.
- Phân biệt độ phức tạp thời gian, bộ nhớ và chi phí trung bình.
- Đo thực nghiệm mà không thay thế phân tích.

11.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Hai chương trình đều xử lý đúng 100 bản ghi, nhưng khi dữ liệu tăng lên một triệu bản ghi, một chương trình hoàn thành trong giây còn chương trình kia kéo dài hàng giờ. Thời gian trên một máy không phải bản chất; điều cần nghiên cứu là số phép toán và bộ nhớ tăng thế nào khi kích thước đầu vào n tăng.

Một chương trình tốt không bắt đầu bằng việc hỏi “phải dùng câu lệnh nào?”. Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng

nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **độ phức tạp như tốc độ tăng chi phí theo quy mô**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

11.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: kích thước đầu vào n, phép toán chủ đạo, trường hợp tốt/xấu/trung bình, thời gian và bộ nhớ phụ.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

11.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$T(n) = an^2 + bn + c, \qquad T(n) \in \Theta(n^2)$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: số phần tử đã xử lý, cấu trúc phụ đang giữ và số phép toán tích lũy?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

11.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Phân tích phải gắn với một mô hình đầu vào và phép toán chủ đạo; bỏ hằng số không đồng nghĩa bỏ mọi chi phí thực tế.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

11.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **độ phức tạp như tốc độ tăng chi phí theo quy mô** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

11.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

So sánh tìm phần tử trùng bằng hai vòng lặp với cách dùng set. Ta đếm số so sánh khi $n = 5, 10, 20$, suy ra quy luật, rồi mới đo thời gian để xác nhận xu hướng thay vì dùng benchmark đơn lẻ.

11.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

11.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

11.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

11.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

11.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

11.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

11.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Đồng nhất Big-O với thời gian chạy chính xác.
- Bỏ qua chi phí bộ nhớ khi đổi cấu trúc dữ liệu.
- Đánh giá từ một kích thước đầu vào quá nhỏ.
- Chỉ xét trường hợp thuận lợi rồi công bố cho mọi dữ liệu.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

11.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

11.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Đếm phép gán và so sánh của một vòng lặp lồng.
2. Lập bảng tăng trưởng $1, \log n, n, n \log n, n^2$.
3. Phân tích đổi thời gian lấy bộ nhớ trong bài tìm trùng.
4. Thiết kế benchmark có warm-up và nhiều kích thước.
5. Tìm điểm mà thuật toán tuyến tính vượt giải pháp bình phương.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ “gõ cho chạy” thành luyện tư duy thiết kế.

11.8 Ta đang đếm điều gì?

Giả sử tìm một mã trong list có n phần tử. Trường hợp xấu nhất phải so sánh n lần. Ta nói thời gian tăng tuyến tính, $O(n)$. Không quan tâm mỗi phép so sánh mất 30 hay 50 nano giây; ta quan tâm khi n gấp đôi, số phép chính xấp xỉ gấp đôi.

```
def contains(values, target):
    for value in values:
        if value == target:
            return True
    return False
```

Trường hợp tốt nhất là phần tử đầu, nhưng khi lập kế hoạch an toàn thường xét trường hợp xấu nhất. Nếu biết phân phối dữ liệu, chi phí trung bình cũng có ý nghĩa.

11.9 Bỏ hằng số và bậc thấp

Nếu thuật toán thực hiện $3n + 20$ phép, tốc độ tăng vẫn tuyến tính: $O(n)$. Nếu $n^2 + 100n$, thành phần n^2 chi phối khi n lớn: $O(n^2)$. Đây là mô hình tăng trưởng, không phải phép tính thời gian tuyệt đối.

Các lớp thường gặp:

Độ phức tạp	Trực giác	Ví dụ
$O(1)$	không tăng theo n	đọc phần tử list theo chỉ số
$O(\log n)$	mỗi bước loại một phần đáng kể	tìm nhị phân
$O(n)$	đi qua dữ liệu một lần	tính tổng
$O(n \log n)$	chia rồi xử lý các mức	sắp xếp so sánh tốt
$O(n^2)$	xét nhiều cặp	hai vòng lặp toàn phần
$O(2^n)$, $O(n!)$	thử tổ hợp tăng rất nhanh	quay lui không cắt tỉa

11.10 Vòng lặp nối tiếp và lồng nhau

Hai vòng lặp nối tiếp qua n phần tử cho $n + n = 2n$, tức $O(n)$. Hai vòng lồng nhau, mỗi vòng n, tạo khoảng n^2.

```python
def has_duplicate_slow(values):
    for i in range(len(values)):
        for j in range(i + 1, len(values)):
            if values[i] == values[j]:
                return True
    return False
```

Dùng set:

```python
def has_duplicate(values):
    seen = set()
    for value in values:
        if value in seen:
            return True
        seen.add(value)
    return False
```

Phiên bản thứ hai trung bình $O(n)$ thời gian nhưng dùng $O(n)$ bộ nhớ. Đây là đánh đổi không thể bỏ qua. Nếu bộ nhớ cực hạn hoặc phần tử không băm được, lựa chọn có thể khác.

11.11 Chi phí ẩn trong một dòng

Một dòng Python không nhất thiết $O(1)$. `x in list` là $O(n)$; `x in set` trung bình $O(1)$; `list.insert(0, x)` phải dời phần tử nên $O(n)$; `append` có chi phí khấu hao $O(1)$. `sorted`

thường $O(n \log n)$.

Một comprehension vẫn có thể là bậc hai

`[x for x in a if x in b]` là $O(|a||b|)$ nếu `b` là list. Chuyển `b` thành set một lần có thể đưa chi phí trung bình về $O(|a| + |b|)$.

11.12 Không gian và dữ liệu trung gian

```
sum(x * x for x in range(1_000_000))
```

Generator không tạo list một triệu bình phương, nên bộ nhớ bổ sung gần $O(1)$. Nếu viết ngoặc vuông, bộ nhớ là $O(n)$. Cả hai vẫn phải tính n giá trị. Tối ưu bộ nhớ đôi khi làm mã phức tạp hoặc không thể duyệt lại; chọn theo yêu cầu.

11.13 Phân tích chia đôi

Trong tìm nhị phân, mỗi lần so sánh loại một nửa khoảng. Sau k bước còn $n/2^k$. Khi còn 1, $2^k = n$, nên $k = \log_2 n$. Một tỷ phần tử chỉ cần khoảng 30 bước so sánh, với điều kiện dữ liệu đã sắp xếp và truy cập ngẫu nhiên rẻ.

11.14 Đo bằng timeit

```
from timeit import timeit

data = list(range(100_000))
lookup_list = timeit(lambda: 99_999 in data, number=100)

data_set = set(data)
lookup_set = timeit(lambda: 99_999 in data_set, number=100)
print(lookup_list, lookup_set)
```

Đo nhiều lần, cô lập bước cần đo và không tính chi phí chuẩn bị nếu so sánh tra cứu. Nhưng đừng kết luận từ một kích thước; đo nhiều n để nhìn đường tăng. Máy, phiên bản Python và dữ liệu thực ảnh hưởng hằng số.

11.15 Tối ưu đúng chỗ

Thứ tự ưu tiên tốt thường là: đúng, rõ, đo được, rồi nhanh. Đừng thay một hàm dễ hiểu bằng mẹo khó đọc chỉ để tiết kiệm vài micro giây ở đoạn chạy một lần. Ngược lại, vòng lặp bậc hai trong luồng xử lý hàng triệu bản ghi đáng được xem sớm.

Bốn câu hỏi trước khi tối ưu

Đầu vào lớn nhất là bao nhiêu? Đoạn này chạy bao nhiêu lần? Nút thắt thật nằm ở CPU, bộ nhớ, đĩa hay mạng? Mức chậm nào ảnh hưởng người dùng? Không có bối cảnh, "nhanh hơn" chỉ là một tính từ.

11.16 Bài tập luyện tập

1. Phân tích tổng quát hàm tính tổng, hàm tìm mọi cặp, hàm chia đôi dãy.
2. Chỉ ra chi phí của `pop(0)` lặp lại trên list và thay bằng cấu trúc phù hợp.
3. So sánh hai cách tìm phần giao danh sách: vòng lặp với `in list` và set.
4. Đo thời gian trên $n = 10^2, 10^3, 10^4, 10^5$; vẽ hoặc lập bảng tỉ lệ tăng.
5. Phân tích thời gian và bộ nhớ của việc tạo list bình phương với generator.
6. Tìm ba thao tác "một dòng" trong mã của bạn có chi phí không hằng.
7. Một thuật toán $O(n \log n)$ có thể chậm hơn $O(n^2)$ ở đầu vào nhỏ không? Giải thích hằng số và điểm giao.
8. Thiết kế ngưỡng thực dụng để chọn thuật toán theo kích thước, rồi viết benchmark kiểm chứng.

11.17 Tóm tắt cuối chương

Bản đồ kiến thức

Big-O mô tả tốc độ tăng, không phải số giây. Vòng nối tiếp cộng chi phí; vòng lồng nhau thường nhân. Một dòng có thể chứa thao tác tuyến tính hoặc sắp xếp. Set đổi bộ nhớ lấy tra cứu nhanh trung bình; generator giảm dữ liệu trung gian. Phân tích cho hướng đi, benchmark kiểm chứng trên môi trường và dữ liệu thật.

Góc nhìn cá nhân

Độ phức tạp dạy tôi một bài học rộng hơn lập trình: giải pháp tốt ở quy mô nhỏ chưa chắc còn tốt khi trách nhiệm lớn lên. Ta không cần ám ảnh tối ưu mọi dòng, nhưng nên tập nhìn thấy đường cong đang ẩn sau một chương trình có vẻ chạy rất nhanh hôm nay.

Chương 12

Tìm kiếm và sắp xếp – Đặt dữ liệu vào một trật tự có ích

Giới thiệu ngắn

Tìm một quyển sách trên bàn bừa khác với tìm nó trên kệ đã xếp theo mã. Trật tự là công việc phải trả trước để những lần tìm sau nhanh hơn. Chương này không nhằm bắt bạn viết lại mọi thuật toán thư viện; ta tự xây một số thuật toán để thấy bất biến, biên chỉ số và đánh đổi giữa chuẩn bị với truy vấn.

Mục tiêu học tập

- Cài đặt và phân tích tìm tuyến tính, tìm nhị phân.
- Hiểu điều kiện tiên quyết và bất biến của tìm nhị phân.
- Mô phỏng selection sort, insertion sort, merge sort.
- Dùng `sorted`, `key` và tính ổn định trong bài toán thật.
- Chọn giữa sắp xếp trước, dictionary, set và quét một lần.

12.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Tra một mã sách trong kho đã sắp xếp khác căn bản với lật từng cuốn trong một chồng hỗn độn. Sắp xếp tạo chi phí ban đầu nhưng mở ra phép tìm kiếm nhanh và nhiều truy vấn như tìm khoảng. Ta phải xét số lần truy vấn, chi phí cập nhật và tiêu chí ổn định trước khi chọn thuật toán.

Một chương trình tốt không bắt đầu bằng việc hỏi “phải dùng câu lệnh nào?”. Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng

nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **tìm kiếm và sắp xếp như khai thác cấu trúc thứ tự**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

12.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: khóa tìm kiếm, thứ tự, khoảng đang xét, số lần so sánh, tính ổn định và chi phí tiền xử lý.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

12.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$\text{linear} : T(n) \sim n, \qquad \text{binary} : T(n) \sim \log_2 n$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: biên trái/phải của vùng còn khả năng chứa đáp án và phần đã đặt đúng vị trí?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

12.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Tìm kiếm nhị phân chỉ loại bỏ vùng dữ liệu khi thứ tự đã được bảo đảm; sau mỗi bước đáp án, nếu tồn tại, phải còn trong khoảng tìm.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

12.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **tìm kiếm và sắp xếp như khai thác cấu trúc thứ tự** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

12.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Tìm mã sinh viên trong danh sách đã sắp theo mã. Ta truy vết left, mid, right, giải thích vì sao bỏ được nửa khoảng và kiểm tra trường hợp không tồn tại, phần tử đầu, phần tử cuối.

12.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

12.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

12.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

12.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

12.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

12.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

12.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Dùng binary search trên dữ liệu chưa sắp.
- Nhầm biên đóng và biên nửa mở gây lặp vô hạn.
- Sắp xếp chỉ để phục vụ một truy vấn nhỏ.
- Quên tiêu chí ổn định khi nhiều bản ghi cùng khóa.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

12.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

12.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Truy vết binary search trên 15 phần tử.
2. Chứng minh bất biến của khoảng tìm.
3. So sánh insertion sort và merge sort trên dữ liệu gần có thứ tự.
4. Thiết kế khóa sắp xếp nhiều trường.
5. Tìm mọi phần tử trong một khoảng bằng hai mốc nhị phân.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

12.8 Tìm tuyến tính

```
def linear_search(values, target):
    for index, value in enumerate(values):
        if value == target:
            return index
    return -1
```

Thuật toán hoạt động với dữ liệu chưa sắp xếp, trả lần xuất hiện đầu tiên và có chi phí xấu nhất $O(n)$. Giá trị `-1` là quy ước; trong API khác có thể trả `None` hoặc phát sinh lỗi. Hãy ghi hợp đồng.

12.9 Tìm nhị phân và bất biến khoảng

Ta duy trì khoảng nửa mở $[left, right)$: bên trái được lấy, bên phải không lấy. Mục tiêu, nếu tồn tại, luôn nằm trong khoảng ấy.

```python
def binary_search(values, target):
    left = 0
    right = len(values)

    while left < right:
        mid = (left + right) // 2
        if values[mid] < target:
            left = mid + 1
        elif values[mid] > target:
            right = mid
        else:
            return mid
    return -1
```

Nếu phần tử giữa nhỏ hơn mục tiêu, cả nửa trái kể cả `mid` bị loại, nên `left = mid + 1`. Nếu lớn hơn, `mid` cũng không thể là đáp án, và với khoảng nửa mở ta đặt `right = mid`. Mỗi nhánh phải làm khoảng nhỏ đi; nếu không, vòng lặp có thể vô hạn.

Tìm nhị phân trên dữ liệu chưa sắp xếp

Thuật toán vẫn chạy và thậm chí đôi lúc trả đúng do may mắn, khiến lỗi khó thấy. Điều kiện "đã sắp theo cùng khóa dùng để so sánh" là một phần hợp đồng, không phải ghi chú phụ.

Thư viện chuẩn có `bisect` để tìm vị trí chèn, rất hữu ích khi duy trì list đã sắp. Việc chèn vào giữa list vẫn $O(n)$ vì phải dời phần tử, dù tìm vị trí chỉ $O(\log n)$.

12.10 Selection sort – chọn nhỏ nhất từng lượt

```python
def selection_sort(values):
    result = values.copy()
    for i in range(len(result)):
        min_index = i
        for j in range(i + 1, len(result)):
            if result[j] < result[min_index]:
                min_index = j
        result[i], result[min_index] = result[min_index], result[i]
    return result
```

Sau mỗi lượt i, đoạn từ 0 đến i đã chứa các phần tử nhỏ nhất đúng vị trí. Thuật toán $O(n^2)$, ít hoán đổi nhưng không phải lựa chọn chung cho dữ liệu lớn.

12.11 Insertion sort – chèn vào đoạn đã có trật tự

```python
def insertion_sort(values):
    result = values.copy()
    for i in range(1, len(result)):
        current = result[i]
        j = i - 1
        while j >= 0 and result[j] > current:
            result[j + 1] = result[j]
            j -= 1
        result[j + 1] = current
    return result
```

Insertion sort nhanh trên dãy nhỏ hoặc gần có thứ tự; xấu nhất $O(n^2)$. Ý tưởng "đoạn trái đã sắp" là bất biến giúp chứng minh.

12.12 Merge sort – chia, giải, ghép

```python
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def merge_sort(values):
    if len(values) <= 1:
        return values.copy()
    mid = len(values) // 2
    return merge(merge_sort(values[:mid]), merge_sort(values[mid:]))
```

Dãy được chia qua $\log n$ mức; mỗi mức ghép tổng cộng n phần tử, nên $O(n \log n)$. Phiên bản này tạo nhiều list con, dùng thêm bộ nhớ. Đây là bài học về ý tưởng, không thay `sorted` trong sản phẩm.

12.13 Sắp xếp thực tế bằng key

Python dùng Timsort, ổn định và khai thác các đoạn đã có trật tự. Ổn định nghĩa là hai phần tử có khóa bằng nhau giữ thứ tự tương đối.

```python
tasks = [
    {"title": "Doc sach", "priority": 2, "due": "2026-09-05"},
    {"title": "Nop bai", "priority": 1, "due": "2026-09-04"},
    {"title": "Tap code", "priority": 1, "due": "2026-09-06"},
]

ordered = sorted(tasks, key=lambda t: (t["priority"], t["due"]))
```

Tuple khóa so sánh từ trái sang phải. Với nhiều hướng tăng/giảm khác nhau, có thể sắp ổn định nhiều lượt từ khóa phụ đến khóa chính hoặc tạo hàm khóa biến đổi. Đừng đảo chuỗi bằng thủ thuật khó hiểu.

12.14 Có cần sắp xếp không?

Nếu chỉ cần min một lần, quét $O(n)$ tốt hơn sắp $O(n \log n)$. Nếu cần tra cứu nhiều lần theo mã, dictionary có thể phù hợp hơn. Nếu cần liên tục lấy phần tử ưu tiên cao nhất, heap là cấu trúc đúng. Sắp xếp là một công cụ, không phải nghi thức bắt buộc trước mọi phân tích.

Chi phí được trả ở đâu?

Một danh sách 1 triệu phần tử chỉ được tìm một lần: sắp rồi tìm nhị phân có thể tốn hơn quét. Nhưng nếu có 100.000 truy vấn và dữ liệu ít đổi, chi phí sắp ban đầu có thể đáng. Hãy tính cả vòng đời, không chỉ một lời gọi.

12.15 Bài tập luyện tập

1. Theo dấu tay tìm nhị phân trên 15 phần tử, ghi `left`, `mid`, `right` mỗi lượt.
2. Sửa tìm nhị phân để trả vị trí đầu tiên của giá trị trùng.
3. Dùng `bisect` tìm vị trí chèn giữ thứ tự.
4. Minh họa từng lượt selection và insertion sort trên cùng một dãy.
5. Viết merge không dùng cắt lát phần dư trong vòng lặp.
6. Sắp danh sách hồ sơ theo trạng thái chưa xong trước, ưu tiên cao trước, hạn sớm trước.
7. Tìm k giá trị lớn nhất; so sánh sắp toàn bộ với `heapq.nlargest`.
8. Thiết kế benchmark cho dữ liệu ngẫu nhiên, đã sắp và gần sắp; giải thích Timsort.

12.16 Tóm tắt cuối chương

Bản đồ kiến thức

Tìm tuyến tính không cần trật tự; tìm nhị phân cần dữ liệu sắp và thu hẹp khoảng đúng bất biến. Các thuật toán sắp đơn giản giúp thấy vòng lặp và chứng minh; merge sort cho mô hình $n \log n$. Trong mã thật, ưu tiên `sorted` cùng khóa rõ. Trước khi sắp, hỏi liệu quét, set, dictionary hay heap có gần bài toán hơn.

Góc nhìn cá nhân

Trật tự không miễn phí. Ta trả chi phí để tạo nó, rồi nhận lại sự nhanh chóng hoặc khả năng nhìn thấy mẫu. Trong học tập cũng vậy: sắp xếp ghi chú không tự tạo hiểu biết, nhưng một cấu trúc phù hợp giúp ta tìm lại và kết nối điều đã hiểu. Điều quan trọng là biết mình sắp xếp để phục vụ câu hỏi nào.

Chương 13

Stack, queue và heap – Thứ tự phục vụ cũng là một thuật toán

Giới thiệu ngắn

Một chồng đĩa lấy chiếc trên cùng trước. Một hàng người phục vụ người đến trước. Một phòng cấp cứu không theo thứ tự đến hoàn toàn mà theo mức khẩn cấp. Cùng là "giữ nhiều phần tử", nhưng quy tắc lấy ra làm hệ thống hành xử khác hẳn. Stack, queue và priority queue đặt quy tắc ấy vào cấu trúc.

Mục tiêu học tập

- Dùng list làm stack theo LIFO.
- Dùng `deque` làm queue hai đầu hiệu quả.
- Áp dụng stack cho ngoặc, lịch sử và duyệt sâu.
- Áp dụng queue cho xử lý theo lượt và duyệt rộng.
- Dùng heap cho hàng đợi ưu tiên và hiểu khóa hòa.

13.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Trình duyệt quay lại trang gần nhất, quầy vé phục vụ người đến trước, còn phòng cấp cứu ưu tiên ca nghiêm trọng. Cả ba đều giữ một tập phần tử nhưng khác luật lấy ra. Chọn cấu trúc dữ liệu chính là chọn chính sách vận hành, không chỉ chọn tên lớp.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra

quyết định? Với chủ đề **stack, queue và heap như luật chọn phần tử tiếp theo**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

13.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: thứ tự đến, mức ưu tiên, thao tác thêm/lấy, kích thước và điều kiện rỗng.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

13.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$\text{stack} = LIFO, \qquad \text{queue} = FIFO, \qquad \text{heap} = \arg\min \,/\, \arg\max$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: các phần tử đang chờ và phần tử hợp lệ tiếp theo theo chính sách?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

13.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Mỗi thao tác lấy ra phải tuân đúng luật LIFO, FIFO hoặc ưu tiên; không phần tử nào biến mất nếu chưa được xử lý hay hủy có chủ đích.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

13.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **stack, queue và heap như luật chọn phần tử tiếp theo** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

13.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Mô hình hóa hàng đợi tác vụ có mức ưu tiên và thời điểm đến. Ta quy định cách phá hòa, truy vết từng thao tác push/pop và kiểm tra rằng cùng mức ưu tiên vẫn giữ quy tắc đã công bố.

13.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

13.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

13.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

13.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

13.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

13.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

13.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Dùng list với `pop(0)` cho hàng đợi lớn.
- Cho rằng heap tự sắp xếp toàn bộ khi duyệt.
- Không quy định cách phá hòa giữa hai ưu tiên bằng nhau.
- Lấy phần tử khi cấu trúc rỗng mà không có hợp đồng lỗi.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

13.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

13.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Mô phỏng undo/redo bằng hai stack.
2. Truy vết quầy phục vụ bằng deque.
3. Thiết kế hàng đợi ưu tiên cho lịch nộp bài.
4. So sánh chi phí từng thao tác.
5. Xây bộ ca thử gồm rỗng, một phần tử và nhiều mức ưu tiên.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ “gõ cho chạy” thành luyện tư duy thiết kế.

13.8 Stack – vào sau, ra trước

List hỗ trợ `append` và `pop` ở cuối với chi phí khấu hao hằng số.

```
stack = []
stack.append("trang_chu")
stack.append("bai_viet")
stack.append("phan_binh_luan")

trang_truoc = stack.pop()
```

Stack xuất hiện trong undo, lịch sử điều hướng, phân tích biểu thức và call stack của hàm. Phần tử đỉnh là quyết định gần nhất chưa được hoàn tất.

13.9 Kiểm tra ngoặc cân bằng

Mỗi dấu đóng cần khớp dấu mở gần nhất

```
def balanced(text):
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    for char in text:
        if char in pairs.values():
            stack.append(char)
        elif char in pairs:
            if not stack or stack.pop() != pairs[char]:
                return False
    return not stack
```

Nếu gặp dấu đóng, nó phải khớp dấu mở gần nhất, đúng bản chất LIFO. Cuối cùng stack phải rỗng; nếu còn, có dấu mở chưa đóng.

13.10 Queue – vào trước, ra trước

Không dùng `list.pop(0)` lặp lại vì mỗi lần phải dời phần còn lại. `collections.deque` thêm/xóa hai đầu hiệu quả.

```
from collections import deque

queue = deque(["An", "Binh"])
queue.append("Chi")
nguoi_tiep_theo = queue.popleft()
```

Queue phù hợp tác vụ chờ, mô phỏng, truyền thông điệp và BFS. `deque(maxlen=n)` còn tạo bộ đệm vòng tự bỏ phần cũ nhất.

13.11 Mô phỏng quầy phục vụ

```
from collections import deque

def simulate(arrivals, service_per_tick=1):
    waiting = deque()
    served = []
    for tick, people in enumerate(arrivals):
        waiting.extend(people)
        for _ in range(min(service_per_tick, len(waiting))):
            served.append((tick, waiting.popleft()))
    return served, list(waiting)
```

Mô phỏng buộc ta định nghĩa rõ thứ tự trong một "tick": người đến được xếp trước hay sau khi phục vụ? Hai lựa chọn cho kết quả khác. Thuật toán không chỉ là code; nó là quy ước thời gian.

13.12 Priority queue bằng heap

Heap bảo đảm phần tử nhỏ nhất ở đầu; `heappush` và `heappop` có chi phí $O(\log n)$.

```
import heapq
from itertools import count

pq = []
sequence = count()

def add_task(priority, title):
    heapq.heappush(pq, (priority, next(sequence), title))

add_task(2, "Doc sach")
add_task(1, "Nop bai")
add_task(1, "Gui email")

while pq:
    priority, _, title = heapq.heappop(pq)
    print(priority, title)
```

Tuple so sánh lần lượt. Bộ đếm giữ thứ tự đến khi cùng ưu tiên và ngăn Python phải so sánh hai đối tượng tác vụ không có thứ tự. Để ưu tiên số lớn trước, có thể lưu số âm.

13.13 Hai stack cho undo/redo

Mỗi thao tác mới đi vào stack undo và xóa stack redo. Undo lấy thao tác gần nhất, thực hiện nghịch đảo rồi đẩy sang redo. Redo làm chiều ngược.

```
class History:
    def __init__(self):
        self.undo_stack = []
        self.redo_stack = []

    def record(self, action):
        self.undo_stack.append(action)
        self.redo_stack.clear()

    def undo(self):
        if not self.undo_stack:
```

```
            return None
        action = self.undo_stack.pop()
        self.redo_stack.append(action)
        return action
```

Trong hệ thống thật, mỗi action cần biết cách làm và hoàn tác; không phải hành động nào cũng đảo được. Ghi email không thể "thu hồi" chỉ bằng xóa lịch sử. Thiết kế undo phải gắn với tác dụng phụ.

13.14 Chọn cấu trúc theo quy tắc lấy ra

Cấu trúc	Phần tử được lấy	Tình huống
Stack	mới nhất	backtracking, undo, biểu thức
Queue	cũ nhất	hàng chờ, BFS, tác vụ theo lượt
Deque	hai đầu	cửa sổ trượt, buffer
Heap	khóa ưu tiên nhỏ/lớn nhất	lập lịch, top-k

Cấu trúc dữ liệu giữ một chính sách

Đừng chọn list rồi tự nhớ phải lấy đầu hay cuối ở từng nơi. Đặt chính sách trong một lớp hoặc dùng kiểu thể hiện trực tiếp. Khi quy tắc nằm trong cấu trúc, mã gọi khó vi phạm hơn.

13.15 Bài tập luyện tập

1. Dùng stack đổi số thập phân sang nhị phân.
2. Tính giá trị biểu thức hậu tố gồm số và bốn phép toán.
3. Mở rộng kiểm tra ngoặc để trả vị trí lỗi đầu tiên.
4. Mô phỏng máy in có queue và thời gian in theo số trang.
5. Dùng `deque(maxlen=5)` giữ năm số đo gần nhất và tính trung bình.
6. Dùng heap ghép k dãy đã sắp xếp.
7. Xây bộ lập lịch tác vụ theo ưu tiên, hạn và thứ tự đến.
8. Thiết kế undo/redo cho trình soạn thảo chỉ có chèn và xóa chuỗi; mỗi action phải mang đủ dữ liệu nghịch đảo.

13.16 Tóm tắt cuối chương

Bản đồ kiến thức

Stack phục vụ phần tử mới nhất; queue phục vụ phần tử cũ nhất; heap phục vụ theo ưu tiên. List phù hợp stack ở cuối, `deque` phù hợp queue, `heapq` phù hợp priority queue. Nhiều thuật toán trở nên tự nhiên khi cấu trúc giữ đúng chính sách lấy phần tử.

Góc nhìn cá nhân

Có những bài toán không khó ở phép tính mà khó ở việc ai được xử lý trước. Khi chính sách thứ tự chưa rõ, chương trình có thể "đúng" từng dòng nhưng bất công hoặc bất ổn ở cấp hệ thống. Stack, queue và heap nhắc tôi rằng cấu trúc kỹ thuật luôn mang theo một quyết định về ưu tiên.

Chương 14

Đệ quy và quay lui – Giải bài toán bằng phiên bản nhỏ hơn của chính nó

Giới thiệu ngắn

Đứng giữa hai chiếc gương, ta thấy một hình chứa hình nhỏ hơn. Đệ quy cũng vậy: một hàm giải bài toán bằng cách gọi chính nó với bài toán nhỏ hơn. Vẻ đẹp này dễ biến thành vòng vô hạn nếu không có điểm dừng và thước đo tiến gần điểm dừng. Quay lui dùng đệ quy để thử lựa chọn, phát hiện đường cụt rồi hoàn tác.

Mục tiêu học tập

- Xác định trường hợp cơ sở và bước thu nhỏ.
- Theo dấu call stack và giá trị trả về.
- So sánh đệ quy với vòng lặp về độ rõ, thời gian và bộ nhớ.
- Viết quay lui theo mẫu chọn–khám phá–bỏ chọn.
- Thêm cắt tỉa và memoization khi có bài toán con lặp lại.

14.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Để tìm đường trong mê cung, tại mỗi ngã rẽ ta thử một hướng, ghi dấu, rồi quay lại nếu gặp ngõ cụt. Đệ quy mô tả bài toán bằng phiên bản nhỏ hơn; quay lui tổ chức việc thử–kiểm tra–hoàn tác. Nếu thiếu trường hợp cơ sở hoặc không thu nhỏ bài toán, lời gọi không thể dừng.

Một chương trình tốt không bắt đầu bằng việc hỏi “phải dùng câu lệnh nào?”. Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào,

đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **đệ quy và quay lui như cây của những bài toán nhỏ hơn**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

14.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: kích thước bài toán, trường hợp cơ sở, lựa chọn, ràng buộc, trạng thái tạm và thao tác hoàn tác.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

14.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$T(n) = T(n-1) + g(n), \qquad \text{solution} = \bigcup_{c \in C} \text{search}(c)$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: đường đi hiện tại, lựa chọn đã thử và miền ứng viên còn lại?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

14.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không

còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Mỗi lời gọi phải tiến gần trường hợp cơ sở; sau khi quay lui, trạng thái phải trở lại đúng như trước khi thử lựa chọn.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

14.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **đệ quy và quay lui như cây của những bài toán nhỏ hơn** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

14.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Giải mê cung nhỏ 4×4. Ta vẽ cây lựa chọn, đánh dấu ô đã đi, loại nhánh đụng tường hoặc lặp, và kiểm chứng thao tác bỏ đánh dấu chỉ khi bài toán cho phép thăm lại theo đường khác.

14.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

14.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

14.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

14.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

14.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

14.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

14.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Viết đệ quy mà kích thước không giảm.
- Quên hoàn tác trạng thái dùng chung.
- Sinh mọi khả năng rồi mới kiểm tra ràng buộc.
- Dùng đệ quy sâu khi mô hình lặp rõ và giới hạn stack nhỏ.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

14.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

14.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Truy vết factorial và Fibonacci bằng cây lời gọi.
2. Viết recurrence cho chi phí.
3. Giải bài sinh chuỗi nhị phân không có hai số 1 liền nhau.
4. Thiết kế quay lui xếp lịch với ràng buộc.
5. Chuyển một DFS đệ quy thành phiên bản dùng stack.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ “gõ cho chạy” thành luyện tư duy thiết kế.

14.8 Hai điều kiện để đệ quy kết thúc

```
def factorial(n):
    if n < 0:
        raise ValueError("n phai khong am")
    if n <= 1:
        return 1
    return n * factorial(n - 1)
```

Trường hợp cơ sở trả trực tiếp. Bước đệ quy biến n thành $n-1$, tiến gần cơ sở. Nếu gọi `factorial(n+1)`, ta đi xa điểm dừng. Mỗi thiết kế cần một đại lượng giảm hoặc tiến đơn điệu.

Với 4!, các lời gọi đi xuống 4, 3, 2, 1; sau đó giá trị đi lên: 1, 2, 6, 24. Hiểu hai pha này giúp đọc thuật toán cây và backtracking.

14.9 Call stack và giới hạn

Mỗi lời gọi giữ tham số, biến cục bộ và vị trí quay về. Đệ quy sâu dùng nhiều bộ nhớ và Python đặt giới hạn để tránh làm tràn stack. Vì Python không tối ưu tail recursion, đếm đến một triệu bằng đệ quy là lựa chọn sai; vòng lặp rõ và an toàn.

```
def factorial_loop(n):
    result = 1
    for value in range(2, n + 1):
        result *= value
    return result
```

Đệ quy phù hợp khi dữ liệu tự có cấu trúc đệ quy: cây, thư mục, biểu thức lồng nhau, lựa chọn. Không nên dùng chỉ để chứng minh có thể.

14.10 Fibonacci và bài toán con lặp lại

```
def fib_slow(n):
    if n <= 1:
        return n
    return fib_slow(n - 1) + fib_slow(n - 2)
```

`fib_slow(40)` tính cùng `fib(30)` rất nhiều lần, dẫn đến thời gian gần lũy thừa. Memoization lưu kết quả:

```
from functools import cache

@cache
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)
```

Giờ mỗi n được tính một lần, thời gian $O(n)$, bộ nhớ $O(n)$. Cache yêu cầu đối số băm được và có thể tăng bộ nhớ; hàm cần thuần khiết theo đối số để kết quả lưu còn đúng.

14.11 Quay lui: chọn, đi tiếp, hoàn tác

Sinh mọi chuỗi nhị phân độ dài n:

```
def binary_strings(n):
    result = []
    path = []

    def backtrack():
```

```
        if len(path) == n:
            result.append("".join(path))
            return
        for choice in "01":
            path.append(choice)   # chon
            backtrack()           # kham pha
            path.pop()            # bo chon

    backtrack()
    return result
```

`path` là trạng thái dùng chung. Phải `pop` sau lời gọi để nhánh sau bắt đầu từ trạng thái đúng. Khi lưu kết quả là list, cần `path.copy()` thay vì lưu chính path.

14.12 Hoán vị và tập lựa chọn

```
def permutations(values):
    result = []
    path = []
    used = [False] * len(values)

    def backtrack():
        if len(path) == len(values):
            result.append(path.copy())
            return
        for i, value in enumerate(values):
            if used[i]:
                continue
            used[i] = True
            path.append(value)
            backtrack()
            path.pop()
            used[i] = False

    backtrack()
    return result
```

Có $n!$ hoán vị, nên không thuật toán nào sinh tất cả mà lại nhanh cho n lớn. Tối ưu code không thắng được kích thước đầu ra.

14.13 Cắt tỉa trên bài toán tổng

Tìm các tổ hợp số dương có tổng bằng mục tiêu:

```
def combinations_sum(values, target):
```

```
    values = sorted(v for v in values if v > 0)
    result = []

    def backtrack(start, remaining, path):
        if remaining == 0:
            result.append(path.copy())
            return
        for i in range(start, len(values)):
            value = values[i]
            if value > remaining:
                break
            path.append(value)
            backtrack(i + 1, remaining - value, path)
            path.pop()

    backtrack(0, target, [])
    return result
```

Sắp xếp cho phép dừng vòng khi giá trị vượt phần còn lại. Tham số `start` ngăn dùng lại phần tử và tránh hoán vị của cùng tổ hợp. Cắt tỉa phải dựa trên một mệnh đề đúng; cắt nhầm làm mất lời giải.

14.14 Đệ quy trên cấu trúc lồng nhau

```
def flatten(value):
    if not isinstance(value, list):
        return [value]
    result = []
    for item in value:
        result.extend(flatten(item))
    return result
```

Độ sâu dữ liệu có thể không đáng tin cậy; với đầu vào từ người dùng, cần giới hạn để tránh cạn stack hoặc từ chối cấu trúc quá lớn.

Bốn câu hỏi của đệ quy

Trường hợp nhỏ nhất là gì? Mỗi lời gọi làm bài toán nhỏ đi theo thước đo nào? Giá trị từ lời gọi con được kết hợp ra sao? Có trạng thái nào phải hoàn tác sau khi quay về không?

14.15 Bài tập luyện tập

1. Theo dấu đệ quy tính tổng list; xử lý list rỗng.

2. Viết tìm kiếm nhị phân đệ quy với khoảng nửa mở.
3. Tính tổng mọi số trong cấu trúc list lồng bất kỳ.
4. Sinh mọi tập con của một danh sách; giải thích vì sao có 2^n kết quả.
5. Sinh chuỗi ngoặc cân bằng với n cặp và cắt tỉa tiền tố sai.
6. Giải mê cung nhỏ bằng backtracking, đánh dấu rồi bỏ đánh dấu khi cần.
7. Viết đổi tiền tối thiểu bằng đệ quy chậm, memoization và quy hoạch động; so sánh.
8. Bài toán tám hậu: mô tả trạng thái, lựa chọn, ràng buộc và cắt tỉa trước khi viết code.

14.16 Tóm tắt cuối chương

Bản đồ kiến thức

Đệ quy cần trường hợp cơ sở và bước tiến chắc chắn về cơ sở. Call stack mang chi phí, nên vòng lặp tốt hơn cho lặp tuyến tính sâu. Memoization tránh bài toán con lặp lại. Quay lui theo nhịp chọn–khám phá–bỏ chọn; cắt tỉa giảm nhánh khi có chứng minh rằng nhánh không thể thành công.

Góc nhìn cá nhân

Đệ quy đẹp vì nó khiến một vấn đề lớn chịu nhường chỗ cho một phiên bản nhỏ hơn. Nhưng vẻ đẹp chỉ đứng vững khi điểm dừng rõ. Tôi thấy điều ấy cũng đúng với việc học: một mục tiêu mơ hồ không nhỏ dần thì trở thành vòng lặp; một mục tiêu được chia đến hành động hôm nay thì bắt đầu có đường quay về.

Chương 15

Cây và đồ thị – Khi dữ liệu trở thành những mối quan hệ

Giới thiệu ngắn

Danh sách mô tả thứ tự, dictionary mô tả ánh xạ. Nhưng bản đồ đường đi, mạng bạn bè, điều kiện tiên quyết và cây thư mục được tạo bởi các mối quan hệ. Cây là đồ thị có cấu trúc phân cấp đặc biệt; đồ thị là tập đỉnh cùng các cạnh nối. Chương này đặt nền cho tìm đường, lập lịch và nhiều mô hình trong AI.

Mục tiêu học tập

- Biểu diễn cây và đồ thị bằng cấu trúc Python.
- Duyệt sâu (DFS) và duyệt rộng (BFS).
- Tìm đường, thành phần liên thông và phát hiện chu trình.
- Hiểu đồ thị có hướng, vô hướng, có trọng số.
- Áp dụng sắp xếp topo cho quan hệ phụ thuộc.

15.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Sơ đồ thư mục, cây gia phả, mạng đường và quan hệ theo dõi đều không thể hiện tốt bằng một dãy phẳng. Nút đại diện thực thể; cạnh đại diện quan hệ. Cùng dữ liệu có thể dùng ma trận kề hoặc danh sách kề tùy độ thưa và loại truy vấn.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người

dùng ra quyết định? Với chủ đề **cây và đồ thị như mô hình của quan hệ**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

15.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: tập đỉnh, tập cạnh, hướng, trọng số, thành phần liên thông, đỉnh đã thăm và frontier.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

15.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$G = (V, E), \qquad \deg(v) = |\{u : (u, v) \in E\}|$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: tập đã thăm, hàng đợi/stack biên và cây cha của quá trình duyệt?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

15.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Mỗi đỉnh chỉ được đánh dấu theo đúng thời điểm đã chọn; frontier chứa đúng những đỉnh đã phát hiện nhưng chưa xử lý.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

15.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **cây và đồ thị như mô hình của quan hệ** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

15.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Tìm đường ngắn nhất theo số chặng giữa các phòng học. Ta dựng danh sách kề, chạy BFS từng lớp khoảng cách, ghi cha khi phát hiện và dựng lại đường đi; sau đó phân biệt với bài có trọng số cần thuật toán khác.

15.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

15.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

15.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

15.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

15.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

15.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

15.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Quên đánh dấu đã thăm gây lặp trên chu trình.
- Dùng BFS cho trọng số bất kỳ.
- Nhầm cây gốc với đồ thị không hướng tổng quát.
- Chọn ma trận kề cho đồ thị rất thưa mà không xét bộ nhớ.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

15.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

15.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Biểu diễn sơ đồ thư mục bằng cây.
2. Truy vết BFS và DFS trên cùng đồ thị.
3. Tìm thành phần liên thông.
4. Thiết kế ca thử có đỉnh cô lập và chu trình.
5. So sánh ma trận kề với danh sách kề bằng công thức bộ nhớ.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ “gõ cho chạy” thành luyện tư duy thiết kế.

15.8 Cây: mỗi nút nhìn xuống các cây con

```
class Node:
    def __init__(self, value, children=None):
        self.value = value
        self.children = list(children or [])

root = Node("Khoa hoc", [
    Node("Python", [Node("Co ban"), Node("Nang cao")]),
    Node("Toan", [Node("Giai tich"), Node("Dai so")]),
])
```

Một nút có giá trị và danh sách con. Nút không con là lá. Độ sâu là khoảng cách từ gốc; chiều cao là đường dài nhất xuống lá. Cấu trúc tự lặp lại nên đệ quy tự nhiên.

```
def print_tree(node, depth=0):
    print("  " * depth + "- " + node.value)
```

```
    for child in node.children:
        print_tree(child, depth + 1)
```

Nếu dữ liệu có thể chứa chu trình hoặc sâu tùy ý, đây không còn là cây an toàn; cần tập `visited` hoặc duyệt lặp.

15.9 Biểu diễn đồ thị bằng danh sách kề

```
graph = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "D"],
    "D": ["B", "C"],
}
```

Với đồ thị vô hướng, mỗi cạnh xuất hiện hai chiều. Với đồ thị có hướng, chỉ ghi chiều đi. Set thay list nếu cần tránh cạnh trùng và kiểm tra kề nhanh; list giữ thứ tự duyệt ổn định.

15.10 DFS bằng stack

```
def dfs(graph, start):
    visited = set()
    stack = [start]
    order = []

    while stack:
        node = stack.pop()
        if node in visited:
            continue
        visited.add(node)
        order.append(node)
        stack.extend(reversed(graph.get(node, [])))
    return order
```

`visited` ngăn lặp vô hạn. Đảo hàng xóm để phần tử đầu trong danh sách được xử lý trước vì stack lấy cuối. DFS đi sâu một nhánh, phù hợp phát hiện chu trình, thành phần, backtracking.

15.11 BFS bằng queue và đường ngắn nhất không trọng số

```
from collections import deque

```

```
def shortest_path(graph, start, goal):
    queue = deque([start])
    parent = {start: None}

    while queue:
        node = queue.popleft()
        if node == goal:
            break
        for neighbor in graph.get(node, []):
            if neighbor not in parent:
                parent[neighbor] = node
                queue.append(neighbor)
    else:
        return None

    path = []
    current = goal
    while current is not None:
        path.append(current)
        current = parent[current]
    return path[::-1]
```

BFS thăm theo từng lớp khoảng cách, nên lần đầu gặp đích là số cạnh ít nhất trong đồ thị không trọng số. Dictionary `parent` vừa đánh dấu đã thấy vừa cho đường quay lại.

15.12 Thành phần liên thông

Trong đồ thị vô hướng, một thành phần là nhóm đỉnh có đường nối lẫn nhau. Ta khởi động DFS từ mỗi đỉnh chưa thăm.

```
def components(graph):
    seen = set()
    result = []
    for start in graph:
        if start in seen:
            continue
        component = set(dfs(graph, start))
        seen.update(component)
        result.append(component)
    return result
```

Ứng dụng gồm cụm thiết bị kết nối, nhóm tài khoản liên quan, đảo trên bản đồ lưới.

15.13 Đồ thị có trọng số và Dijkstra

Khi mỗi cạnh có chi phí không âm, BFS không đủ. Dijkstra luôn mở rộng đỉnh có khoảng cách tạm nhỏ nhất bằng heap.

```python
import heapq

def dijkstra(graph, start):
    distance = {start: 0}
    parent = {start: None}
    heap = [(0, start)]

    while heap:
        dist, node = heapq.heappop(heap)
        if dist != distance[node]:
            continue
        for neighbor, weight in graph.get(node, []):
            new_dist = dist + weight
            if new_dist < distance.get(neighbor, float("inf")):
                distance[neighbor] = new_dist
                parent[neighbor] = node
                heapq.heappush(heap, (new_dist, neighbor))
    return distance, parent
```

Điều kiện trọng số không âm là bắt buộc. Phần tử heap cũ không bị xóa khi có đường tốt hơn; ta bỏ qua nếu khoảng cách không còn khớp.

15.14 Phụ thuộc và sắp xếp topo

Các học phần tạo đồ thị có hướng: cạnh $A \to B$ nghĩa là A phải trước B. Nếu có chu trình, kế hoạch không thể hoàn thành.

```python
from collections import deque

def topological_sort(graph):
    indegree = {node: 0 for node in graph}
    for node, neighbors in graph.items():
        for neighbor in neighbors:
            indegree.setdefault(neighbor, 0)
            indegree[neighbor] += 1

    queue = deque(node for node, deg in indegree.items() if deg == 0)
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
```

```
        for neighbor in graph.get(node, []):
            indegree[neighbor] -= 1
            if indegree[neighbor] == 0:
                queue.append(neighbor)

    if len(order) != len(indegree):
        raise ValueError("Do thi co chu trinh")
    return order
```

Có thể có nhiều thứ tự hợp lệ. Thuật toán Kahn lần lượt lấy nút không còn điều kiện trước.

Tên cạnh phải được định nghĩa bằng lời

Trong đồ thị phụ thuộc, $A \to B$ có thể được hiểu "A phụ thuộc B" hoặc "A mở khóa B". Hai quy ước đảo nhau. Viết một câu định nghĩa và kiểm tra bằng ví dụ hai nút trước khi xây thuật toán.

15.15 Bài tập luyện tập

1. Tính số nút, số lá và chiều cao một cây.
2. Biểu diễn hệ thống thư mục và in đường dẫn đầy đủ đến mỗi tệp.
3. Dùng BFS tìm số bước ít nhất trên lưới có vật cản.
4. Tìm mọi thành phần liên thông trong ma trận 0–1.
5. Phát hiện chu trình trong đồ thị có hướng bằng ba màu DFS.
6. Dùng Dijkstra cho bản đồ nhỏ; phục hồi đường chứ không chỉ khoảng cách.
7. Lập thứ tự học từ danh sách môn tiên quyết và báo chu trình cụ thể.
8. Mô hình mạng cộng tác; tính bậc mỗi người và khoảng cách kết nối giữa hai người.

Bản đồ phụ thuộc cho kế hoạch học

Mỗi chủ đề có danh sách kiến thức trước. Hãy đọc cấu trúc từ JSON, kiểm tra khóa thiếu, phát hiện chu trình, rồi sinh một hoặc nhiều thứ tự học hợp lệ. Sau đó thêm thời lượng để ước lượng đường găng đơn giản.

15.16 Tóm tắt cuối chương

Bản đồ kiến thức

Cây mô tả phân cấp; đồ thị mô tả quan hệ tổng quát. Danh sách kề tiết kiệm cho đồ thị thưa. DFS dùng stack và đi sâu; BFS dùng queue, tìm đường ít cạnh. Dijkstra xử lý trọng số không âm; topo sắp phụ thuộc có hướng không chu trình. Tập visited là hàng rào chống lặp.

Góc nhìn cá nhân

Khi nhìn dữ liệu như đồ thị, tôi thôi hỏi một đối tượng "là gì" và bắt đầu hỏi nó "liên quan đến ai". Nhiều vấn đề tưởng thuộc cá nhân hóa ra nằm trong cấu trúc quan hệ: một môn khó vì thiếu nút tiên quyết, một dự án chậm vì đường phụ thuộc, một ý tưởng mới vì nối hai vùng trước đó tách rời.

Phần IV

Nhìn chương trình như một thế giới đối tượng

Chương 16

Lớp và đối tượng – Gom trạng thái cùng hành vi

Giới thiệu ngắn

Dictionary biểu diễn một công việc khá tốt. Nhưng khi mọi nơi đều tự sửa `task["done"]`, quy tắc "không hoàn thành hai lần", "ghi thời điểm hoàn thành" hoặc "tiêu đề không rỗng" dễ bị bỏ quên. Lớp cho phép tạo một kiểu dữ liệu mới, đặt trạng thái và các hành vi hợp lệ cạnh nhau. Hướng đối tượng không phải biến mọi danh từ thành lớp; nó là công cụ quản lý trách nhiệm và bất biến.

Mục tiêu học tập

- Phân biệt lớp, đối tượng, thuộc tính và phương thức.
- Xây dựng đối tượng hợp lệ bằng `__init__`.
- Hiểu `self`, thuộc tính instance và class.
- Thiết kế phương thức thay vì để mã ngoài sửa trạng thái tùy ý.
- Nhận biết khi hàm và kiểu có sẵn là đủ.

16.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một tài khoản học tập có tên, mục tiêu, số phút đã học và các hành vi ghi phiên học, tính tiến độ. Nếu dữ liệu nằm ở nhiều dictionary còn hàm nằm rải rác, rất khó bảo đảm mọi cập nhật tuân quy tắc. Lớp tạo một kiểu mới để tập trung trạng thái và những phép biến đổi hợp lệ.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu

bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **lớp và đối tượng như mô hình gắn trạng thái với hành vi**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

16.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: thuộc tính, phương thức, identity, vòng đời, bất biến và API công khai.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

16.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$O = (S, M), \qquad S_{t+1} = M_i(S_t, a)$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: giá trị thuộc tính của từng instance và quan hệ giữa các đối tượng?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

16.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không

còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Mọi instance hợp lệ phải thỏa bất biến của lớp; phương thức công khai không được để đối tượng ở trạng thái nửa cập nhật.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

16.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **lớp và đối tượng như mô hình gắn trạng thái với hành vi** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

16.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Thiết kế lớp `StudyGoal`. Ta xác định trách nhiệm, constructor, phương thức thêm phút, tính tỉ lệ và từ chối thời lượng âm; giao diện chỉ dùng API thay vì sửa thuộc tính tùy ý.

16.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

16.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

16.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

16.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

16.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

16.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

16.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Tạo lớp chỉ để thay dictionary mà không có hành vi.
- Đưa quá nhiều trách nhiệm vào một God object.
- Nhầm thuộc tính lớp với thuộc tính instance.
- Để constructor thực hiện I/O hoặc công việc khó thất bại an toàn.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

16.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

16.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Viết thẻ CRC cho lớp quản lý mục tiêu học.
2. Xác định bất biến và API tối thiểu.
3. Vẽ ba instance với state khác nhau.
4. Tách trách nhiệm lưu tệp khỏi đối tượng miền.
5. Thiết kế kiểm thử chuyển trạng thái.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ “gõ cho chạy” thành luyện tư duy thiết kế.

16.8 Lớp là bản mô tả, đối tượng là một trường hợp cụ thể

```
class Task:
    def __init__(self, task_id, title, priority=2):
        title = title.strip()
        if not title:
            raise ValueError("Tieu de khong duoc rong")
        if priority not in {1, 2, 3}:
            raise ValueError("Priority phai la 1, 2 hoac 3")

        self.task_id = task_id
        self.title = title
        self.priority = priority
        self.done = False

```

```
    def complete(self):
        self.done = True
```

`Task` là lớp. `Task("T01", "Hoc Python")` gọi lớp để tạo instance. `self` là đối tượng đang nhận lời gọi; Python truyền tự động trong `task.complete()`.

```
task = Task("T01", "Hoc Python", priority=1)
task.complete()
print(task.title, task.done)
```

16.9 Trạng thái hợp lệ ngay từ khi sinh ra

Constructor nên tạo đối tượng dùng được. Nếu tiêu đề rỗng là không hợp lệ, đừng tạo rồi chờ nơi khác sửa. Tuy nhiên, tránh làm I/O nặng, gọi mạng hoặc hỏi `input` trong `__init__`; việc tạo đối tượng nên dễ dự đoán và kiểm thử.

Khi có nhiều cách tạo, class method có thể làm constructor thay thế:

```
class Task:
    # __init__ nhu tren

    @classmethod
    def from_dict(cls, data):
        task = cls(data["id"], data["title"], data.get("priority", 2))
        task.done = bool(data.get("done", False))
        return task
```

`cls` là lớp nhận lời gọi, giúp phương thức hoạt động đúng cả với lớp con.

16.10 Phương thức instance, class và static

Phương thức instance cần trạng thái một đối tượng và nhận `self`. Class method liên quan việc tạo hoặc cấu hình toàn lớp, nhận `cls`. Static method chỉ là hàm đặt trong không gian tên lớp; nó không nhận tự động đối tượng hay lớp.

```
class Task:
    default_priority = 2

    @staticmethod
    def valid_priority(value):
        return value in {1, 2, 3}
```

Nếu static method không liên quan mạnh đến khái niệm lớp, đặt nó làm hàm module thường đơn giản hơn. Đừng dùng decorator chỉ để chứng minh đã học.

16.11 Thuộc tính lớp và cạm bẫy dữ liệu mutable

```
class Classroom:
    students = []  # Sai neu moi lop can danh sach rieng

    def __init__(self, name):
        self.name = name
```

`students` thuộc lớp và bị chia sẻ giữa mọi instance. Danh sách riêng phải tạo trong `__init__`:

```
class Classroom:
    def __init__(self, name):
        self.name = name
        self.students = []
```

Thuộc tính lớp phù hợp hằng số chung hoặc bộ đếm có chủ đích. Khi gán `self.x`, instance có thuộc tính riêng che tên ở lớp.

16.12 Đối tượng hợp tác với nhau

```
class TaskList:
    def __init__(self):
        self._tasks = []

    def add(self, task):
        if any(x.task_id == task.task_id for x in self._tasks):
            raise ValueError("ID da ton tai")
        self._tasks.append(task)

    def pending(self):
        return [task for task in self._tasks if not task.done]

    def find(self, task_id):
        for task in self._tasks:
            if task.task_id == task_id:
                return task
        raise KeyError(task_id)
```

`TaskList` giữ tập hợp và quy tắc duy nhất mã. `Task` giữ quy tắc một công việc. Đặt trách nhiệm theo nơi có đủ dữ liệu để bảo vệ nó.

16.13 Danh tính và bằng nhau

Mặc định, hai instance khác nhau không bằng nhau dù thuộc tính giống. Toán tử `is` hỏi có cùng đối tượng; `==` hỏi có bằng theo quy tắc. Ta có thể định nghĩa `__eq__`, hoặc dùng dataclass ở chương 19.

```
a = Task("T01", "Hoc")
b = Task("T01", "Hoc")
print(a is b)  # False
print(a == b)  # False neu chua dinh nghia __eq__
```

Danh tính miền bài toán cũng cần rõ. Hai Task cùng ID là cùng công việc hay xung đột dữ liệu? Quyết định đó không tự đến từ Python.

16.14 Khi nào không cần lớp?

Một hàm tính toán thuần trên vài giá trị không cần bọc lớp. Một dictionary cấu hình nhỏ chỉ được đọc có thể đủ. Dấu hiệu lớp hữu ích:

- Có trạng thái sống qua nhiều thao tác.
- Có bất biến cần bảo vệ tại một chỗ.
- Nhiều hành vi cùng làm việc trên một nhóm dữ liệu.
- Cần nhiều instance độc lập của cùng khái niệm.

Lớp "Manager" biết tất cả

Một lớp có hàng chục phương thức đọc tệp, gọi API, tính điểm, in giao diện thường đang gom theo sự tiện tay chứ không theo trách nhiệm. Hãy tìm những phần thay đổi vì lý do khác nhau và tách bằng hợp thành.

16.15 Bài tập luyện tập

1. Tạo lớp `BankAccount` có gửi, rút và ngăn số dư âm.
2. Tạo lớp `Rectangle` có diện tích, chu vi và kiểm tra kích thước dương.
3. Tạo `Student` và `GradeBook`; quy tắc mã duy nhất thuộc lớp nào?
4. Thêm thời điểm hoàn thành cho Task và ngăn complete lần hai, hoặc giải thích vì sao cho phép idempotent.
5. Viết `to_dict`/`from_dict` và kiểm thử round-trip.
6. Tìm một lớp đang chỉ chứa static method; chuyển thành module hàm và so sánh.
7. Thiết kế lớp lịch hẹn phát hiện thời gian kết thúc trước bắt đầu.

8. Viết sơ đồ trách nhiệm cho ứng dụng thư viện: Book, Member, Loan, Catalog.

16.16 Tóm tắt cuối chương

Bản đồ kiến thức

Lớp tạo kiểu mới; instance giữ trạng thái riêng. `self` là đối tượng nhận phương thức. Constructor thiết lập bất biến, phương thức cung cấp đường thay đổi hợp lệ. Thuộc tính mutable đặt ở lớp bị chia sẻ. Hướng đối tượng hữu ích khi trạng thái và hành vi cùng tạo một trách nhiệm; không phải mọi hàm đều cần lớp.

Góc nhìn cá nhân

Tôi không xem lớp là cách mô phỏng mọi thứ ngoài đời. Một chiếc ghế ngoài đời không bắt buộc phải thành `class Chair`. Tôi dùng lớp khi cần tạo một ranh giới: bên trong ranh giới, trạng thái được bảo vệ; bên ngoài, người dùng chỉ cần biết những hành động hợp lệ.

Chương 17

Đóng gói, property và bất biến – Bảo vệ trạng thái khỏi những thay đổi vô tình

Giới thiệu ngắn

Nếu mọi thuộc tính đều có thể bị sửa thành bất kỳ giá trị nào, lớp chỉ là dictionary viết bằng dấu chấm. Đóng gói không có nghĩa che giấu bí mật bằng dấu gạch dưới; nó là việc cung cấp một giao diện nhỏ và giữ các quy tắc đúng sau mọi thao tác. Python tin người lập trình, nhưng vẫn có công cụ để thể hiện ranh giới.

Mục tiêu học tập

- Hiểu quy ước public, protected-by-convention và name mangling.
- Dùng property để kiểm soát đọc/ghi mà giữ giao diện thuộc tính.
- Thiết kế bất biến và phương thức chuyển trạng thái.
- Tránh getter/setter máy móc và trạng thái nửa hợp lệ.
- Cân nhắc đối tượng bất biến và bản sao phòng vệ.

17.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Nhiệt độ đặt cho một lò mô phỏng chỉ hợp lệ trong một khoảng. Nếu mọi mã bên ngoài sửa trực tiếp thuộc tính, đối tượng có thể đi vào trạng thái nguy hiểm. Đóng gói không phải giấu dữ liệu vì bí mật; nó tập trung các đường cập nhật để mọi thay đổi đều được kiểm tra.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **đóng gói và property như cơ chế bảo vệ bất biến**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

17.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: trạng thái hợp lệ, API đọc/ghi, quy tắc xác thực, lỗi và khả năng tương thích.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

17.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$S = \{x \mid L \le x \le U\}, \qquad T : S \times A \to S$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: giá trị nội bộ đã xác nhận và lịch sử chuyển trạng thái cần thiết?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

17.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Không API công khai nào được tạo trạng thái ngoài miền; nếu cập nhật thất bại, trạng thái cũ phải còn nguyên.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

17.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **đóng gói và property như cơ chế bảo vệ bất biến** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

17.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Thiết kế lớp thermostat với property nhiệt độ đặt. Setter kiểm tra miền trước khi gán, phương thức thay đổi tương đối tái sử dụng cùng quy tắc và test xác nhận lỗi không làm mất giá trị cũ.

17.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

17.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

17.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

17.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

17.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

17.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

17.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Dùng dấu gạch dưới rồi cho rằng dữ liệu đã được bảo vệ tuyệt đối.
- Setter vừa gán vừa làm nhiều tác dụng phụ.
- Xác thực sau khi đã sửa một phần trạng thái.

- Tạo property cho mọi thuộc tính dù không có bất biến.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

17.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

17.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Viết bất biến cho tài khoản không cho số dư vượt hạn mức.
2. Thiết kế property chỉ đọc.
3. Kiểm thử atomicity của cập nhật thất bại.
4. So sánh phương thức với property theo ý nghĩa API.
5. Refactor lớp cho phép sửa trực tiếp thành API có kiểm soát.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

17.8 Dấu gạch dưới là một lời nhắc

Thuộc tính `_balance` nói "chi tiết nội bộ; không nên phụ thuộc trực tiếp". Python vẫn cho truy cập. Tên `__balance` kích hoạt name mangling thành dạng có tên lớp, chủ yếu tránh va chạm ở lớp con, không tạo bảo mật.

```
class BankAccount:
    def __init__(self, owner, opening=0):
        if opening < 0:
```

```
            raise ValueError("So du dau ky khong am")
        self.owner = owner
        self._balance = opening

    @property
    def balance(self):
        return self._balance
```

Không có setter nên `account.balance = -10` thất bại. Thay đổi phải qua `deposit` hoặc `withdraw`.

17.9 Property giữ giao diện đơn giản

```
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        value = float(value)
        if value < -273.15:
            raise ValueError("Duoi do khong tuyet doi")
        self._celsius = value

    @property
    def fahrenheit(self):
        return self._celsius * 9 / 5 + 32
```

Người dùng đọc `temp.celsius` như thuộc tính, trong khi lớp kiểm tra lúc gán. `fahrenheit` là giá trị tính, không cần lưu hai bản dễ lệch nhau.

Property có tác dụng phụ nặng

Người đọc kỳ vọng truy cập thuộc tính nhanh và ít bất ngờ. Không nên để `obj.value` âm thầm gọi mạng hoặc ghi tệp. Hành động nặng nên là phương thức có tên động từ như `fetch_value()`.

17.10 Bất biến phải đúng sau mọi phương thức công khai

Tài khoản có số dư không âm. Không chỉ constructor kiểm tra; rút tiền cũng phải bảo vệ.

```
class BankAccount:
    # __init__ va property nhu tren

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("So tien gui phai duong")
        self._balance += amount

    def withdraw(self, amount):
        if amount <= 0:
            raise ValueError("So tien rut phai duong")
        if amount > self._balance:
            raise ValueError("Khong du so du")
        self._balance -= amount
```

Nếu chuyển tiền giữa hai tài khoản và bước thứ hai lỗi sau khi đã trừ, bất biến từng tài khoản có thể đúng nhưng giao dịch tổng thể sai. Ta cần kiểm tra trước hoặc cơ chế giao dịch có hoàn tác. Bất biến tồn tại ở nhiều cấp.

17.11 Không để lộ danh sách nội bộ

```
class Course:
    def __init__(self):
        self._students = []

    @property
    def students(self):
        return tuple(self._students)
```

Nếu trả trực tiếp list, người gọi có thể `course.students.clear()` và bỏ qua quy tắc. Trả tuple hoặc bản sao là phòng vệ. Đổi lại, sao chép list lớn mỗi lần có chi phí; có thể cung cấp iterator hoặc view đọc tùy nhu cầu.

17.12 Đối tượng bất biến

Một giá trị như tọa độ, khoảng thời gian hoặc số tiền thường dễ lý luận nếu bất biến: thay đổi nghĩa là tạo giá trị mới. Dataclass `frozen=True` sẽ xuất hiện ở chương sau. Bất biến giúp băm, chia sẻ an toàn và giảm trạng thái chuyển tiếp, nhưng không phù hợp mọi đối tượng có vòng đời.

17.13 Máy trạng thái nhỏ

Một Task không chỉ có boolean done; nó có thể ở `TODO`, `DOING`, `DONE`, `CANCELLED`. Không phải chuyển đổi nào cũng hợp lệ.

```
from enum import Enum, auto

class Status(Enum):
    TODO = auto()
    DOING = auto()
    DONE = auto()
    CANCELLED = auto()

class Task:
    def __init__(self, title):
        self.title = title
        self._status = Status.TODO

    @property
    def status(self):
        return self._status

    def start(self):
        if self._status is not Status.TODO:
            raise ValueError("Chi task TODO moi co the bat dau")
        self._status = Status.DOING

    def complete(self):
        if self._status is not Status.DOING:
            raise ValueError("Task phai dang thuc hien")
        self._status = Status.DONE
```

Phương thức có tên diễn đạt sự kiện và bảo vệ chuyển trạng thái. Setter `status` cho phép mọi giá trị sẽ làm mất mô hình.

17.14 Getter/setter không phải mục tiêu

Viết `get_name` và `set_name` cho mọi trường chỉ tạo nghi thức. Python cho thuộc tính public khi không có bất biến. Bắt đầu đơn giản; chuyển sang property khi cần kiểm tra mà không đổi cách dùng. Giao diện tốt không có nhiều cổng nhất, mà có đúng cổng cần thiết.

Hỏi "ai được phép làm điều này?"

Nếu mã ngoài phải phối hợp ba thuộc tính theo đúng thứ tự, hành vi đó nên nằm trong lớp. Nếu lớp chỉ kiểm tra một giá trị không liên quan trạng thái, một hàm có

thể đủ. Đóng gói là đặt quy tắc gần dữ liệu mà nó bảo vệ.

17.15 Bài tập luyện tập

1. Thêm phí rút tiền vào BankAccount mà không cho số dư âm.
2. Thiết kế lớp `Percentage` luôn trong 0–100, hỗ trợ biểu diễn dạng 0–1.
3. Trả danh sách sách mượn theo cách người gọi không thể xóa nội bộ.
4. Xây máy trạng thái đơn hàng: mới, xác nhận, giao, hoàn thành, hủy.
5. Viết test mọi chuyển trạng thái hợp lệ và không hợp lệ.
6. Tìm dữ liệu được lưu trùng có thể lệch nhau; thay một trường bằng property tính.
7. So sánh lớp mutable và giá trị bất biến cho tọa độ; nêu hệ quả khi làm khóa dictionary.
8. Thiết kế chuyển tiền nguyên tử ở mức bộ nhớ: nếu bất kỳ bước nào lỗi, cả hai số dư giữ nguyên.

17.16 Tóm tắt cuối chương

Bản đồ kiến thức

Dấu gạch dưới thể hiện chi tiết nội bộ theo quy ước; name mangling không phải bảo mật. Property kiểm soát thuộc tính và tính giá trị dẫn xuất. Bất biến phải đúng sau mọi hành vi công khai. Trả bản sao, tuple hoặc iterator ngăn người gọi sửa nội bộ. Không tạo setter khi điều cần là một phương thức chuyển trạng thái có ý nghĩa.

Góc nhìn cá nhân

Ranh giới tốt không được xây vì thiếu tin tưởng người dùng, mà vì con người dễ quên những quy tắc không được viết xuống. Một phương thức `complete()` có thể nhỏ, nhưng nó giữ lại câu chuyện “một công việc được hoàn thành như thế nào” để mọi nơi không phải tự nhớ.

Chương 18

Kế thừa, hợp thành và đa hình – Tái sử dụng mà không trói chặt

Giới thiệu ngắn

Khi hai lớp có đoạn mã giống nhau, kế thừa thường là ý nghĩ đầu tiên. Nhưng giống vài dòng chưa chứng minh quan hệ "là một". Hợp thành xây đối tượng từ những cộng sự nhỏ; kế thừa tạo một kiểu chuyên biệt có thể thay thế kiểu cha. Chọn sai khiến sửa một lớp làm rung cả cây.

Mục tiêu học tập

- Hiểu kế thừa, ghi đè và `super`.
- Dùng đa hình dựa trên hành vi thay vì chuỗi `isinstance` dài.
- Ưu tiên hợp thành khi thay đổi chính sách hoặc cộng sự.
- Áp dụng nguyên tắc thay thế ở mức trực giác.
- Thiết kế giao diện bằng abstract base class hoặc protocol khi cần.

18.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một báo cáo có thể xuất ra terminal, tệp hoặc JSON. Các bộ xuất cùng thực hiện hợp đồng nhưng không nhất thiết có quan hệ bản chất giống nhau. Kế thừa phù hợp quan hệ is-a ổn định; hợp thành phù hợp quan hệ has-a và cho phép thay thành phần.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người

dùng ra quyết định? Với chủ đề **kế thừa, hợp thành và đa hình như lựa chọn về quan hệ**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

18.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: hợp đồng phương thức, quan hệ is-a/has-a, phụ thuộc, khả năng thay thế và điểm mở rộng.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

18.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$\text{behavior} = f(\text{interface}), \qquad \text{system} = \text{core} \circ \text{adapter}$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: đối tượng thành phần hiện tại và hành vi được chọn qua interface?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

18.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Mọi implementation phải thỏa hợp đồng để có thể thay thế mà không làm khách hàng đổi logic.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

18.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **kế thừa, hợp thành và đa hình như lựa chọn về quan hệ** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

18.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Thiết kế dịch vụ báo cáo nhận một writer qua constructor. Writer terminal và writer JSON cùng có phương thức `write`; dịch vụ không kiểm tra kiểu cụ thể. Ta so sánh với cây kế thừa sâu và đo mức liên kết.

18.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: “Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?” Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

18.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

18.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

18.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

18.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

18.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

18.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Dùng kế thừa chỉ để tái sử dụng vài dòng code.
- Override làm yếu tiền điều kiện hoặc phá hậu điều kiện.
- Kiểm tra `isinstance` ở mọi nơi thay cho đa hình.
- Tạo cây lớp sâu khiến thay đổi lan rộng.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

18.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

18.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Phân loại năm quan hệ thành is-a hoặc has-a.
2. Thiết kế Strategy cho cách tính điểm.
3. Tìm vi phạm nguyên lý thay thế.
4. Refactor một cây kế thừa thành hợp thành.
5. Viết contract test dùng chung cho hai implementation.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ “gõ cho chạy” thành luyện tư duy thiết kế.

18.8 Một ví dụ kế thừa hợp lý

```
from abc import ABC, abstractmethod

class Notification(ABC):
    @abstractmethod
    def send(self, message):
        raise NotImplementedError

class EmailNotification(Notification):
    def __init__(self, address):
        self.address = address

    def send(self, message):
        print(f"Email to {self.address}: {message}")

class ConsoleNotification(Notification):
```

```
    def send(self, message):
        print(message)
```

Mã gọi chỉ cần biết `send`:

```
def remind(notification, task):
    notification.send(f"Sap den han: {task.title}")
```

Đây là đa hình: nhiều kiểu phản hồi cùng một giao diện. Thêm kênh mới không sửa hàm `remind`.

18.9 Ghi đè và super

```
class TimedTask(Task):
    def __init__(self, task_id, title, minutes, priority=2):
        super().__init__(task_id, title, priority)
        if minutes <= 0:
            raise ValueError("Minutes phai duong")
        self.minutes = minutes
```

`super()` gọi triển khai theo thứ tự phân giải phương thức, không đơn giản chỉ là "lớp cha trực tiếp" trong đa kế thừa. Với dự án mới, tránh cây kế thừa sâu; hai hoặc ba tầng đã cần lý do rõ.

18.10 Nguyên tắc thay thế

Nếu `TimedTask` là Task, mọi nơi chấp nhận Task phải dùng được TimedTask mà không bất ngờ. Lớp con không nên siết tiền điều kiện vô lý, đổi kiểu trả về không tương thích hoặc phá bất biến cha.

Ví dụ kinh điển hình vuông kế thừa hình chữ nhật có setter rộng/cao độc lập thường vi phạm kỳ vọng: đặt rộng làm cao đổi theo. Mô hình toán học "hình vuông là hình chữ nhật" không tự đảm bảo giao diện mutable có thể thay thế.

18.11 Hợp thành: có một thay vì là một

TaskList cần kho lưu và bộ tạo ID. Thay vì kế thừa JsonStore, nó *có* một store.

```
class TaskService:
    def __init__(self, store, id_generator):
        self.store = store
        self.id_generator = id_generator

    def add(self, title, priority=2):
        tasks = self.store.load()
```

```
        task = Task(self.id_generator(), title, priority)
        tasks.append(task)
        self.store.save(tasks)
        return task
```

Trong test, truyền MemoryStore; khi chạy thật, truyền JsonStore hoặc SqliteStore. Hợp thành làm chính sách thay được và phụ thuộc lộ rõ.

18.12 Duck typing và Protocol

Python thường không cần lớp cha. Nếu đối tượng có `load`/`save` đúng hành vi, TaskService dùng được. Type hint Protocol mô tả giao diện cấu trúc:

```
from typing import Protocol

class TaskStore(Protocol):
    def load(self) -> list[Task]: ...
    def save(self, tasks: list[Task]) -> None: ...

class TaskService:
    def __init__(self, store: TaskStore):
        self.store = store
```

Không cần JsonStore khai báo kế thừa Protocol; công cụ kiểm tra kiểu nhìn vào phương thức. Abstract base class phù hợp khi cần triển khai chung hoặc kiểm tra runtime; Protocol linh hoạt cho hợp đồng hành vi.

18.13 Strategy thay cho if dài

```
class ByPriority:
    def key(self, task):
        return task.priority

class ByTitle:
    def key(self, task):
        return task.title.casefold()

def order_tasks(tasks, strategy):
    return sorted(tasks, key=strategy.key)
```

Trong Python, thậm chí truyền thẳng hàm key có thể gọn hơn lớp strategy. Pattern là tên cho một lực thiết kế, không phải mẫu phải sao chép nguyên xi.

18.14 Tránh kiểm tra kiểu lan rộng

```
# Mui code: moi loai moi phai sua ham nay
if isinstance(shape, Circle):
    area = ...
elif isinstance(shape, Rectangle):
    area = ...
```

Đặt `area()` vào mỗi kiểu nếu hành vi thuộc về kiểu. Tuy nhiên, nếu cần xuất nhiều định dạng không muốn sửa lớp miền, visitor hoặc hàm ngoài có thể tốt. Không có nguyên tắc "mọi if theo kiểu đều xấu"; hãy xem trục thay đổi nào cần mở rộng.

Ưu tiên quan hệ yếu vừa đủ

Hàm là quan hệ yếu nhất, rồi hợp thành qua giao diện, rồi kế thừa. Quan hệ càng mạnh càng tái sử dụng nhiều chi tiết nhưng càng trói thay đổi. Bắt đầu bằng công cụ ít ràng buộc nhất đáp ứng bài toán.

18.15 Bài tập luyện tập

1. Tạo ba kênh thông báo cùng phương thức `send`; test bằng kênh giả ghi vào list.
2. Thay chuỗi if tính phí vận chuyển bằng strategy.
3. Phân tích trường hợp Employee và Manager: nên kế thừa hay vai trò hợp thành?
4. Viết MemoryStore thay thế JsonStore mà TaskService không đổi.
5. Tạo Protocol cho clock để test thời gian.
6. Tìm một lớp con vi phạm thay thế do phát sinh ngoại lệ mới hoặc trả kiểu khác.
7. Tái cấu trúc cây kế thừa ba tầng thành các thành phần nhỏ.
8. Thiết kế game nhân vật có hành vi di chuyển, tấn công thay đổi; so sánh kế thừa với component.

18.16 Tóm tắt cuối chương

Bản đồ kiến thức

Kế thừa biểu đạt "là một" và đòi hỏi khả năng thay thế; `super` hợp tác theo MRO. Đa hình cho mã gọi dựa trên hành vi. Hợp thành biểu đạt "có một", giúp thay cộng sự và kiểm thử. Protocol mô tả giao diện mà không ép quan hệ thừa kế. Tái sử dụng vài dòng không đủ làm lý do tạo cây lớp.

Góc nhìn cá nhân

Tái sử dụng dễ khiến ta vui ở ngày đầu và trả giá ở tháng sau. Tôi chọn thiết kế không chỉ bằng câu hỏi “viết ít đi bao nhiêu”, mà bằng “khi yêu cầu đổi, bao nhiêu phần buộc phải đổi cùng”. Hợp thành thường cho tôi quyền thay một mảnh mà không xin phép cả gia phả.

Chương 19

Dataclass và các phương thức đặc biệt – Làm kiểu dữ liệu hòa vào Python

Giới thiệu ngắn

Nhiều lớp chủ yếu giữ dữ liệu và lặp lại `__init__`, `__repr__`, `__eq__`. Dataclass sinh phần nghi thức để ta tập trung vào quy tắc. Các phương thức đặc biệt (dunder) cho đối tượng tham gia `len`, so sánh, vòng lặp và context manager. Sức mạnh này cần dùng theo kỳ vọng của ngôn ngữ, không để toán tử quen thuộc làm việc bất ngờ.

Mục tiêu học tập

- Tạo data class với giá trị mặc định an toàn.
- Hiểu equality, ordering, frozen, slots ở mức ứng dụng.
- Định nghĩa `__repr__`, `__str__`, `__len__`, `__iter__`.
- Giữ quy tắc hash/equality nhất quán.
- Tránh mutable default và toán tử có ngữ nghĩa bất ngờ.

19.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một bản ghi điểm có nhiều trường dữ liệu và cần so sánh, in ra, dùng trong tập hợp hoặc sắp xếp. Dataclass sinh phần mã lặp, còn dunder quy định đối tượng tham gia các phép toán của Python. Mỗi dunder là một hợp đồng ngữ nghĩa, không phải mẹo làm code ngắn.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào,

đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **dataclass và phương thức đặc biệt như hợp đồng hòa nhập với Python**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

19.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: trường dữ liệu, giá trị mặc định, equality, ordering, hash, biểu diễn và tính bất biến.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

19.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$\text{eq}(a, b) \Rightarrow \text{hash}(a) = \text{hash}(b)$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: các trường instance và các giá trị dùng để xác định identity logic?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

19.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không

còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Equality, ordering và hash phải nhất quán; biểu diễn dành cho debug không được gây hiểu nhầm về trạng thái.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

19.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **dataclass và phương thức đặc biệt như hợp đồng hòa nhập với Python** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

19.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Thiết kế `ScoreRecord` bất biến gồm mã môn và điểm. Ta quyết định trường nào tham gia equality, kiểm tra hash, sắp xếp bằng key rõ nghĩa và dùng `__repr__` để log đủ ngữ cảnh.

19.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

19.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

19.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

19.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

19.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

19.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

19.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Dùng `unsafe_hash` khi đối tượng còn thay đổi.
- Định nghĩa thứ tự không có ý nghĩa nghiệp vụ.
- Cho list làm giá trị mặc định dùng chung.
- Viết dunder trả kiểu hoặc giá trị sai hợp đồng.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

19.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

19.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Tạo dataclass có validation sau khởi tạo.
2. Giải thích quan hệ equality–hash.
3. So sánh frozen với bất biến sâu.
4. Thiết kế `__repr__` hữu ích cho debug.
5. Viết ca thử cho default factory.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

19.8 Dataclass đầu tiên

```python
from dataclasses import dataclass, field
from datetime import date

@dataclass
class Task:
    task_id: str
    title: str
    priority: int = 2
    done: bool = False
    tags: list[str] = field(default_factory=list)

    def __post_init__(self):
        self.title = self.title.strip()
        if not self.title:
            raise ValueError("Tieu de rong")
```

```
        if self.priority not in {1, 2, 3}:
            raise ValueError("Priority khong hop le")
```

Dataclass tạo constructor, repr và equality theo trường. `default_factory=list` tạo list mới cho mỗi instance; viết `tags=[]` sẽ bị từ chối vì nguy cơ chia sẻ.

19.9 repr và str

`repr(obj)` dành cho lập trình viên, lý tưởng là biểu diễn rõ loại và trạng thái. `str(obj)` dành cho người dùng. Dataclass sinh repr hữu ích; ta có thể thêm str:

```
@dataclass
class Task:
    # cac truong...
    def __str__(self):
        mark = "x" if self.done else " "
        return f"[{mark}] {self.title} (P{self.priority})"
```

Không đưa token hoặc dữ liệu nhạy cảm vào repr vì log/debugger có thể ghi lại.

19.10 Frozen và hash

```
@dataclass(frozen=True, order=True)
class Point:
    x: float
    y: float
```

`frozen` ngăn gán trường thông thường và cho phép tạo hash khi các trường băm được. `order=True` sinh so sánh từ thứ tự trường, nhưng chỉ nên dùng nếu thứ tự đó có ý nghĩa tự nhiên. Task không nhất thiết "nhỏ hơn" Task chỉ vì ID đứng trước.

Hash phải nhất quán: nếu $a == b$, `hash(a)` phải bằng `hash(b)`. Một đối tượng mutable không nên làm khóa vì thay đổi có thể khiến nó nằm sai bucket. Đừng bật `unsafe_hash` khi chưa hiểu hậu quả.

19.11 slots

```
@dataclass(slots=True)
class SensorReading:
    timestamp: float
    value: float
```

Slots giảm bộ nhớ mỗi instance và ngăn thêm thuộc tính tùy ý, hữu ích khi có rất nhiều bản ghi. Đổi lại, nó ảnh hưởng một số kỹ thuật động và kế thừa. Chỉ tối ưu sau khi đo; đừng

dùng như trang sức cú pháp.

19.12 Làm một collection giống Python

```
class TaskList:
    def __init__(self, tasks=None):
        self._tasks = list(tasks or [])

    def __len__(self):
        return len(self._tasks)

    def __iter__(self):
        return iter(self._tasks)

    def __contains__(self, task_id):
        return any(t.task_id == task_id for t in self._tasks)

    def __getitem__(self, index):
        return self._tasks[index]
```

Giờ dùng `len(tasks)`, `for task in tasks`, `"T01" in tasks`, `tasks[0]`. Mỗi dunder nên tôn trọng kỳ vọng: `len` trả integer không âm; indexing phát sinh `IndexError`; iteration có thứ tự ổn định nếu collection hứa vậy.

19.13 Toán tử chỉ khi ngữ nghĩa tự nhiên

Một lớp Vector có cộng vector hợp lý. Cộng hai TaskList có thể hiểu là nối, nhưng xử lý ID trùng ra sao? Nếu cần cả hợp nhất, ghi đè và nối, các phương thức tên rõ tốt hơn một dấu `+` mơ hồ.

```
@dataclass(frozen=True)
class Vector2D:
    x: float
    y: float

    def __add__(self, other):
        if not isinstance(other, Vector2D):
            return NotImplemented
        return Vector2D(self.x + other.x, self.y + other.y)
```

Trả `NotImplemented` cho kiểu không hỗ trợ để Python thử phép phản chiếu hoặc báo TypeError đúng cách; khác với phát sinh `NotImplementedError`.

19.14 Serialization không tự nhiên hoàn toàn

`dataclasses.asdict` chuyển đệ quy thành dictionary, tiện nhưng có thể sao chép sâu và không xử lý datetime theo JSON. Một phương thức rõ giúp kiểm soát schema.

```
def task_to_dict(task):
    return {
        "id": task.task_id,
        "title": task.title,
        "priority": task.priority,
        "done": task.done,
        "tags": list(task.tags),
    }
```

Đừng gắn định dạng lưu trữ quá sâu vào domain nếu dự án cần nhiều định dạng; mapper ở tầng storage giữ sự độc lập.

Dunder là một hợp đồng với người dùng Python

Khi định nghĩa một phương thức đặc biệt, bạn mượn trực giác đã có của `+`, `len`, vòng lặp. Chỉ mượn khi hành vi của lớp không phản bội trực giác ấy.

19.15 Bài tập luyện tập

1. Chuyển Task viết tay thành dataclass và giữ validation.
2. Tạo Money bất biến gồm amount theo đơn vị nhỏ và currency; chỉ cộng cùng currency.
3. Xây Polynomial hỗ trợ `len`, gọi như hàm tại x, và repr dễ debug.
4. Thêm slicing cho TaskList; quyết định trả list hay TaskList.
5. Đo bộ nhớ nhiều instance có và không slots.
6. Tạo một lỗi hash do trường mutable, rồi giải thích vì sao set không tìm lại phần tử.
7. Viết mapper Task–JSON không phụ thuộc `asdict`.
8. Xem một toán tử bạn định overload; viết ba cách diễn giải có thể có. Nếu mơ hồ, thay bằng phương thức có tên.

19.16 Tóm tắt cuối chương

Bản đồ kiến thức

Dataclass giảm code nghi thức cho lớp dữ liệu; `default_factory` tránh chia sẻ mutable. Frozen hỗ trợ giá trị bất biến; order chỉ dùng khi thứ tự trường là thứ tự miền. Dunder cho kiểu tham gia giao thức Python nhưng phải giữ kỳ vọng. Equality và hash cần nhất quán; serialization cần schema có chủ đích.

Góc nhìn cá nhân

Một kiểu dữ liệu "hòa vào Python" không phải vì nó có thật nhiều dấu gạch dưới kép. Nó hòa vào vì người khác có thể đoán hành vi từ những điều đã biết. Sự quen thuộc là một loại giao diện; dùng đúng, nó làm chương trình nhỏ lại trong đầu người đọc.

Phần V

Viết Python theo cách của Python

Chương 20

Iterator và generator – Xử lý dữ liệu từng phần

Giới thiệu ngắn

Một danh sách giống như bày toàn bộ bữa ăn lên bàn. Iterator giống người phục vụ đưa từng món khi được hỏi. Với vài chục phần tử, khác biệt nhỏ. Với log hàng gigabyte, luồng cảm biến hoặc dãy vô hạn, việc tạo mọi thứ trước là không thể. Generator cho phép mô tả cách sinh giá trị và tạm dừng giữa các lần lấy.

Mục tiêu học tập

- Hiểu iterable, iterator, `iter` và `next`.
- Viết generator function bằng `yield`.
- Xây pipeline lười để xử lý tệp lớn.
- Nhận biết iterator dùng một lần, trạng thái và việc tiêu thụ.
- Dùng công cụ `itertools` cho kết hợp phổ biến.

20.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một tệp log hàng chục gigabyte không thể nạp toàn bộ vào RAM chỉ để đếm lỗi. Ta cần một giao thức trả từng phần tử khi được yêu cầu. Iterator giữ vị trí tiến trình; generator diễn đạt quá trình sinh dữ liệu và tạm dừng, giúp bộ nhớ phụ không tăng theo toàn bộ kích thước đầu vào.

Một chương trình tốt không bắt đầu bằng việc hỏi “phải dùng câu lệnh nào?”. Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng

nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **iterator và generator như mô hình tiêu thụ dữ liệu từng phần**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

20.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: nguồn dữ liệu, phần tử hiện tại, trạng thái tạm dừng, điều kiện kết thúc và số lần duyệt.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

20.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$x_k = \text{next}(I), \qquad M(n) \in \Theta(1) \text{ cho xử lý dòng tuần tự}$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: vị trí trong nguồn, biến cục bộ được giữ giữa hai lần next và trạng thái đã cạn?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

20.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không

còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Mỗi lần gọi trả đúng phần tử kế tiếp; sau khi cạn, iterator không tự quay lại trừ khi tạo iterator mới.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

20.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **iterator và generator như mô hình tiêu thụ dữ liệu từng phần** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

20.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Đếm mức ERROR trong tệp log theo từng dòng. Ta dựng pipeline đọc–lọc–trích trường–cộng dồn, đo RAM và kiểm tra rằng không bước nào vô tình biến luồng thành list.

20.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: “Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?” Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

20.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

20.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

20.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

20.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

20.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

20.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Dùng generator rồi duyệt hai lần như một collection.
- Gọi `list` quá sớm làm mất lợi ích streaming.
- Giữ tham chiếu lớn trong closure của generator.
- Nuốt lỗi giữa dòng khiến không biết dữ liệu dừng ở đâu.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

20.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

20.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Viết iterator đếm lùi có hợp đồng kết thúc.
2. Chuyển pipeline list thành generator.
3. Đo bộ nhớ khi xử lý một triệu số.
4. Giải thích `yield from`.
5. Thiết kế ca thử cho nguồn rỗng và iterator đã cạn.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

20.8 Giao thức lặp

Một iterable có thể tạo iterator bằng `iter`. Iterator trả phần tử kế tiếp bằng `next` và phát sinh `StopIteration` khi hết.

```
values = [10, 20, 30]
it = iter(values)
print(next(it))
print(next(it))
print(next(it))
```

Vòng `for` thực hiện cơ chế này và tự bắt `StopIteration`. List là iterable có thể tạo iterator mới nhiều lần. Chính iterator thường trả bản thân từ `iter(it)` và bị tiêu thụ.

```
it = iter([1, 2, 3])
print(list(it))  # [1, 2, 3]
print(list(it))  # []
```

20.9 Generator function

Một hàm chứa `yield` không chạy ngay khi gọi; nó trả generator. Mỗi `next` chạy đến `yield`, gửi giá trị ra và giữ trạng thái để lần sau tiếp tục.

```
def count_up_to(limit):
    value = 1
    while value <= limit:
        yield value
        value += 1

for number in count_up_to(3):
    print(number)
```

Biến `value` sống giữa các lần. Generator kết thúc khi hàm `return` hoặc đi hết thân.

20.10 Đọc bản ghi từng dòng

```
import json
from pathlib import Path

def read_json_lines(path):
    with Path(path).open(encoding="utf-8") as file:
        for line_number, line in enumerate(file, 1):
            if not line.strip():
                continue
            try:
                yield json.loads(line)
            except json.JSONDecodeError as error:
                raise ValueError(f"Invalid line {line_number}") from error
```

Tệp vẫn mở trong suốt thời gian duyệt generator. Nếu người gọi dừng sớm, Python thường đóng khi generator được đóng/thu gom, nhưng nên dùng nó trong phạm vi rõ hoặc xây context manager nếu tài nguyên cần thời điểm đóng chắc chắn.

20.11 Pipeline lười

```
records = read_json_lines("sessions.jsonl")
python_sessions = (r for r in records if r["course"] == "Python")
minutes = (r["minutes"] for r in python_sessions)
total = sum(minutes)
```

Mỗi bản ghi đi qua toàn đường ống rồi được bỏ, không tạo list trung gian. Điều này giảm bộ nhớ và có thể cho kết quả sớm. Đổi lại, pipeline chỉ chạy lúc tiêu thụ; lỗi cũng xuất hiện

muộn, tại `sum`, không phải lúc định nghĩa.

Generator không phải danh sách rẻ hơn trong mọi tình huống

Nếu cần duyệt lại, truy cập chỉ số, biết độ dài hoặc giữ snapshot ổn định, list phù hợp hơn. Chuyển generator thành list để dùng nhiều lần là hợp lý khi dữ liệu vừa bộ nhớ.

20.12 yield from

`yield from iterable` chuyển tiếp mọi phần tử.

```
def flatten(tree):
    for item in tree:
        if isinstance(item, list):
            yield from flatten(item)
        else:
            yield item
```

Không xây list kết quả; người gọi có thể dừng sau vài phần tử. Với cấu trúc sâu không tin cậy, vẫn có giới hạn stack.

20.13 Iterator lớp tùy chỉnh

```
class Countdown:
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self

    def __next__(self):
        if self.current < 0:
            raise StopIteration
        value = self.current
        self.current -= 1
        return value
```

Generator thường ngắn và dễ hơn. Lớp iterator hữu ích khi có nhiều thao tác trạng thái hoặc muốn API rõ. Nếu một collection cần lặp nhiều lần, `__iter__` nên trả iterator mới chứ không để collection tự là iterator dùng một lần.

20.14 itertools

```
from itertools import chain, islice, pairwise
```

```
2
3 combined = chain([1, 2], [3, 4])
4 first_ten = islice(count_up_to(1_000_000), 10)
5 changes = (b - a for a, b in pairwise([10, 12, 9, 15]))
```

`chain` nối lười, `islice` cắt iterator, `pairwise` tạo cặp kề. Còn có `groupby`, `product`, `combinations`. `groupby` chỉ nhóm các phần tử kề cùng khóa, nên thường phải sắp trước nếu muốn nhóm toàn cục.

20.15 Dãy vô hạn và hàng rào

```
1 from itertools import count, takewhile
2
3 squares = (n * n for n in count(1))
4 small = takewhile(lambda x: x < 1000, squares)
5 print(list(small))
```

Không gọi `list(count())`. Dãy vô hạn cần một bộ giới hạn như `islice`, `takewhile` hoặc điều kiện dừng của người tiêu thụ.

Ai sở hữu việc tiêu thụ?

Khi truyền iterator cho hàm, hãy giả định hàm có thể tiêu thụ nó. Nếu cần dùng sau, tạo iterator khác hoặc materialize. Tài liệu hàm nên nói nhận iterable hay sequence và có duyệt một hay nhiều lần.

20.16 Bài tập luyện tập

1. Viết generator sinh Fibonacci vô hạn; lấy 20 số bằng `islice`.
2. Đọc tệp lớn và yield các dòng chứa từ khóa, kèm số dòng.
3. Tạo cửa sổ trượt kích thước k bằng deque.
4. Viết generator chia iterable thành batch, xử lý batch cuối ngắn.
5. Dùng `yield from` duyệt cây thư mục nhưng bỏ thư mục ẩn theo quy tắc tự chọn.
6. Chứng minh một bug do duyệt iterator hai lần; sửa bằng thay đổi hợp đồng hoặc list.
7. Xây pipeline JSONL lọc, ánh xạ, tính top-k mà không giữ toàn bộ dữ liệu.
8. So sánh thời gian/bộ nhớ list comprehension và generator trên dữ liệu lớn.

20.17 Tóm tắt cuối chương

Bản đồ kiến thức

Iterable tạo iterator; iterator nhớ vị trí và thường dùng một lần. Generator tạm dừng tại `yield`, cho phép xử lý lười và dữ liệu lớn/vô hạn. Pipeline giảm bộ nhớ nhưng lỗi xảy ra khi tiêu thụ. `itertools` cung cấp các khối ghép lười. Hãy chọn list khi cần snapshot, độ dài, chỉ số hoặc nhiều lượt.

Góc nhìn cá nhân

Generator thay đổi cách tôi nghĩ về một kết quả. Kết quả không nhất thiết là thứ đã nằm sẵn trong tay; nó có thể là một lời hứa biết tạo phần tiếp theo khi cần. Tư duy này đặc biệt hữu ích khi bài toán lớn hơn bộ nhớ và khi ta chỉ cần vài câu trả lời đầu tiên.

Chương 21

Hàm bậc cao, closure và decorator – Truyền cả cách xử lý

Giới thiệu ngắn

Trong Python, hàm là đối tượng: có thể gán vào biến, truyền làm đối số và trả về từ hàm khác. Nhờ vậy, ta không chỉ truyền dữ liệu mà còn truyền một chiến lược xử lý. Decorator bọc một hàm để thêm hành vi chung như đo thời gian, phân quyền hoặc retry mà không sửa thân hàm. Công cụ này mạnh, nên cần giữ chữ ký, lỗi và tác dụng phụ dễ nhìn.

Mục tiêu học tập

- Truyền và trả hàm như giá trị.
- Dùng `key`, `map/filter` và comprehension có chủ đích.
- Hiểu closure giữ biến từ phạm vi ngoài.
- Viết decorator bằng `functools.wraps`.
- Phân biệt decorator cấu hình và context manager.

21.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một hệ thống xử lý điểm phải chuẩn hóa, kiểm tra, tính và ghi log. Nếu copy logic logging vào mọi hàm, thay đổi định dạng sẽ lan khắp dự án. Hàm bậc cao cho phép truyền hành vi; closure giữ ngữ cảnh; decorator bọc một hợp đồng mà không đổi lời gọi của khách hàng.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào,

đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **hàm bậc cao, closure và decorator như phép ghép hành vi**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

21.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: hàm đầu vào, hàm kết quả, dữ liệu được đóng, hợp đồng gọi, giá trị trả về và ngoại lệ.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

21.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$(f \circ g)(x) = f(g(x)), \qquad h = \text{decorate}(f)$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: tham chiếu hàm, môi trường closure và metadata của hàm được bọc?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

21.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Hàm được bọc phải giữ hợp đồng quan sát được trừ phần hành vi bổ sung đã công bố; đối số, giá trị trả về và lỗi không bị làm sai lệch.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

21.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **hàm bậc cao, closure và decorator như phép ghép hành vi** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

21.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Tạo decorator đo thời gian cho hàm báo cáo. Ta giữ metadata bằng công cụ chuẩn, dùng `try/finally` để log cả khi lỗi và kiểm tra decorator không nuốt giá trị trả về.

21.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi

tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

21.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

21.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

21.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

21.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

21.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

21.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Decorator quên return kết quả của hàm gốc.
- Closure bắt biến vòng lặp theo late binding.
- Bọc nhiều lớp nhưng không bảo toàn metadata.
- Dùng decorator cho logic nghiệp vụ khiến luồng khó thấy.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

21.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

21.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Viết hàm nhận một predicate để lọc.
2. Truy vết môi trường của closure bộ đếm.
3. Thiết kế decorator retry có giới hạn.
4. Kiểm thử decorator khi hàm gốc phát ngoại lệ.
5. So sánh decorator với gọi hàm bọc tường minh.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

21.8 Hàm là giá trị

```
def by_priority(task):
    return task.priority

strategy = by_priority
ordered = sorted(tasks, key=strategy)
```

Không có dấu ngoặc ở `strategy = by_priority`; ta truyền bản thân hàm, không gọi nó. `sorted` sẽ gọi với từng phần tử.

21.9 Hàm nhận hàm

```
def apply_to_valid(values, validator, transform):
    result = []
    for value in values:
        if validator(value):
            result.append(transform(value))
    return result

positive_squares = apply_to_valid(
    [-2, -1, 0, 3, 4],
    lambda x: x > 0,
```

```
    lambda x: x * x,
)
```

Lambda chỉ chứa một biểu thức. Nếu logic cần tên, nhiều bước hoặc tái sử dụng, `def` dễ đọc hơn. Comprehension có thể tự nhiên hơn `map/filter`: `[x*x for x in values if x>0]`.

21.10 Closure

```
def make_minimum_validator(minimum):
    def validate(value):
        return value >= minimum
    return validate

adult = make_minimum_validator(18)
print(adult(20))
```

Sau khi hàm ngoài kết thúc, hàm trong vẫn giữ `minimum`. Closure là cách tạo hàm đã được cấu hình mà không cần lớp. Nếu trạng thái có nhiều hành vi hoặc cần biểu diễn rõ, lớp callable có thể tốt hơn.

21.11 Cạm bẫy late binding

```
functions = []
for i in range(3):
    functions.append(lambda: i)

print([f() for f in functions])  # [2, 2, 2]
```

Lambda tra `i` khi được gọi, lúc đó vòng lặp đã kết thúc. Gắn giá trị qua đối số mặc định:

```
functions.append(lambda i=i: i)
```

Đây không phải lỗi riêng lambda; closure trên biến thay đổi có cùng hành vi.

21.12 Decorator đầu tiên

```
from functools import wraps
from time import perf_counter

def timed(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = perf_counter()
```

```
        try:
            return func(*args, **kwargs)
        finally:
            elapsed = perf_counter() - start
            print(f"{func.__name__}: {elapsed:.6f}s")
    return wrapper

@timed
def calculate():
    return sum(range(1_000_000))
```

`@timed` tương đương `calculate = timed(calculate)`. `wraps` giữ tên, docstring và metadata, giúp debugger và công cụ tài liệu không thấy mọi hàm đều tên `wrapper`.

21.13 Decorator có tham số

```
from functools import wraps
import time

def retry(attempts=3, exceptions=(OSError,), delay=0.0):
    if attempts < 1:
        raise ValueError("attempts phai >= 1")

    def decorate(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, attempts + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions:
                    if attempt == attempts:
                        raise
                    time.sleep(delay)
        return wrapper
    return decorate
```

Retry chỉ phù hợp lỗi tạm thời và thao tác an toàn lặp lại. Gửi tiền hoặc tạo đơn hàng có thể thực hiện thành công ở máy chủ nhưng phản hồi bị mất; thử lại mù quáng gây trùng. Cần idempotency key hoặc giao thức nghiệp vụ.

21.14 Thứ tự decorator

```
@log_calls
@require_auth
```

```
def delete_item(item_id):
    ...
```

Tương đương `delete_item = log_calls(require_auth(delete_item))`. Lời gọi đi từ decorator ngoài vào trong. Thứ tự có thể ảnh hưởng: ghi log trước xác thực có làm lộ đối số không? Cache trước phân quyền có trả dữ liệu sai người không? Decorator không làm biến mất quyết định bảo mật.

21.15 Khi context manager rõ hơn

Decorator bọc toàn lời gọi hàm. Nếu chỉ đo một đoạn hoặc tài nguyên có phạm vi block, context manager rõ hơn:

```
with timer("doc tep"):
    data = load_file(path)
    validate(data)
```

Đừng dùng decorator khiến một hàm có hành vi ẩn khó đoán. Những điều quan trọng như transaction, quyền và retry cần tên, tài liệu và test.

Trừu tượng hóa điều thay đổi, giữ đường đi nhìn thấy được

Hàm bậc cao hữu ích khi phần khung giống nhau và chiến lược khác nhau. Decorator hữu ích cho hành vi cắt ngang. Nếu người đọc phải nhảy qua năm lớp bọc để hiểu một lời gọi, sự tái sử dụng đã vượt quá độ rõ.

21.16 Bài tập luyện tập

1. Viết hàm nhóm dữ liệu theo một hàm key được truyền vào.
2. Tạo closure chuyển đổi điểm theo hệ số đã cấu hình.
3. Tái hiện late binding với callback trong vòng lặp và sửa.
4. Viết decorator đếm số lần gọi, giữ metadata.
5. Viết decorator validate đối số dương; cân nhắc type hint và chữ ký.
6. Thêm exponential backoff cùng jitter vào retry và chỉ bắt lỗi chỉ định.
7. Test thứ tự hai decorator bằng list sự kiện.
8. So sánh decorator timing với context manager timing cho một pipeline nhiều bước.

21.17 Tóm tắt cuối chương

Bản đồ kiến thức

Hàm có thể được truyền như dữ liệu. Closure giữ biến ngoài và tạo hành vi đã cấu hình. Late binding đọc biến lúc gọi. Decorator thay hàm bằng hàm bọc; luôn dùng `wraps` khi phù hợp. Retry phải hiểu tính idempotent. Thứ tự decorator là thứ tự hành vi, không phải trang trí.

Góc nhìn cá nhân

Khả năng truyền một hàm khiến chương trình giống cuộc đối thoại: “hãy sắp dữ liệu này, còn cách đánh giá tôi sẽ đưa sau”. Nhưng mỗi tầng trừu tượng cũng lấy đi một chút khả năng nhìn thấy. Tôi chỉ bọc hành vi khi tên của lớp bọc giúp người đọc hiểu thêm, không phải để mã trông kỳ ảo hơn.

Chương 22

Context manager và type hints – Quản lý vòng đời, ghi rõ hợp đồng

Giới thiệu ngắn

Một tệp phải được đóng. Một khóa phải được nhả. Một giao dịch phải commit hoặc rollback. Context manager gắn bước vào và bước ra quanh một khối lệnh. Type hints không ép Python đổi thành ngôn ngữ tĩnh, nhưng giúp con người và công cụ thấy dữ liệu được mong đợi đi qua hệ thống như thế nào.

Mục tiêu học tập

- Dùng và tự viết context manager.
- Bảo đảm dọn dẹp khi khối lệnh lỗi.
- Viết type hints cho collection, optional, callable và kiểu tự tạo.
- Dùng Protocol/TypedDict ở biên phù hợp.
- Hiểu type checker không thay validation runtime và test.

22.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Mở tệp hoặc khóa tài nguyên tạo một khoảng thời gian mà tài nguyên phải được đóng đúng, kể cả khi lỗi. Đồng thời, một hàm xử lý bản ghi cần công bố kiểu dữ liệu để người đọc và công cụ phát hiện mâu thuẫn trước khi chạy. Context manager quản lý vòng đời; type hints ghi hợp đồng tĩnh nhưng không tự xác thực dữ liệu lúc chạy.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng

nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **context manager và type hints như hợp đồng vòng đời và dữ liệu**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

22.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: tài nguyên, điểm nhận/trả, kiểu tham số, kiểu trả về, optional, generic và lỗi.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

22.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$\text{acquire} \to \text{use} \to \text{release}, \qquad f : X \to Y$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: tài nguyên đang sở hữu và giá trị nào có thể vắng mặt?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

22.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không

còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Mỗi tài nguyên đã nhận phải được trả đúng một lần; mọi đường trả về của hàm phải phù hợp hợp đồng kiểu đã công bố.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

22.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **context manager và type hints như hợp đồng vòng đời và dữ liệu** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

22.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Viết context manager cho một transaction mô phỏng: bắt đầu, thực hiện, commit khi thành công và rollback khi lỗi. Type hints mô tả protocol storage để thay implementation trong test.

22.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: “Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?” Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

22.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

22.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

22.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

22.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

22.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

22.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Cho rằng type hints tự chặn dữ liệu sai lúc chạy.
- Trả tài nguyên ở cuối hàm nhưng bỏ sót đường ngoại lệ.
- Dùng `Any` để tắt mọi kiểm tra.
- Annotation phức tạp hơn giá trị mà nó giải thích.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

22.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

22.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Vẽ vòng đời của một tệp trong khối with.
2. Viết annotation cho hàm nhận iterable.
3. Phân biệt Optional với tham số có giá trị mặc định.
4. Thiết kế Protocol cho kho dữ liệu.
5. Kiểm thử cleanup khi thân context phát lỗi.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ “gõ cho chạy” thành luyện tư duy thiết kế.

22.8 Giao thức context manager

```
class Timer:
    def __init__(self, label):
        self.label = label

    def __enter__(self):
        from time import perf_counter
        self._start = perf_counter()
        return self

    def __exit__(self, exc_type, exc, traceback):
        from time import perf_counter
        self.elapsed = perf_counter() - self._start
        print(f"{self.label}: {self.elapsed:.4f}s")
        return False

```

```
with Timer("tinh tong") as timer:
    total = sum(range(1_000_000))
```

`__enter__` chuẩn bị và giá trị trả được gán sau `as`. `__exit__` nhận thông tin ngoại lệ. Trả `False` hoặc `None` để ngoại lệ đi tiếp; trả `True` để nuốt, điều chỉ nên làm khi có chủ đích rõ.

22.9 contextmanager bằng generator

```
from contextlib import contextmanager
from pathlib import Path
import shutil

@contextmanager
def backup_before_edit(path):
    path = Path(path)
    backup = path.with_suffix(path.suffix + ".bak")
    if path.exists():
        shutil.copy2(path, backup)
    try:
        yield path
    except Exception:
        if backup.exists():
            shutil.copy2(backup, path)
        raise
```

Phần trước `yield` là enter; phần sau là exit. Không xóa backup ở đây vì chính sách giữ bản sao cần được quyết định. Với transaction, nhánh thành công commit và nhánh lỗi rollback.

22.10 Type hints cơ bản

```
def average(values: list[float]) -> float:
    if not values:
        raise ValueError("empty")
    return sum(values) / len(values)

def find_task(tasks: list[Task], task_id: str) -> Task | None:
    return next((t for t in tasks if t.task_id == task_id), None)
```

Annotation không tự kiểm tra lúc chạy. Gọi `average(["a"])` vẫn vào hàm rồi lỗi ở phép cộng. Type checker chạy riêng có thể phát hiện trước nếu luồng được gắn kiểu đủ.

Nên nhận kiểu trừu tượng đủ dùng: nếu chỉ duyệt, `Iterable[float]` linh hoạt hơn `list[float]`; nếu cần len và chỉ số, `Sequence` phù hợp.

```python
from collections.abc import Iterable

def average(values: Iterable[float]) -> float:
    values = list(values)
    if not values:
        raise ValueError("empty")
    return sum(values) / len(values)
```

Việc chuyển list cho phép duyệt hai lần nhưng dùng bộ nhớ; hoặc tính tổng và đếm một lượt.

22.11 Callable, TypeAlias và Literal

```python
from collections.abc import Callable
from typing import Literal, TypeAlias

TaskId: TypeAlias = str
SortMode = Literal["priority", "due", "title"]

def select(
    tasks: list[Task],
    predicate: Callable[[Task], bool],
) -> list[Task]:
    return [task for task in tasks if predicate(task)]
```

Alias giúp nói ý nghĩa nhưng không tạo kiểu runtime mới. `NewType` có thể phân biệt ID với str trong type checker. `Literal` giới hạn lựa chọn tĩnh; dữ liệu người dùng vẫn cần validation.

22.12 TypedDict cho dữ liệu dạng dictionary

```python
from typing import TypedDict, NotRequired

class TaskPayload(TypedDict):
    id: str
    title: str
    priority: int
    done: bool
    due: NotRequired[str | None]
```

TypedDict mô tả hình dạng dictionary cho công cụ tĩnh, không tạo instance đặc biệt và không kiểm tra JSON runtime. Nó tốt ở biên serialization; trong lõi nhiều hành vi, dataclass có thể phù hợp hơn.

22.13 Generic đơn giản

```python
from typing import TypeVar
from collections.abc import Iterable, Callable

T = TypeVar("T")
K = TypeVar("K")

def first_by(items: Iterable[T], key: Callable[[T], K]) -> T:
    iterator = iter(items)
    try:
        best = next(iterator)
    except StopIteration as error:
        raise ValueError("empty items") from error
    for item in iterator:
        if key(item) < key(best):
            best = item
    return best
```

TypeVar giữ mối quan hệ: kiểu trả về giống kiểu phần tử đầu vào. Đừng biến annotation thành câu đố; một kiểu phức tạp khó đọc có thể cần thiết kế API đơn giản hơn.

22.14 Type checker, validation và test

Type checker phát hiện sai hợp đồng trong mã ta kiểm soát. Validation kiểm tra dữ liệu bên ngoài lúc chạy. Test kiểm tra ví dụ hành vi. Ba lớp bổ sung nhau.

Any là lỗ thoát có chủ đích

`Any` tắt nhiều kiểm tra và lan qua biểu thức. Dùng ở biên thư viện thiếu type hoặc dữ liệu thật sự động, rồi thu hẹp sớm bằng validation/cast có bằng chứng. Đừng dùng Any chỉ để type checker im lặng.

22.15 Bài tập luyện tập

1. Viết context manager đổi thư mục tạm thời và luôn quay lại thư mục cũ.
2. Viết transaction giả trên list: sao chép, rollback nếu block lỗi.
3. Gắn kiểu cho các hàm TaskService và sửa lỗi type checker phát hiện.
4. Thay tham số list bằng Iterable/Sequence phù hợp; giải thích lựa chọn.
5. Tạo TypedDict cho payload API và hàm validation runtime tương ứng.
6. Dùng Protocol cho đối tượng có `write(str)`.

7. Viết generic `chunked` giữ kiểu phần tử.
8. Tìm annotation quá cụ thể làm hạn chế người gọi; nới theo nguyên tắc nhận trừu tượng, trả cụ thể.

22.16 Tóm tắt cuối chương

Bản đồ kiến thức

Context manager quản lý enter/exit và dọn dẹp kể cả khi lỗi. Có thể viết bằng lớp hoặc `contextmanager`. Type hints ghi hợp đồng cho người và công cụ, nhưng không kiểm tra dữ liệu runtime. TypedDict mô tả dictionary, Protocol mô tả hành vi, generic giữ quan hệ kiểu. Type checking, validation và test giải quyết ba loại bằng chứng khác nhau.

Góc nhìn cá nhân

Tôi thích type hints khi chúng giống ghi chú bên lề của một người đi trước: "dữ liệu đến đây có hình dạng này, và nếu không, có lẽ bạn đã rẽ sai ở đâu đó". Nhưng ghi chú không thay hàng rào. Dữ liệu từ thế giới bên ngoài vẫn phải được kiểm tra, vì thế giới không đọc annotation của chúng ta.

Chương 23

Bất đồng bộ và đồng thời – Làm việc khác trong lúc chờ

Giới thiệu ngắn

Chương trình tải mười trang web thường dành phần lớn thời gian chờ mạng. Nếu cứ gửi một yêu cầu rồi ngồi yên, tài nguyên bị bỏ phí. Đồng thời cho nhiều công việc tiến triển chồng thời gian; bất đồng bộ dùng một luồng điều phối các tác vụ biết nhường khi chờ. Nó không làm phép tính CPU tự nhanh hơn và đưa vào những lỗi mới về thứ tự, hủy và giới hạn tài nguyên.

Mục tiêu học tập

- Phân biệt tuần tự, đồng thời, song song và bất đồng bộ.
- Hiểu coroutine, event loop, `async`/`await`.
- Chạy tác vụ đồng thời bằng `gather`/`TaskGroup`.
- Giới hạn đồng thời, timeout và xử lý hủy.
- Chọn thread, process hoặc asyncio theo loại công việc.

23.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một chương trình tải mười tài nguyên chủ yếu dành thời gian chờ mạng. Chạy tuần tự làm thời gian chờ cộng lại; bất đồng bộ cho tác vụ khác tiến trong lúc một tác vụ chờ. Điều đó không làm CPU tính nhanh hơn và không tự giải quyết trạng thái dùng chung.

Một chương trình tốt không bắt đầu bằng việc hỏi “phải dùng câu lệnh nào?”. Nó bắt

đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **bất đồng bộ và đồng thời như bài toán xếp lịch khi phải chờ**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

23.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: tác vụ, thời gian chờ, thời gian CPU, mức đồng thời, timeout, thứ tự kết quả và tài nguyên dùng chung.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

23.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$T_{\text{seq}} \approx \sum_i (L_i + C_i), \qquad T_{\text{async}} \approx \max_i L_i + \sum_i C_i$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: tập tác vụ sẵn sàng/đang chờ, hàng đợi sự kiện và giới hạn đồng thời?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

23.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Không tác vụ nào bị bỏ quên; tài nguyên dùng chung được bảo vệ và mức đồng thời không vượt giới hạn dịch vụ.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

23.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **bất đồng bộ và đồng thời như bài toán xếp lịch khi phải chờ** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

23.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Gọi nhiều API với semaphore giới hạn năm kết nối. Ta ghi thời điểm bắt đầu/kết thúc, đặt timeout, gom lỗi theo từng yêu cầu và trả kết quả theo mã thay vì giả định thứ tự hoàn thành.

23.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

23.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

23.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

23.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

23.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

23.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định “kết quả ấy có vẻ được”, ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là “thuật toán giữ bất biến và tạo hậu điều kiện”. Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

23.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Quên await nên giữ coroutine chưa chạy.
- Tạo vô hạn tác vụ và làm nghẽn dịch vụ.
- Chặn event loop bằng công việc CPU nặng.
- Giả định thứ tự hoàn thành giống thứ tự khởi tạo.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

23.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

23.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Vẽ timeline tuần tự và bất đồng bộ.
2. Thiết kế giới hạn đồng thời.
3. Xử lý timeout từng tác vụ.
4. Mô phỏng race condition trên trạng thái chung.
5. Phân biệt async, thread và process theo loại công việc.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

23.8 Chờ I/O và tính CPU

I/O-bound dành thời gian chờ mạng, đĩa, cơ sở dữ liệu. CPU-bound dành thời gian tính toán. Asyncio phù hợp nhiều I/O có API bất đồng bộ. Thread có thể bọc I/O đồng bộ. Process

tận dụng nhiều lõi cho CPU-bound, đổi lại chi phí truyền dữ liệu và quản lý.

Công cụ	Phù hợp	Lưu ý
asyncio	nhiều I/O async	mọi thao tác chờ cần biết nhường
thread	I/O blocking, thư viện đồng bộ	trạng thái chia sẻ, khóa
process	CPU-bound độc lập	khởi tạo và truyền dữ liệu tốn kém

23.9 Coroutine đầu tiên

```python
import asyncio

async def study_session(name, seconds):
    print(f"Bat dau {name}")
    await asyncio.sleep(seconds)
    print(f"Xong {name}")
    return name

async def main():
    result = await study_session("Python", 1)
    print(result)

asyncio.run(main())
```

Gọi hàm `async def` tạo coroutine, chưa chạy đến khi await hoặc lập lịch. `await` chỉ dùng trong async function và nhường quyền khi đối tượng chờ chưa hoàn tất.

23.10 Chạy chồng thời gian

```python
async def main():
    results = await asyncio.gather(
        study_session("Python", 1.0),
        study_session("Math", 1.5),
        study_session("English", 0.5),
    )
    print(results)
```

Tổng thời gian gần tác vụ lâu nhất thay vì tổng, vì `sleep` tượng trưng I/O biết nhường. Nếu thay bằng vòng CPU nặng không await, event loop bị chặn và mọi tác vụ đứng.

Trên Python hiện đại, `TaskGroup` cho structured concurrency:

```python
async def main():
    async with asyncio.TaskGroup() as group:
        a = group.create_task(study_session("Python", 1))
        b = group.create_task(study_session("Math", 1))
```

```
    print(a.result(), b.result())
```

Khối chỉ kết thúc khi các task con kết thúc; nếu một task lỗi, nhóm hủy các task còn lại và báo nhóm ngoại lệ. Vòng đời không bị thả nổi.

23.11 Giới hạn đồng thời

Khởi tạo 100.000 yêu cầu cùng lúc có thể làm nghẽn chính ta hoặc dịch vụ. Semaphore đặt số tác vụ trong vùng tài nguyên.

```
async def worker(item, semaphore):
    async with semaphore:
        return await process_item(item)

async def run_all(items, limit=10):
    semaphore = asyncio.Semaphore(limit)
    return await asyncio.gather(
        *(worker(item, semaphore) for item in items)
    )
```

Với luồng đầu vào rất lớn, còn cần queue và số worker cố định để không tạo hết task từ đầu.

23.12 Timeout, hủy và dọn dẹp

```
async def fetch_with_timeout():
    try:
        async with asyncio.timeout(5):
            return await fetch_data()
    except TimeoutError:
        return None
```

Khi task bị hủy, `CancelledError` xuất hiện tại điểm await. Dùng `try/finally` hoặc async context manager để đóng kết nối. Không nuốt hủy vô thức; thường phải dọn rồi `raise` lại.

23.13 Race condition vẫn có trong một luồng

Task chỉ chuyển quyền ở await, nhưng nếu đọc trạng thái, await, rồi ghi, task khác có thể chen vào.

```
lock = asyncio.Lock()

async def increment(counter):
```

```
    async with lock:
        counter["value"] += 1
```

Nếu vùng bảo vệ có await I/O dài, lock giữ lâu. Tốt hơn giảm trạng thái chia sẻ, dùng queue truyền thông điệp hoặc giữ vùng khóa nhỏ.

23.14 Đưa hàm blocking sang thread

```
result = await asyncio.to_thread(blocking_file_operation, path)
```

Điều này giữ event loop không bị chặn với I/O đồng bộ, nhưng không biến thư viện thành async hoàn hảo và không tự làm CPU-bound nhanh do GIL trong triển khai CPython. Đo và chọn process khi thích hợp.

Async lan qua giao diện

Một hàm async phải được await, nên quyết định async thường lan từ tầng I/O lên điểm chạy. Trộn sync/async tùy tiện dẫn đến event loop lồng hoặc coroutine không được chờ. Giữ ranh giới rõ và cung cấp wrapper tại biên nếu cần.

23.15 Bài tập luyện tập

1. Mô phỏng năm I/O bằng sleep; so sánh tuần tự và gather.
2. Viết hàng đợi có ba worker, sentinel kết thúc và thu kết quả.
3. Thêm semaphore và đo các giới hạn 1, 5, 20.
4. Tạo timeout cho từng tác vụ và timeout toàn lô; so sánh.
5. Viết tác vụ có `finally`, hủy nó và chứng minh dọn dẹp chạy.
6. Tái hiện race condition do await giữa đọc/ghi, rồi sửa bằng lock hoặc queue.
7. Chọn thread/process/asyncio cho: resize ảnh, tải URL, đọc 1.000 tệp, tính số nguyên tố.
8. Thiết kế retry async có backoff nhưng tôn trọng cancellation.

23.16 Tóm tắt cuối chương

Bản đồ kiến thức

Đồng thời chồng thời gian tiến triển; song song thực thi cùng lúc trên nhiều tài nguyên. Asyncio phù hợp I/O biết nhường, không tăng tốc CPU-bound. Coroutine chạy khi await/lập lịch. TaskGroup giữ vòng đời có cấu trúc. Luôn giới hạn đồng thời, đặt

timeout, xử lý hủy và tránh trạng thái chia sẻ qua điểm await.

Góc nhìn cá nhân

Bất đồng bộ không làm máy tính "làm mọi thứ cùng lúc" một cách kỳ diệu. Nó dạy chương trình biết nhường khi chưa thể tiến tiếp. Nhưng càng nhiều việc chen nhau, ta càng phải nói rõ ai sở hữu tài nguyên, ai được hủy và khi một phần thất bại thì những phần còn lại sống ra sao.

Phần VI

Mang kiến thức ra khỏi trang giấy

Chương 24

Tự động hóa – Để máy tính nhận những việc lặp lại

Giới thiệu ngắn

Tự động hóa không bắt đầu từ robot phức tạp. Nó bắt đầu khi bạn đổi tên 300 tệp bằng tay và nhận ra mình đang thực hiện cùng một quy tắc. Một script tốt không chỉ chạy nhanh; nó có chế độ xem trước, log, xử lý trường hợp trùng tên và có thể chạy lại mà không gây hỏng. Chương này kết hợp pathlib, CSV, lỗi và logging thành các công cụ nhỏ có trách nhiệm.

Mục tiêu học tập

- Nhận diện công việc lặp có quy tắc và biên đầu vào rõ.
- Xây chế độ dry-run, xác nhận và kế hoạch hoàn tác.
- Đổi tên, phân loại, tổng hợp tệp an toàn.
- Dùng `argparse` tạo giao diện dòng lệnh.
- Thiết kế script idempotent và có log kiểm toán.

24.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Mỗi tuần một người tải tệp, đổi tên, kiểm tra cột, tạo báo cáo và gửi vào thư mục lưu trữ. Tự động hóa không chỉ thay tay bằng script; nó phải mô tả tiền điều kiện, khả năng chạy lại, log, đường thất bại và cách không làm hỏng dữ liệu khi dừng giữa chừng.

Một chương trình tốt không bắt đầu bằng việc hỏi “phải dùng câu lệnh nào?”. Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng

nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **tự động hóa như biến quy trình lặp thành một hệ thống có kiểm soát**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

24.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: tác vụ, đầu vào, đầu ra, thứ tự, lịch chạy, quyền truy cập, checkpoint và chính sách chạy lại.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

24.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$\text{workflow} = s_1 \circ s_2 \circ \cdots \circ s_k, \qquad s_i(s_i(x)) = s_i(x) \text{ nếu idempotent}$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: bước đã hoàn tất, artifact đã tạo và trạng thái phục hồi?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

24.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không

còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Chạy lại cùng đầu vào không được tạo kết quả trùng hoặc phá dữ liệu; mỗi bước chỉ đánh dấu xong sau khi đầu ra đã hợp lệ.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

24.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **tự động hóa như biến quy trình lặp thành một hệ thống có kiểm soát** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

24.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Tự động sắp xếp tệp bài nộp. Ta quét thư mục, parse tên, kiểm tra schema, lập kế hoạch di chuyển ở chế độ dry-run, rồi mới áp dụng và ghi manifest để có thể kiểm tra.

24.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

24.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

24.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

24.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

24.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

24.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

24.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Thao tác trực tiếp mà không có dry-run.
- Xóa/ghi đè trước khi kiểm tra đầu ra.
- Giả định đường dẫn và múi giờ cố định.
- Không phân biệt lỗi có thể thử lại với lỗi dữ liệu.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

24.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

24.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Vẽ workflow cho báo cáo tuần.
2. Thiết kế manifest và checkpoint.
3. Thêm chế độ dry-run.
4. Chứng minh một bước có tính idempotent.
5. Thử dừng giữa quy trình và phục hồi.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

24.8 Một quy trình trước khi viết code

Hãy ghi năm cột: đầu vào, đầu ra, quy tắc, ngoại lệ, cách kiểm tra. Ví dụ phân loại thư mục tải về: đầu vào là tệp ở một thư mục; đầu ra là các thư mục theo đuôi; không chạm thư mục con; trùng tên thì không ghi đè; dry-run in kế hoạch; số lượng trước và sau phải bằng nhau.

Không tự động hóa một quy trình bạn chưa làm đúng bằng tay. Máy chỉ khuếch đại quy tắc, kể cả quy tắc sai.

24.9 Lập kế hoạch trước khi thay đổi

```
from dataclasses import dataclass
from pathlib import Path

@dataclass(frozen=True)
class Move:
```

```
    source: Path
    destination: Path

def plan_by_extension(folder):
    folder = Path(folder)
    plan = []
    for source in folder.iterdir():
        if not source.is_file():
            continue
        category = source.suffix.lower().lstrip(".") or "no_extension"
        destination = folder / category / source.name
        plan.append(Move(source, destination))
    return plan
```

Hàm lập kế hoạch không thay đổi đĩa, nên dễ test. Bước kiểm tra phát hiện đích trùng, nguồn lặp, đích nằm ngoài thư mục cho phép. Bước thực thi chỉ chạy kế hoạch hợp lệ.

```
def execute(plan, dry_run=True):
    for move in plan:
        print(f"{move.source} -> {move.destination}")
        if dry_run:
            continue
        if move.destination.exists():
            raise FileExistsError(move.destination)
        move.destination.parent.mkdir(parents=True, exist_ok=True)
        move.source.replace(move.destination)
```

24.10 Đổi tên hai pha

Nếu A đổi thành B và B đổi thành A, đổi trực tiếp gây va chạm. Một chiến lược là đổi mọi nguồn sang tên tạm duy nhất, rồi từ tạm sang đích. Phải ghi journal để khôi phục nếu giữa chừng lỗi.

```
import uuid

def temporary_name(path):
    return path.with_name(f".__rename_{uuid.uuid4().hex}")
```

Trong script thật, journal JSON ghi nguồn, tạm, đích và trạng thái sau từng bước. Khi khởi động lại, chương trình phát hiện phiên chưa hoàn tất và đề nghị tiếp tục hoặc rollback.

24.11 Idempotency

Một thao tác idempotent chạy lần hai cho trạng thái cuối như lần một. “Tạo thư mục nếu thiếu” idempotent; “nối một dòng báo cáo” không. Script định kỳ cần tránh nhân đôi.

Ví dụ xử lý tệp đầu vào bằng cách ghi kết quả theo content hash và đánh dấu ID đã xử lý. Hoặc thiết kế đầu ra được tái tạo toàn bộ từ nguồn trong tệp tạm rồi thay thế.

Chạy lại là một trường hợp bình thường

Máy có thể mất điện, người dùng có thể bấm hai lần, scheduler có thể retry. Hãy thiết kế câu trả lời cho “nếu chạy lại từ giữa thì sao” trước khi gọi tự động hóa là hoàn chỉnh.

24.12 argparse cho CLI

```
import argparse

def build_parser():
    parser = argparse.ArgumentParser(
        description="Phan loai tep theo phan mo rong"
    )
    parser.add_argument("folder", type=Path)
    parser.add_argument(
        "--apply", action="store_true",
        help="Thuc hien thay vi chi xem truoc",
    )
    return parser

def main(argv=None):
    args = build_parser().parse_args(argv)
    plan = plan_by_extension(args.folder)
    execute(plan, dry_run=not args.apply)
```

Mặc định dry-run; người dùng phải thêm `--apply` cho thao tác thật. `argv` truyền được giúp test parser mà không sửa `sys.argv`.

24.13 Tự động hóa bảng dữ liệu

Một script nhận nhiều CSV điểm, kiểm tra schema, thêm cột nguồn, gộp và xuất báo cáo. Các bước nên tách:

1. Khám phá tệp theo mẫu cố định.
2. Đọc từng tệp và ghi lỗi kèm đường dẫn/dòng.

3. Chuẩn hóa mã, kiểu số, tên cột.
4. Kiểm tra trùng khóa và quy tắc nghiệp vụ.
5. Tính báo cáo bằng hàm thuần.
6. Ghi tệp tạm, tạo summary và log.

Nếu một tệp lỗi, lựa chọn dừng toàn bộ hay bỏ qua phụ thuộc mục đích. Báo cáo tài chính có thể cần all-or-nothing; tập log khảo sát có thể chấp nhận quarantine dòng lỗi. Quy tắc phải được nói rõ.

24.14 Bảo mật trong script

Không nối trực tiếp dữ liệu người dùng thành lệnh shell. Dùng API Python hoặc `subprocess.run([...], check=True)` với danh sách đối số, không `shell=True`. Không log token. Giới hạn đường dẫn thao tác trong thư mục đã xác nhận và tránh theo symlink nếu mô hình đe dọa yêu cầu.

Tự động hóa xóa và ghi đè

Ưu tiên di chuyển vào thùng rác/quarantine, giữ manifest và có thời gian xem lại. Nếu bắt buộc xóa, yêu cầu đích cụ thể, từ chối thư mục gốc/rỗng, in số lượng và xác nhận. "Nhanh" không bù được dữ liệu không thể phục hồi.

24.15 Bài tập luyện tập

1. Viết công cụ đổi tên ảnh theo ngày và số thứ tự, dry-run mặc định.
2. Phát hiện file trùng bằng hash theo khối; không xóa, chỉ báo cáo nhóm.
3. Gộp nhiều CSV có cùng schema và thêm cột nguồn.
4. Tạo script backup chỉ sao chép tệp mới/thay đổi và sinh manifest hash.
5. Xây journal cho đổi tên hai pha và chức năng rollback.
6. Viết test trên thư mục tạm cho mọi thao tác; không dùng thư mục cá nhân thật.
7. Thêm logging xoay vòng và mã thoát CLI khác nhau cho lỗi dữ liệu, quyền, cấu hình.
8. Chọn một công việc tuần của bạn, viết đặc tả năm cột rồi ước lượng lợi ích/rủi ro trước khi code.

24.16 Tóm tắt cuối chương

Bản đồ kiến thức

Tự động hóa tốt tách lập kế hoạch khỏi thực thi, dry-run mặc định và kiểm tra xung đột trước thay đổi. Script cần chạy lại an toàn, có journal hoặc rollback cho chuỗi thao tác. Argparse tạo giao diện rõ; log và mã thoát giúp vận hành. Hãy tự động hóa quy trình đã hiểu, với biên an toàn tương xứng hậu quả.

Góc nhìn cá nhân

Tôi thích tự động hóa không phải vì nó giúp né công việc, mà vì nó buộc mình nói chính xác công việc là gì. Khi một quy trình không thể viết thành quy tắc, đôi khi vấn đề không nằm ở code; nó nằm ở việc chính ta vẫn đang quyết định bằng cảm giác ở mỗi lần lặp.

Chương 25

API và SQLite – Giao tiếp với dịch vụ, lưu dữ liệu có truy vấn

Giới thiệu ngắn

JSON trên đĩa phù hợp ứng dụng nhỏ. Khi cần lọc hàng nghìn công việc theo trạng thái, cập nhật một bản ghi, bảo đảm ID duy nhất và thực hiện nhiều thay đổi cùng nhau, cơ sở dữ liệu trở nên tự nhiên. API HTTP cho chương trình trao đổi với dịch vụ xa. Cả hai đều là ranh giới: dữ liệu có thể chậm, lỗi, không hợp lệ và cần giao dịch.

Mục tiêu học tập

- Hiểu request, response, status code và JSON ở mức client.
- Đặt timeout, kiểm tra lỗi và phân trang API.
- Tạo bảng, CRUD và truy vấn có tham số trong SQLite.
- Dùng transaction để giữ thay đổi nguyên tử.
- Tách repository khỏi logic nghiệp vụ.

25.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một ứng dụng lấy dữ liệu từ dịch vụ bên ngoài rồi lưu cục bộ. Mạng có thể timeout, API có giới hạn, schema có thể thiếu trường; cơ sở dữ liệu có thể từ chối ràng buộc. Thiết kế phải tách lời gọi xa không chắc chắn khỏi giao dịch cục bộ và quyết định khi nào dữ liệu đủ tin cậy để commit.

Một chương trình tốt không bắt đầu bằng việc hỏi “phải dùng câu lệnh nào?”. Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng

nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **API và SQLite như hai ranh giới dữ liệu có thất bại và giao dịch**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

25.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: endpoint, tham số, status, timeout, schema phản hồi, khóa chính, ràng buộc và biên giao dịch.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

25.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$\text{request} \to \text{validate} \to \text{transaction} \to \{\text{commit}, \text{rollback}\}$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: yêu cầu đang thử, dữ liệu đã xác nhận và transaction đang mở?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

25.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không

còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Chỉ commit dữ liệu đã qua kiểm tra; retry không được tạo bản ghi trùng và rollback phải đưa cơ sở dữ liệu về trạng thái trước giao dịch.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

25.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **API và SQLite như hai ranh giới dữ liệu có thất bại và giao dịch** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

25.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Đồng bộ danh sách nhiệm vụ từ API giả lập vào SQLite. Ta dùng mã ngoài làm khóa duy nhất, timeout hữu hạn, validate từng bản ghi, upsert trong một transaction và lưu thời điểm đồng bộ.

25.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

25.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

25.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

25.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

25.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

25.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

25.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Không đặt timeout cho request.
- Gọi API bên trong transaction kéo dài.
- Xây SQL bằng nối chuỗi.
- Retry POST không idempotent mà không có khóa yêu cầu.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

25.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

25.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Thiết kế schema và ràng buộc cho nhiệm vụ.
2. Vẽ các lớp lỗi HTTP.
3. Viết chiến lược retry có backoff.
4. Thử rollback khi bản ghi thứ ba sai.
5. Giải thích tham số hóa truy vấn.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

25.8 Một yêu cầu HTTP gồm gì?

Client gửi phương thức (GET, POST, PUT/PATCH, DELETE), URL, headers và có thể body. Server trả status, headers, body. Các nhóm mã: 2xx thành công, 4xx vấn đề yêu cầu/quyền, 5xx lỗi phía dịch vụ. Mã thành công không bảo đảm schema đúng; vẫn kiểm tra body.

Ví dụ với thư viện `urllib` chuẩn để hiểu nền:

```
import json
from urllib.request import Request, urlopen
from urllib.error import HTTPError, URLError

def get_json(url, timeout=10):
    request = Request(url, headers={"Accept": "application/json"})
    try:
        with urlopen(request, timeout=timeout) as response:
            charset = response.headers.get_content_charset() or "utf-8"
```

```
            body = response.read().decode(charset)
            return json.loads(body)
    except HTTPError as error:
        raise RuntimeError(f"HTTP {error.code}") from error
    except URLError as error:
        raise RuntimeError("Khong the ket noi") from error
```

Trong dự án thật, thư viện như `httpx` hoặc `requests` có API tiện hơn; vẫn phải đặt timeout và kiểm tra lỗi.

25.9 Không đặt bí mật trong mã

API key nên đến từ biến môi trường hoặc kho bí mật, không commit Git và không in log.

```
import os

token = os.environ.get("MY_API_TOKEN")
if not token:
    raise RuntimeError("Thieu MY_API_TOKEN")
```

Biến môi trường không giải quyết mọi bảo mật nhưng tách bí mật khỏi mã nguồn. Khi token lộ, thu hồi/rotate; xóa khỏi commit mới không xóa lịch sử Git.

25.10 Phân trang, rate limit và retry

API thường trả từng trang. Client phải theo con trỏ/link cho đến hết, đồng thời giới hạn số trang để tránh vòng vô hạn. Mã 429 có thể kèm thời gian chờ. Retry chỉ cho lỗi tạm thời, có backoff và jitter, tôn trọng giới hạn. POST không idempotent cần khóa idempotency do API hỗ trợ.

25.11 SQLite trong thư viện chuẩn

```
import sqlite3

def connect(path):
    connection = sqlite3.connect(path)
    connection.row_factory = sqlite3.Row
    connection.execute("PRAGMA foreign_keys = ON")
    return connection

with connect("study.db") as conn:
    conn.execute("""
        CREATE TABLE IF NOT EXISTS tasks (
            id TEXT PRIMARY KEY,
```

```
            title TEXT NOT NULL,
            priority INTEGER NOT NULL CHECK(priority BETWEEN 1 AND 3),
            done INTEGER NOT NULL DEFAULT 0,
            due TEXT
        )
    """)
```

SQLite lưu một tệp, không cần server, hỗ trợ transaction và SQL phong phú. Nó phù hợp ứng dụng cá nhân, prototype và nhiều workload đọc; không phải lựa chọn cho mọi hệ thống nhiều máy ghi đồng thời.

25.12 Truy vấn có tham số

```python
def add_task(conn, task):
    conn.execute(
        "INSERT INTO tasks(id, title, priority, done, due) "
        "VALUES (?, ?, ?, ?, ?)",
        (task.task_id, task.title, task.priority, task.done, task.due),
    )
```

Không dùng f-string để đưa giá trị vào SQL. Placeholder tách câu lệnh khỏi dữ liệu, xử lý escape và ngăn SQL injection. Tên bảng/cột không truyền như giá trị; nếu cần động, chọn từ allowlist.

SQL injection không cần người dùng ác ý

Một tiêu đề chứa dấu nháy đã có thể làm câu SQL nối chuỗi bị vỡ. Parameterized query vừa an toàn vừa đúng dữ liệu. Đây nên là thói quen mặc định, không phải bản vá sau sự cố.

25.13 CRUD và ánh xạ

```python
def list_pending(conn, limit=50):
    rows = conn.execute(
        """
        SELECT id, title, priority, done, due
        FROM tasks
        WHERE done = 0
        ORDER BY priority ASC, due IS NULL, due ASC
        LIMIT ?
        """,
        (limit,),
    ).fetchall()
    return [row_to_task(row) for row in rows]
```

```
13
14 def complete(conn, task_id):
15     cursor = conn.execute(
16         "UPDATE tasks SET done = 1 WHERE id = ? AND done = 0",
17         (task_id,),
18     )
19     if cursor.rowcount == 0:
20         raise KeyError(task_id)
```

`due IS NULL` đưa hạn thật trước giá trị null trong thứ tự này. Repository chuyển row sang domain object để phần nghiệp vụ không biết SQL.

25.14 Transaction

```
1 with connect("study.db") as conn:
2     debit(conn, source_id, amount)
3     credit(conn, destination_id, amount)
```

Context manager của connection commit khi block thành công và rollback khi exception. Transaction giữ nhóm thao tác all-or-nothing. Nhưng không nên giữ transaction mở trong lúc hỏi input hoặc gọi mạng lâu; khóa bị giữ và kết quả ngoài hệ thống không rollback cùng SQLite.

25.15 Migration schema

Không chỉnh bảng thủ công trên máy mỗi người. Giữ số phiên bản trong `PRAGMA user_version`, chạy các migration tuần tự trong transaction, sao lưu trước thay đổi lớn và test trên bản sao dữ liệu.

```
1 def migrate(conn):
2     version = conn.execute("PRAGMA user_version").fetchone()[0]
3     if version == 0:
4         create_schema_v1(conn)
5         conn.execute("PRAGMA user_version = 1")
6         version = 1
7     if version == 1:
8         conn.execute("ALTER TABLE tasks ADD COLUMN course TEXT")
9         conn.execute("PRAGMA user_version = 2")
10        version = 2
11    if version != 2:
12        raise RuntimeError(f"Unsupported DB version {version}")
```

25.16 Bài tập luyện tập

1. Viết client GET có timeout và thông báo riêng cho 404, 429, 5xx.
2. Đọc API phân trang giả từ tệp hoặc server test; ngăn con trỏ lặp.
3. Thiết kế bảng courses, tasks với foreign key và index phù hợp.
4. Viết CRUD đầy đủ bằng truy vấn tham số.
5. Test rollback khi thao tác thứ hai trong transaction lỗi.
6. Dùng `EXPLAIN QUERY PLAN` trước/sau index cho truy vấn thường dùng.
7. Viết migration thêm bảng `study_sessions` và tổng hợp phút theo course.
8. Tách Protocol TaskRepository, cài JsonRepository và SqliteRepository, chạy chung bộ contract test.

25.17 Tóm tắt cuối chương

Bản đồ kiến thức

API HTTP là ranh giới chậm và không tin cậy: đặt timeout, kiểm tra status/schema, phân trang và retry có giới hạn. SQLite cung cấp SQL và transaction trong một tệp. Luôn dùng tham số cho giá trị SQL, bật foreign key và migration có phiên bản. Repository tách lưu trữ khỏi quy tắc nghiệp vụ.

Góc nhìn cá nhân

Khi chương trình bắt đầu nói chuyện với mạng hoặc cơ sở dữ liệu, nó không còn toàn quyền quyết định thế giới. Dịch vụ có thể im lặng, schema có thể đổi, đĩa có thể đầy. Tôi xem mỗi ranh giới như nơi cần sự khiêm tốn kỹ thuật: timeout, kiểm tra, transaction và một lời báo lỗi trung thực.

Chương 26

Dự án 1: Trình quản lý công việc dòng lệnh

Bối cảnh dự án

Ta sẽ xây *Blanket Tasks*, một ứng dụng CLI cho phép thêm, liệt kê, bắt đầu, hoàn thành, xóa và tìm công việc. Dữ liệu lưu JSON an toàn; logic có test; giao diện có mã thoát. Dự án nhỏ đủ để hoàn thành trong vài buổi nhưng có cấu trúc có thể mang sang web.

Mục tiêu học tập

- Chuyển yêu cầu thành use case và tiêu chí chấp nhận.
- Tách domain, repository, service và CLI.
- Xây luồng lỗi không làm mất dữ liệu.
- Viết test đơn vị và test tích hợp tệp.
- Đóng gói dự án Overleaf không liên quan; mã nguồn có thể chạy độc lập.

26.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một trình quản lý việc cần thêm, liệt kê, hoàn thành và lưu dữ liệu qua nhiều lần chạy. Nếu bắt đầu bằng menu, logic nghiệp vụ sẽ dính vào input/print. Ta cần mô hình nhiệm vụ, use case, storage, giao diện CLI và chiến lược kiểm thử tách biệt.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào, đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **dự án CLI như quá trình chuyển yêu cầu thành kiến**

trúc và bằng chứng, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

26.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: lệnh, tham số, mã thoát, nhiệm vụ, trạng thái, định dạng lưu, lỗi người dùng và lỗi hệ thống.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

26.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$\text{CLI} \to \text{use case} \to \text{domain} \to \text{repository}$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: danh sách nhiệm vụ hợp lệ, giao dịch use case và kết quả cần trình bày?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

26.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Mọi thay đổi nhiệm vụ đi qua use case; CLI không sửa storage trực tiếp và lỗi phải tạo mã thoát có nghĩa.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

26.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **dự án CLI như quá trình chuyển yêu cầu thành kiến trúc và bằng chứng** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

26.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Xây luồng `add`: parser nhận đối số, use case kiểm tra tiêu đề, domain tạo nhiệm vụ, repository lưu atomic, presenter in mã mới. Mỗi lớp được kiểm thử ở ranh giới của nó.

26.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

26.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

26.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

26.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

26.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

26.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

26.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Đưa toàn bộ chương trình vào một vòng while.
- Dùng print làm giá trị trả về của nghiệp vụ.
- Không có khóa ổn định cho nhiệm vụ.
- Sửa tệp trực tiếp trước khi validation.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

26.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

26.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Viết user story và acceptance criteria.
2. Vẽ kiến trúc bốn lớp.
3. Thiết kế command add/list/done.
4. Kiểm thử CLI bằng mã thoát.
5. Thử tệp hỏng và phục hồi.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

26.8 Yêu cầu và điều không làm

Phiên bản 1 có:

- `add TITLE --priority 1|2|3 --due YYYY-MM-DD`
- `list --status todo|doing|done --course NAME`
- `start ID`, `complete ID`, `delete ID`
- `search QUERY`
- JSON có version, ghi tạm rồi thay; ID không trùng.

Không đồng bộ cloud, không đăng nhập, không GUI. Viết rõ "không làm" bảo vệ dự án khỏi nở vô hạn.

Tiêu chí cho complete: ID tồn tại; trạng thái phải doing; sau lệnh, tệp lưu done; nếu lưu lỗi, báo thất bại và không nói thành công; chạy list thấy trạng thái mới.

26.9 Cấu trúc

```
blanket_tasks/
    pyproject.toml
    src/blanket_tasks/
        __init__.py
        model.py
        repository.py
        service.py
        cli.py
        __main__.py
    tests/
        test_model.py
        test_service.py
        test_json_repository.py
```

26.10 Domain model

```python
from dataclasses import dataclass
from datetime import date
from enum import Enum

class Status(str, Enum):
    TODO = "todo"
    DOING = "doing"
    DONE = "done"

@dataclass
class Task:
    id: str
    title: str
    priority: int = 2
    status: Status = Status.TODO
    due: date | None = None
    course: str | None = None

    def __post_init__(self):
        self.title = self.title.strip()
        if not self.title:
            raise ValueError("Title must not be empty")
        if self.priority not in {1, 2, 3}:
            raise ValueError("Priority must be 1, 2 or 3")

    def start(self):
        if self.status is not Status.TODO:
```

```
            raise ValueError("Only todo task can start")
        self.status = Status.DOING

    def complete(self):
        if self.status is not Status.DOING:
            raise ValueError("Only doing task can complete")
        self.status = Status.DONE
```

Không để service tự gán status; đối tượng giữ chuyển trạng thái.

26.11 Repository contract và memory fake

```
from typing import Protocol

class TaskRepository(Protocol):
    def all(self) -> list[Task]: ...
    def get(self, task_id: str) -> Task | None: ...
    def save(self, task: Task) -> None: ...
    def delete(self, task_id: str) -> bool: ...

class MemoryRepository:
    def __init__(self, tasks=()):
        self.items = {task.id: task for task in tasks}

    def all(self):
        return list(self.items.values())

    def get(self, task_id):
        return self.items.get(task_id)

    def save(self, task):
        self.items[task.id] = task

    def delete(self, task_id):
        return self.items.pop(task_id, None) is not None
```

Fake giúp test nghiệp vụ không đụng đĩa. JsonRepository có contract test riêng.

26.12 Service use cases

```
from uuid import uuid4

class TaskService:
    def __init__(self, repository, id_factory=None):
        self.repository = repository
```

```
        self.id_factory = id_factory or (lambda: uuid4().hex[:8])

    def add(self, title, priority=2, due=None, course=None):
        task = Task(
            id=self.id_factory(),
            title=title,
            priority=priority,
            due=due,
            course=course,
        )
        if self.repository.get(task.id):
            raise RuntimeError("ID collision")
        self.repository.save(task)
        return task

    def complete(self, task_id):
        task = self._required(task_id)
        task.complete()
        self.repository.save(task)
        return task

    def _required(self, task_id):
        task = self.repository.get(task_id)
        if task is None:
            raise KeyError(task_id)
        return task
```

Tiêm `id_factory` giúp test ID cố định. Một vấn đề tinh tế: fake trả chính đối tượng; nếu `complete` thay đổi rồi save lỗi, trạng thái fake vẫn đổi. Repository thật có thể khác. Trong thiết kế nghiêm, dùng đối tượng bất biến hoặc unit of work/transaction. Dự án ghi nhận giới hạn thay vì giả vờ không có.

26.13 JSON repository

Tệp lưu object gồm version và tasks. Mapper chuyển date sang ISO, enum sang value. Mỗi thao tác save có thể đọc–sửa–ghi toàn bộ, phù hợp dữ liệu cá nhân nhỏ. Nếu nhiều tiến trình cùng ghi, có lost update; đây là điểm chuyển sang SQLite.

```
def task_to_data(task):
    return {
        "id": task.id,
        "title": task.title,
        "priority": task.priority,
        "status": task.status.value,
        "due": task.due.isoformat() if task.due else None,
        "course": task.course,
```

```
    }
```

JsonRepository tải vào dictionary theo ID, kiểm tra trùng, và ghi bằng tệp tạm. Không để JSONDecodeError lọt nguyên lên CLI; dịch thành RepositoryError có nguyên nhân.

26.14 CLI là adapter

```
def handle_complete(args, service):
    try:
        task = service.complete(args.id)
    except KeyError:
        print(f"Khong tim thay ID {args.id}")
        return 2
    except ValueError as error:
        print(f"Khong the hoan thanh: {error}")
        return 3
    except RepositoryError as error:
        print(f"Loi luu tru: {error}")
        return 4
    print(f"Da hoan thanh: {task.title}")
    return 0
```

Không in thành công trước save. Mã thoát 0 thành công, khác 0 cho automation nhận biết.

26.15 Test theo tầng

```
def test_complete_doing_task():
    task = Task("T1", "Hoc", status=Status.DOING)
    repo = MemoryRepository([task])
    service = TaskService(repo)

    result = service.complete("T1")

    assert result.status is Status.DONE
    assert repo.get("T1").status is Status.DONE
```

Test model cho chuyển trạng thái. Test service cho use case với fake. Test repository dùng thư mục tạm và round-trip. Test CLI có thể truyền argv, capture output và fake service. Một end-to-end test chạy vài lệnh trên tệp tạm.

26.16 Các mốc triển khai

1. Viết model và test chuyển trạng thái.
2. Viết repository protocol, memory fake.
3. Viết add/list/start/complete service và test.
4. Viết mapper và JsonRepository với round-trip, corrupted file.
5. Viết parser/handler, help và mã thoát.
6. Chạy scenario thật, sửa thông điệp.
7. Thêm README, ví dụ và checklist phát hành.

Tiêu chí hoàn thành

Một người khác có thể tạo môi trường từ README, chạy test, xem help và hoàn thành luồng thêm–bắt đầu–hoàn thành mà không hỏi tác giả. Tệp hỏng không bị ghi đè. Lệnh sai trả mã khác 0. Mọi hành vi cốt lõi có ít nhất một test thành công và một test thất bại.

26.17 Bài tập mở rộng

1. Thêm edit nhưng không cho đổi ID.
2. Thêm archive thay vì xóa cứng.
3. Xuất CSV và import có dry-run.
4. Thêm SQLite repository và chạy cùng contract tests.
5. Thêm recurring task nhưng viết rõ quy tắc ngày cuối tháng.
6. Tạo plugin sắp xếp bằng callable.
7. Thêm log có correlation ID cho mỗi lệnh.
8. Đo dữ liệu bao nhiêu task thì JSON toàn tệp không còn phù hợp.

26.18 Tóm tắt cuối chương

Bản đồ kiến thức

Dự án được chia theo domain–repository–service–CLI. Model giữ bất biến; repository che định dạng; service điều phối use case; CLI chuyển input/output. Fake làm test nhanh, contract test giữ các repository cùng hành vi. Lưu an toàn và báo lỗi trước thông điệp thành công là một phần chức năng, không phải phần phụ.

Góc nhìn cá nhân

Một dự án nhỏ hoàn thành dạy nhiều hơn một dự án lớn luôn ở trạng thái “sắp xong”. Giới hạn tính năng không làm ý tưởng nhỏ đi; nó tạo một đường biên để ta trải nghiệm trọn vòng đời từ yêu cầu, thiết kế, lỗi, kiểm thử đến lúc người khác có thể dùng.

Chương 27

Dự án 2: Hệ thống quản lý việc học cá nhân

Bối cảnh dự án

Ứng dụng cuối sách không cố trở thành một "siêu app". Nó trả lời ba câu hỏi thiết thực: hôm nay nên làm gì, thời gian đã đi đâu, và kế hoạch có đang vượt khả năng hay không. Hệ thống gồm courses, tasks, study sessions và prerequisites; lưu SQLite; CLI là giao diện đầu, còn lõi đủ sạch để sau này thêm web.

Mục tiêu học tập

- Đi từ vấn đề đến mô hình miền và kiến trúc.
- Thiết kế schema, transaction và báo cáo.
- Kết hợp đồ thị phụ thuộc với lịch công việc.
- Xây kế hoạch kiểm thử, migration, backup và phát hành.
- Đánh giá giới hạn đạo đức và trải nghiệm của gợi ý tự động.

27.1 Từ một tình huống thật đến mô hình có thể kiểm tra

Bài toán gốc trước khi có mã nguồn

Một hệ thống học tập không chỉ lưu danh sách việc. Nó phải nối mục tiêu dài hạn với phiên học hằng ngày, ghi bằng chứng, tính tiến độ và giúp người dùng điều chỉnh. Nếu mô hình chỉ tối đa số giờ, người học có thể bận rộn nhưng không tiến gần năng lực cần đạt.

Một chương trình tốt không bắt đầu bằng việc hỏi "phải dùng câu lệnh nào?". Nó bắt đầu bằng việc làm rõ thế giới mà chương trình đang mô tả. Trong thế giới ấy có đối tượng nào,

đại lượng nào thay đổi, quy tắc nào không được vi phạm và kết quả nào giúp người dùng ra quyết định? Với chủ đề **hệ thống quản lý học tập như mô hình mục tiêu–hành động–phản hồi**, nếu bỏ qua bước này, ta rất dễ viết được một đoạn mã chạy nhưng lại giải sai bài toán.

Ta tách quá trình suy nghĩ thành ba lớp. *Lớp thực tế* chứa câu chuyện, con người và các giới hạn thật. *Lớp mô hình* thay câu chuyện bằng dữ liệu, quan hệ và quy tắc. *Lớp chương trình* mới chuyển mô hình thành những thao tác Python. Mỗi khi kết quả sai, việc phân lớp giúp ta biết phải kiểm tra giả định thực tế, công thức hay cách cài đặt.

27.1.1 Bước 1: khoanh ranh giới của bài toán

Trước hết, hãy nói lại yêu cầu bằng một câu không chứa tên hàm Python. Câu ấy cần chỉ ra ai cung cấp dữ liệu, chương trình chịu trách nhiệm đến đâu và điều gì nằm ngoài phạm vi. Một mô hình nhỏ nhưng rõ ràng tốt hơn một mô hình tham lam cố giải mọi thứ. Khi phạm vi đã ổn định, ta mới lập danh sách các đại lượng cần quan sát: mục tiêu, tiêu chí hoàn thành, nhiệm vụ, phiên học, bằng chứng, thời lượng, phản hồi và chu kỳ rà soát.

Mỗi đại lượng phải đi kèm ý nghĩa và đơn vị. Một con số `30` có thể là 30 điểm, 30 phút hoặc 30 phần tử. Máy tính chỉ giữ mẫu bit; ý nghĩa nằm trong mô hình của người viết. Vì vậy, tên biến, kiểu dữ liệu và kiểm tra đầu vào không phải phần trang trí. Chúng là cách ta giữ cho ý nghĩa không bị đánh rơi khi bài toán đi qua nhiều bước xử lý.

27.1.2 Bước 2: viết quan hệ trước khi viết cú pháp

Mô hình tối thiểu của chương này có thể ghi dưới dạng:

$$\text{progress} = \frac{\text{evidence accepted}}{\text{evidence planned}}, \qquad \text{plan}_{t+1} = F(\text{plan}_t, \text{feedback}_t)$$

Công thức trên không nhất thiết là một phép tính số học. Nó có thể là ánh xạ từ đầu vào sang đầu ra, một quan hệ thứ tự, một điều kiện đúng–sai hoặc một phép biến đổi trạng thái. Điều quan trọng là ta đã có một mệnh đề độc lập với ngôn ngữ lập trình. Ta có thể thử nó bằng giấy bút, bằng vài trường hợp nhỏ, rồi mới chọn cấu trúc Python phù hợp.

Thành phần	Câu hỏi phải trả lời
Đầu vào	Dữ liệu đến từ đâu, có thể thiếu hoặc sai ở điểm nào?
Trạng thái	Thông tin nào phải được nhớ sau mỗi bước: kế hoạch hiện tại, phiên đã ghi, bằng chứng được chấp nhận và điều chỉnh đang chờ?
Quy tắc	Quan hệ nào biến trạng thái cũ thành trạng thái mới?
Đầu ra	Kết quả dành cho máy xử lý tiếp hay dành cho con người đọc?
Biên	Giá trị nhỏ nhất, lớn nhất, rỗng, trùng hoặc không hợp lệ là gì?

27.1.3 Bước 3: xác định bất biến

Bất biến là điều phải còn đúng trước và sau mỗi thao tác hợp lệ. Nó giống một hàng rào: bên trong hàng rào, chương trình có thể thay đổi rất nhiều; vượt qua hàng rào, dữ liệu không còn đáng tin. Với chương này, bất biến trung tâm là:

Điều không được phép đánh mất

Tiến độ chỉ tăng khi có bằng chứng gắn với tiêu chí; một phiên học không được tính hai lần và lịch sử không bị sửa âm thầm.

Khi một thuật toán có nhiều bước, hãy kiểm tra bất biến sau từng bước thay vì chỉ nhìn kết quả cuối. Cách làm này đặc biệt hữu ích khi dữ liệu có nhiều phần tử, khi chương trình có nhánh rẽ hoặc khi một đối tượng bị sửa tại chỗ. Nó biến câu nói mơ hồ "có vẻ đúng" thành một chuỗi bằng chứng nhỏ.

27.2 Mổ xẻ cơ chế: dữ liệu, trạng thái và phép biến đổi

Chủ đề **hệ thống quản lý học tập như mô hình mục tiêu–hành động–phản hồi** có thể được nhìn như một hệ thống nhận trạng thái S_t, nhận thêm một tác động a_t, rồi tạo trạng thái mới:

$$S_{t+1} = F(S_t, a_t).$$

Ký hiệu này rất gần với cách học hàm số ở phổ thông. F không chỉ là một công thức; nó là toàn bộ quy tắc xử lý. Nếu cùng S_t và a_t luôn cho cùng kết quả, ta có một phép biến đổi xác định. Nếu kết quả còn phụ thuộc tệp, thời gian, mạng hoặc người dùng, ta phải ghi những yếu tố ấy vào mô hình thay vì giả vờ chúng không tồn tại.

Ở mức chương trình, mỗi thao tác nên trả lời được bốn câu hỏi. Thao tác đọc gì? Nó thay đổi gì? Nó có thể thất bại thế nào? Sau khi kết thúc, điều gì được bảo đảm? Bốn câu hỏi này giúp người mới thoát khỏi thói quen đọc code theo từng dòng rời rạc. Ta bắt đầu nhìn được hợp đồng của cả một khối xử lý.

Một điểm quan trọng khác là phân biệt *giá trị* với *tên dùng để truy cập giá trị*. Trong Toán học, ký hiệu x thường đại diện cho một giá trị tại thời điểm xét. Trong chương trình, một tên có thể được gắn lại, nhiều tên có thể dẫn tới cùng một đối tượng, và một đối tượng có thể thay đổi. Khi chủ đề liên quan đến dữ liệu có thể sửa, sự khác biệt này giải thích nhiều lỗi tưởng như bí ẩn.

27.3 Một ca làm mẫu đi đủ các bước

Bối cảnh cần giải quyết

Thiết kế chu kỳ một tuần: đặt mục tiêu đo được, chia nhiệm vụ, ghi phiên, gắn artifact, tổng hợp và phản tư. Ta mô hình domain trước, xây repository, service báo cáo và CLI/web adapter sau.

27.3.1 Bước 1: phát biểu lại yêu cầu

Viết một câu theo mẫu: "Với dữ liệu đầu vào đã được kiểm tra, hệ thống phải tạo ra kết quả nào và giữ điều kiện nào luôn đúng?" Việc phát biểu lại buộc ta tách nhu cầu thật khỏi chi tiết giao diện. Nếu chưa thể nói rõ bằng lời, việc viết code lúc này chỉ che sự mơ hồ bằng cú pháp.

27.3.2 Bước 2: dựng ví dụ nhỏ bằng tay

Chọn trường hợp đủ nhỏ để có thể mô phỏng mà không cần máy. Ghi trạng thái ban đầu, thực hiện đúng một thao tác, rồi ghi trạng thái mới. Lặp lại cho đến kết quả. Bảng truy vết nên có dạng:

Bước	Trạng thái trước	Quy tắc áp dụng	Trạng thái sau
0	Dữ liệu vừa được tiếp nhận	Kiểm tra tiền điều kiện	Dữ liệu hợp lệ
1	Trạng thái hợp lệ	Thực hiện phép biến đổi chính	Trạng thái trung gian
2	Trạng thái trung gian	Kiểm tra bất biến và biên	Kết quả có thể công bố

Bảng này không nhằm thay code. Nó là bản thiết kế để sau này đối chiếu. Khi chương trình cho kết quả khác, ta so sánh từng hàng và tìm đúng bước đầu tiên hai quá trình lệch nhau.

27.3.3 Bước 3: chọn biểu diễn dữ liệu

Không có kiểu dữ liệu tốt một cách tuyệt đối; chỉ có kiểu dữ liệu phù hợp với thao tác chủ đạo. Nếu thường xuyên truy cập theo vị trí, biểu diễn có thứ tự là tự nhiên. Nếu tra cứu theo khóa, cần một ánh xạ. Nếu mục tiêu là loại trùng, cần một cấu trúc thể hiện tính duy nhất. Nếu dữ liệu chỉ là một giá trị, đừng dựng cấu trúc lớn hơn mức cần thiết.

Khi chọn biểu diễn, phải xét cả dữ liệu xấu. Dữ liệu rỗng có hợp lệ không? Hai bản ghi giống nhau được giữ hay gộp? Thứ tự có mang ý nghĩa không? Giá trị có được phép thay đổi sau khi tạo? Những câu hỏi đó quyết định cấu trúc trước cả khi ta nghĩ đến tối ưu hiệu năng.

27.3.4 Bước 4: viết thuật toán bằng động từ

Một thuật toán sáng rõ thường có thể diễn đạt bằng các động từ tuần tự: nhận, kiểm tra, chuẩn hóa, biến đổi, kiểm chứng và trình bày. Mỗi động từ nên tương ứng với một khối trách nhiệm. Nếu một bước vừa đọc dữ liệu, vừa sửa trạng thái, vừa in kết quả và vừa bắt mọi lỗi, nó đang mang quá nhiều vai trò; hãy tách nhỏ trước khi chuyển sang hàm hoặc lớp.

27.3.5 Bước 5: chứng minh đủ dùng

Ta không cần một chứng minh hình thức dài cho mọi chương trình nhập môn, nhưng cần ít nhất ba mảnh lập luận. Thứ nhất, mọi đầu vào hợp lệ đều đi được qua các bước. Thứ hai, bất biến vẫn đúng sau mỗi phép biến đổi. Thứ ba, quá trình có điều kiện kết thúc. Ba mảnh này tương ứng với tính đầy đủ, tính đúng và tính dừng của thuật toán.

27.4 Kiểm thử là thí nghiệm của mô hình

Một ví dụ chạy đúng chỉ chứng minh chương trình đúng với đúng ví dụ đó. Muốn có niềm tin mạnh hơn, hãy thiết kế tập kiểm thử theo lớp tương đương: trường hợp bình thường, trường hợp biên, dữ liệu rỗng, dữ liệu lặp, dữ liệu sai kiểu và dữ liệu lớn. Mỗi kiểm thử phải nêu trước kết quả mong đợi; nếu chạy trước rồi mới quyết định "kết quả ấy có vẻ được", ta đang để chương trình tự chấm chính nó.

Có thể xem kiểm thử như thí nghiệm khoa học. Giả thuyết là "thuật toán giữ bất biến và tạo hậu điều kiện". Dữ liệu thử là cách can thiệp. Kết quả quan sát được so với dự đoán. Khi sai, ta thu hẹp phạm vi, tìm bước đầu tiên lệch và sửa mô hình hoặc cài đặt. Cách nhìn này quan trọng hơn việc nhớ tên một thư viện kiểm thử.

Năm câu hỏi trước khi tin kết quả

1. Đầu vào đã được kiểm tra về kiểu, miền giá trị và ý nghĩa chưa?
2. Trường hợp nhỏ nhất và trường hợp rỗng cho kết quả gì?
3. Bất biến có được kiểm tra tại điểm dữ liệu thay đổi không?
4. Kết quả có thể được tính lại bằng một cách độc lập hay không?
5. Khi thất bại, thông báo có chỉ ra nguyên nhân và dữ liệu liên quan không?

27.5 Sai rồi sửa: những nhầm lẫn cần phát hiện sớm

Các đường tắt làm mất phần suy luận

- Đồng nhất thời gian ngồi học với tiến bộ.
- Cho phép sửa lịch sử mà không có dấu vết.
- Đưa mọi dữ liệu vào một bảng không có ràng buộc.

- Xây dashboard trước khi định nghĩa chỉ số.

Điểm chung của các lỗi trên là nhảy từ đề bài sang cú pháp. Cách sửa không phải học thuộc thêm một câu lệnh, mà quay lại mô hình: viết rõ đại lượng, trạng thái, bất biến và trường hợp biên. Khi bốn thành phần ấy đã rõ, lỗi cú pháp chỉ còn là lớp ngoài cùng và thường dễ xử lý hơn nhiều.

27.6 Vận dụng và mở rộng mô hình

Sau khi giải được trường hợp cơ bản, hãy thay đổi đúng một giả định. Dữ liệu lớn hơn thì điều gì trở thành nút thắt? Dữ liệu đến từng phần thay vì có sẵn thì trạng thái nào phải giữ? Nhiều người cùng cập nhật thì xung đột xuất hiện ở đâu? Kết quả cần lưu qua lần chạy sau thì định dạng và lỗi I/O thay đổi thế nào? Đây là cách một bài tập lớp học tiến dần thành một hệ thống thật.

Ta cũng nên phân biệt tối ưu cục bộ và thiết kế tổng thể. Rút ngắn một dòng code không có ý nghĩa nếu khiến mô hình khó kiểm tra. Ngược lại, một phiên bản dài hơn nhưng có bước chuẩn hóa, kiểm tra bất biến và tách trách nhiệm thường đáng tin hơn. Chỉ tối ưu tốc độ sau khi đã đo và biết phần nào thực sự tốn chi phí.

27.7 Bài tập ba tầng: từ hiểu đến tự thiết kế

1. Viết tiêu chí hoàn thành cho một mục tiêu Python.
2. Thiết kế schema mục tiêu–phiên–bằng chứng.
3. Tạo công thức tiến độ không đếm trùng.
4. Viết retrospective từ dữ liệu tuần.
5. Lập kế hoạch kiểm thử end-to-end.

Với mỗi bài, không nộp code ngay. Hãy nộp kèm bốn dòng: đầu vào, đầu ra, bất biến và ba ca kiểm thử. Ở tầng cuối, bổ sung một đoạn giải thích vì sao biểu diễn dữ liệu được chọn phù hợp với thao tác chính. Yêu cầu này biến bài tập từ "gõ cho chạy" thành luyện tư duy thiết kế.

27.8 Khám phá vấn đề trước tính năng

Một người học không thiếu danh sách việc; họ thiếu một cách nhìn khối lượng trong thời gian có hạn. Vì vậy, chỉ số chính không phải số task tạo ra, mà là tỉ lệ hoàn thành kế hoạch vừa sức và khả năng nhìn lại thời gian thật.

Các câu chuyện người dùng:

– Tôi ghi một task với môn, hạn, ước lượng và ưu tiên.

- Tôi bắt đầu/kết thúc study session và gắn task/môn.
- Tôi xem tổng phút theo tuần, môn và chênh lệch ước lượng–thực tế.
- Tôi khai báo chủ đề tiên quyết và biết thứ tự học hợp lệ.
- Tôi nhận cảnh báo khi tải kế hoạch vượt số phút rảnh, không nhận phán xét.

Không biến thời gian học thành thước đo giá trị cá nhân. Báo cáo mô tả dữ liệu, cho phép ngày nghỉ và dữ liệu thiếu.

27.9 Mô hình miền

Các thực thể:

- **Course**: ID, tên, màu, trạng thái.
- **Task**: tiêu đề, course, deadline, estimate, priority, status.
- **StudySession**: start, end, course, task tùy chọn, note.
- **Topic**: course, tên; cạnh prerequisite giữa topics.
- **Availability**: ngày, số phút dự kiến có thể dùng.

Bất biến: end sau start; estimate dương; task done có completed_at; cạnh tiên quyết không tự nối; ID tham chiếu tồn tại.

27.10 Schema SQLite

```
CREATE TABLE courses (
    id TEXT PRIMARY KEY,
    name TEXT NOT NULL UNIQUE,
    archived INTEGER NOT NULL DEFAULT 0
);

CREATE TABLE tasks (
    id TEXT PRIMARY KEY,
    course_id TEXT NOT NULL REFERENCES courses(id),
    title TEXT NOT NULL,
    due TEXT,
    estimate_minutes INTEGER CHECK(estimate_minutes > 0),
    priority INTEGER NOT NULL CHECK(priority BETWEEN 1 AND 3),
    status TEXT NOT NULL CHECK(status IN ('todo','doing','done')),
    completed_at TEXT
);

CREATE TABLE study_sessions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    course_id TEXT NOT NULL REFERENCES courses(id),
    task_id TEXT REFERENCES tasks(id),
    started_at TEXT NOT NULL,
    ended_at TEXT NOT NULL,
    note TEXT,
    CHECK(ended_at > started_at)
);
```

SQLite so sánh chuỗi ISO cùng múi giờ đúng thứ tự. Ứng dụng nên lưu UTC có timezone và chuyển sang giờ địa phương lúc hiển thị. CHECK không thể biểu đạt mọi quy tắc; service và test bổ sung.

27.11 Use case ghi phiên học nguyên tử

Khi kết thúc session, hệ thống thêm bản ghi và có thể cập nhật task. Hai thay đổi cùng transaction.

```
def finish_session(conn, session, complete_task=False):
    with conn:
        conn.execute(
            """INSERT INTO study_sessions
               (course_id, task_id, started_at, ended_at, note)
               VALUES (?, ?, ?, ?, ?)""",
            session.to_row(),
        )
        if complete_task and session.task_id:
            cursor = conn.execute(
                """UPDATE tasks
                   SET status='done', completed_at=?
                   WHERE id=? AND status!='done'""",
                (session.ended_at, session.task_id),
            )
            if cursor.rowcount != 1:
                raise ValueError("Task khong ton tai hoac da xong")
```

Nếu update thất bại, insert rollback. Lệnh giao diện chỉ báo thành công sau block.

27.12 Báo cáo tuần bằng truy vấn và hàm thuần

```
SELECT c.name,
       COUNT(s.id) AS session_count,
       SUM((julianday(s.ended_at) - julianday(s.started_at)) * 1440)
           AS total_minutes
FROM courses c
LEFT JOIN study_sessions s
```

```
  ON s.course_id = c.id
 AND s.started_at >= ? AND s.started_at < ?
GROUP BY c.id, c.name
ORDER BY total_minutes DESC;
```

Làm tròn ở Python. Chênh lệch ước lượng chỉ tính task có session liên kết và cần quy tắc phân bổ khi một task có nhiều session. Hãy ghi định nghĩa cạnh biểu đồ; số liệu không tự giải thích mình.

27.13 Gợi ý kế hoạch vừa sức

Một heuristic minh bạch:

1. Lọc task chưa xong, prerequisite đã đủ.
2. Tính điểm từ độ khẩn, priority và thời gian chờ.
3. Sắp theo điểm, sau đó chọn task cho đến khi tổng estimate đạt 80% availability.
4. Dành 20% làm buffer; task quá lớn được đề nghị chia, không tự cắt.

```
def urgency(task, today):
    if task.due is None:
        due_score = 0
    else:
        days = (task.due - today).days
        due_score = 100 if days < 0 else max(0, 30 - days)
    return due_score + (4 - task.priority) * 10
```

Đây không phải AI và không nên được trình bày như chân lý. Người dùng thấy vì sao một task được chọn, thay estimate và bỏ qua gợi ý. Sau này mô hình học máy cũng cần cùng yêu cầu minh bạch, quyền kiểm soát và đánh giá tác hại.

27.14 Đồ thị prerequisites

Bảng `topic_prerequisites(topic_id, prerequisite_id)` tạo cạnh "prerequisite trước topic". Khi thêm cạnh, chạy DFS/BFS kiểm tra liệu từ prerequisite đã đi đến topic theo chiều ngược tạo chu trình. Có thể thực hiện trong service trước insert và giữ unique constraint.

Khi sinh lộ trình, dùng topological sort trên các topic chưa hoàn thành. Nếu nhiều nút indegree 0, ưu tiên theo course đang học hoặc độ dài ước lượng, nhưng vẫn giữ tất cả thứ tự hợp lệ về logic.

27.15 Kiến trúc và ranh giới

`study_system/`

```
domain/       # entity, value object, rule
application/  # use case, ports
adapters/
    sqlite/   # repository, migration
    cli/      # parser, formatter
    clock.py
tests/
```

Domain không import sqlite3 hay argparse. Application phụ thuộc các giao diện repository và clock. Adapter phụ thuộc vào application, domain và triển khai chi tiết. Đây là hướng phụ thuộc vào trong.

27.16 Kế hoạch test

- Unit: trạng thái task, duration, urgency, planner, topo.
- Repository contract: CRUD, unique, foreign key, rollback.
- Migration: tạo DB v1, thêm dữ liệu, nâng lên phiên bản mới và kiểm tra giữ dữ liệu.
- Integration: finish session + complete task nguyên tử.
- CLI: parser, exit code, thông điệp lỗi.
- Property: kế hoạch không vượt capacity; topo luôn đặt prerequisite trước.

Kiểm thử timezone tại DST nếu đối tượng dùng khu vực có đổi giờ; Việt Nam hiện UTC+7 không DST, nhưng dữ liệu có thể chuyển vùng. Chọn thư viện zoneinfo và lưu UTC.

27.17 Backup, export và quyền riêng tư

Dữ liệu việc học có thể tiết lộ lịch sinh hoạt. Mặc định lưu cục bộ, không telemetry. Export JSON/CSV do người dùng chủ động. Backup SQLite nên dùng API backup hoặc đảm bảo không copy tệp giữa lúc ghi. Có lệnh `doctor` chạy integrity check, kiểm schema và báo vị trí DB.

27.18 Lộ trình tám mốc

1. Viết problem statement, use cases, điều không làm.
2. Model và unit tests.
3. Schema v1, migration runner, repository contract.
4. Course/task CLI.
5. Timer/session và báo cáo tuần.
6. Prerequisite graph và lộ trình.

7. Planner minh bạch cùng capacity/buffer.
8. Backup, export, doctor, README và bản phát hành 0.1.

Mỗi mốc kết thúc bằng demo có thể chạy và commit nhỏ. Đừng chờ toàn hệ thống mới tích hợp.

Bài bảo vệ dự án

Chuẩn bị một demo 7 phút: vấn đề 45 giây; thêm course/task; chạy planner và giải thích điểm; ghi session; xem báo cáo; cố nhập dữ liệu sai để thấy lỗi; mở sơ đồ kiến trúc; chạy test. Sau demo, trình bày ba giới hạn và kế hoạch phiên bản tiếp theo. Khả năng nói điều chưa làm là một phần của kỹ thuật.

27.19 Bài tập phát triển sau sách

1. Thêm giao diện web nhưng giữ nguyên application service.
2. Đồng bộ nhiều thiết bị: nghiên cứu conflict, vector clock hoặc chiến lược server-authoritative trước khi code.
3. Thêm Pomodoro nhưng không làm session phụ thuộc cứng vào 25 phút.
4. Tạo báo cáo calibration estimate: sai số trung vị theo course và loại task.
5. Thêm import từ calendar theo adapter; không để domain biết API cụ thể.
6. Viết benchmark 100.000 sessions và index theo truy vấn thật.
7. Thêm encryption/OS permissions sau khi viết threat model.
8. Thiết kế nghiên cứu nhỏ: planner có buffer có giảm số task trễ không? Xác định dữ liệu và quyền riêng tư.

27.20 Tóm tắt cuối chương

Bản đồ kiến thức

Dự án cuối kết hợp model, bất biến, SQLite transaction, báo cáo, đồ thị và heuristic. Bắt đầu từ vấn đề, giới hạn scope và định nghĩa số liệu. Kiến trúc giữ domain độc lập adapter; test theo tầng và contract. Gợi ý phải minh bạch, có buffer và quyền bỏ qua. Backup, migration, quyền riêng tư là chức năng cốt lõi của sản phẩm giữ dữ liệu cá nhân.

Góc nhìn cá nhân

Điều tôi muốn giữ lại sau dự án này không phải số dòng code. Đó là khả năng đi từ một nỗi khó chịu thật đến một hệ thống vừa đủ: biết mình đo gì, giữ dữ liệu nào, thất bại ra sao và không hứa điều chưa làm được. Một sản phẩm trưởng thành bắt đầu khi người viết chấp nhận rằng người dùng sẽ trao cho nó một phần đời sống thật.

Lời kết

Bạn bắt đầu Tập 2 bằng một danh sách. Từ chiếc kệ nhỏ ấy, dữ liệu dần có tên, có hình dạng, có tệp để ở lại, có ngoại lệ để nói điều sai và có test để giữ điều đúng. Thuật toán cho bạn cách nhìn đường tăng của công việc; lớp và giao thức cho bạn cách đặt ranh giới; generator và async cho bạn cách sống với dữ liệu lớn và thời gian chờ; hai dự án gom chúng thành những quyết định có hậu quả.

Nếu cuốn sách hoàn thành mục tiêu, bạn không chỉ có thêm cú pháp. Bạn có thể mở một bài toán chưa biết, viết đầu vào/đầu ra, chọn cấu trúc, nêu bất biến, xây ví dụ nhỏ, đo khi cần và để lỗi đủ rõ. Bạn cũng biết lúc nào nên dừng thêm tính năng để hoàn thiện thứ đang có.

Tập 1 đặt câu hỏi: "Làm thế nào để nói với máy tính?" Tập 2 đặt câu hỏi: "Làm thế nào để điều mình nói vẫn đúng khi chương trình lớn lên?" Câu trả lời không nằm ở một pattern hay thư viện. Nó nằm trong thói quen kiểm chứng từng giả định và sự tử tế với người sẽ đọc, dùng, sửa chương trình sau mình.

Hãy chọn một dự án trong sách, làm nó thành sản phẩm của riêng bạn, rồi thay một yêu cầu. Chính lúc phải thay đổi mà không đập bỏ mọi thứ, bạn sẽ biết phần nào của cuốn sách đã thật sự ở lại.

Phụ lục A

Gợi ý và lời giải chọn lọc

Phần này không nhằm biến bài tập thành việc chép mã. Mỗi lời giải chỉ là một đường đi, thường ưu tiên độ rõ và khả năng kiểm tra. Trước khi đọc, hãy viết ít nhất đầu vào, đầu ra, ba trường hợp biên và mã giả. Sau khi đọc, đóng sách và viết lại bằng tên biến của bạn. Một lời giải chỉ thực sự có ích khi bạn có thể thay yêu cầu mà không phải nhìn lại từng dòng.

Chương 1 – Chia danh sách thành lô

Hợp đồng: kích thước là số nguyên dương; đầu vào không bị sửa; lô cuối có thể ngắn.

```
def chia_lo(values, size):
    if not isinstance(size, int) or isinstance(size, bool):
        raise TypeError("size phai la so nguyen")
    if size <= 0:
        raise ValueError("size phai duong")
    return [values[i:i + size] for i in range(0, len(values), size)]

assert chia_lo([1, 2, 3, 4, 5], 2) == [[1, 2], [3, 4], [5]]
assert chia_lo([], 3) == []
```

Với dữ liệu rất lớn, thay list kết quả bằng generator `yield values[i:i+size]`. Nếu đầu vào chỉ là iterable không có chỉ số, tích lũy một buffer và yield khi đủ.

Chương 2 – Pangram

```
import string

def is_pangram(text):
    present = {char for char in text.casefold() if char in string.
    ascii_lowercase}
    return present == set(string.ascii_lowercase)

assert is_pangram("The quick brown fox jumps over the lazy dog")
assert not is_pangram("Python")
```

Ta chủ động định nghĩa pangram tiếng Anh ASCII. Nếu mục tiêu là một bảng chữ cái khác, truyền alphabet vào thay vì giấu quy tắc trong hàm.

Chương 3 – So sánh hai hồ sơ

```python
MISSING = object()

def diff_dict(old, new):
    changes = {}
    for key in old.keys() | new.keys():
        before = old.get(key, MISSING)
        after = new.get(key, MISSING)
        if before != after:
            changes[key] = {
                "old": None if before is MISSING else before,
                "new": None if after is MISSING else after,
                "old_missing": before is MISSING,
                "new_missing": after is MISSING,
            }
    return changes
```

Hai cờ thiếu giúp phân biệt "không có khóa" với "khóa có giá trị None". Với cấu trúc lồng, cần diff đệ quy và xử lý list theo chính sách miền.

Chương 4 – Kiểm tra mật khẩu có báo đủ lỗi

```python
def password_errors(password):
    errors = []
    if len(password) < 12:
        errors.append("Can it nhat 12 ky tu")
    if not any(c.islower() for c in password):
        errors.append("Can chu thuong")
    if not any(c.isupper() for c in password):
        errors.append("Can chu hoa")
    if not any(c.isdigit() for c in password):
        errors.append("Can chu so")
    return errors
```

Đây chỉ là kiểm tra hình thức, không đo mật khẩu đã bị lộ hay khả năng chống đoán. Trong sản phẩm, ưu tiên độ dài, password manager, kiểm tra danh sách mật khẩu rò rỉ theo cách bảo vệ riêng tư và MFA; không tự lưu mật khẩu thô.

Chương 5 – Tổng hàng và cột ma trận

```
def matrix_sums(matrix):
    if not matrix:
        return [], []
    width = len(matrix[0])
    if any(len(row) != width for row in matrix):
        raise ValueError("Ma tran khong chu nhat")

    row_sums = [sum(row) for row in matrix]
    column_sums = [sum(column) for column in zip(*matrix)]
    return row_sums, column_sums
```

Nếu hàng đầu rỗng, mọi hàng phải rỗng và tổng cột là list rỗng. Hợp đồng cần nói có chấp nhận ma trận $n \times 0$ không.

Chương 6 – N dòng dài nhất mà không đọc toàn bộ

```
import heapq
from pathlib import Path

def longest_lines(path, count=10):
    if count < 0:
        raise ValueError("count khong am")
    heap = []
    with Path(path).open(encoding="utf-8") as file:
        for number, line in enumerate(file, 1):
            item = (len(line.rstrip("\n")), number, line.rstrip("\n"))
            if len(heap) < count:
                heapq.heappush(heap, item)
            elif item > heap[0]:
                heapq.heapreplace(heap, item)
    return sorted(heap, reverse=True)
```

Bộ nhớ tỉ lệ k, không phải số dòng. Tuple dùng số dòng làm khóa hòa. Với `count=0`, điều kiện hiện tại cần trả sớm để tránh truy cập `heap[0]`; hãy thêm `if count == 0: return []`. Đây là ví dụ một biên dễ bỏ sót.

Chương 7 – Import CSV có vùng cách ly lỗi

```
import csv

def import_rows(path):
    valid = []
```

```
    errors = []
    with open(path, encoding="utf-8", newline="") as file:
        reader = csv.DictReader(file)
        required = {"id", "title", "priority"}
        missing = required - set(reader.fieldnames or [])
        if missing:
            raise ValueError(f"Thieu cot: {sorted(missing)}")

        for line, row in enumerate(reader, 2):
            try:
                priority = int(row["priority"])
                if priority not in {1, 2, 3}:
                    raise ValueError("priority")
                title = row["title"].strip()
                if not title:
                    raise ValueError("title")
                valid.append({**row, "title": title, "priority": priority})
            except (ValueError, TypeError) as error:
                errors.append({"line": line, "row": row, "error": str(error)})
    return valid, errors
```

Chỉ quarantine khi nghiệp vụ cho phép dùng một phần. Trước khi ghi, có thể yêu cầu tỉ lệ lỗi dưới ngưỡng và xuất báo cáo để người dùng xác nhận.

Chương 8 – Số lần thử hữu hạn

```
class TooManyAttemptsError(Exception):
    pass

def read_int(prompt, attempts=3, input_fn=input):
    if attempts < 1:
        raise ValueError("attempts phai >= 1")
    last_error = None
    for _ in range(attempts):
        try:
            return int(input_fn(prompt))
        except ValueError as error:
            last_error = error
    raise TooManyAttemptsError("Khong nhan duoc so nguyen") from last_error
```

Tiêm `input_fn` giúp test bằng iterator chuỗi. Ngoại lệ cuối giữ nguyên nhân chuyển đổi gần nhất.

Chương 9 – Hướng phụ thuộc package

Một cấu trúc tối thiểu nên tuân thủ:

Tầng	Được import	Không nên import
domain	thư viện chuẩn thuần	CLI, sqlite, HTTP
application	domain, protocol	argparse, chi tiết DB
adapter	application, domain	adapter khác nếu không cần
entrypoint	mọi adapter để lắp ghép	quy tắc nghiệp vụ mới

Nếu vòng import xuất hiện, thường có khái niệm chung đặt sai tầng hoặc hai module biết quá nhiều. Đừng chữa bằng import bên trong hàm trước khi hiểu vòng trách nhiệm.

Chương 10 – Test kho JSON trong thư mục tạm

```
import tempfile
import unittest
from pathlib import Path

class TestJsonStore(unittest.TestCase):
    def test_round_trip(self):
        with tempfile.TemporaryDirectory() as folder:
            path = Path(folder) / "data.json"
            store = JsonStore(path)
            expected = {"version": 2, "tasks": [{"id": "T1"}]}

            store.save(expected)
            actual = store.load()

            self.assertEqual(actual, expected)

    def test_corrupt_json_is_not_treated_as_empty(self):
        with tempfile.TemporaryDirectory() as folder:
            path = Path(folder) / "data.json"
            path.write_text("{broken", encoding="utf-8")
            with self.assertRaises(ValueError):
                JsonStore(path).load()
```

Test thứ hai bảo vệ một quy tắc quan trọng: dữ liệu hỏng không bị coi là kho rỗng, vì lần save sau sẽ ghi đè mất cơ hội phục hồi.

Chương 11 – Giao hai danh sách

```
def intersection_preserving_order(left, right):
    allowed = set(right)
```

```
    emitted = set()
    result = []
    for item in left:
        if item in allowed and item not in emitted:
            result.append(item)
            emitted.add(item)
    return result
```

Thời gian trung bình $O(n + m)$, bộ nhớ $O(m + r)$. Khác với `list(set(left) & set(right))`, nó giữ thứ tự lần xuất hiện đầu ở list trái. Định nghĩa "giao" trong sản phẩm phải nói có giữ trùng và thứ tự không.

Chương 12 – Vị trí đầu tiên bằng lower bound

```
def first_position(values, target):
    left, right = 0, len(values)
    while left < right:
        mid = (left + right) // 2
        if values[mid] < target:
            left = mid + 1
        else:
            right = mid
    if left < len(values) and values[left] == target:
        return left
    return -1
```

Bất biến: mọi vị trí trước `left` nhỏ hơn target; mọi vị trí từ `right` trở đi lớn hơn hoặc bằng target. Khi hai biên gặp, đó là vị trí chèn trái.

Chương 13 – Máy in dùng queue

```
from collections import deque
from dataclasses import dataclass

@dataclass
class PrintJob:
    name: str
    pages: int
    arrived_at: int

def simulate_printer(arrivals, pages_per_tick=1):
    queue = deque()
    completed = []
    current = None
```

```
14
15      for tick, new_jobs in enumerate(arrivals):
16          queue.extend(new_jobs)
17          if current is None and queue:
18              current = queue.popleft()
19          if current is not None:
20              current.pages -= pages_per_tick
21              if current.pages <= 0:
22                  completed.append((current.name, tick + 1 - current.arrived_at))
23                  current = None
24      return completed, current, list(queue)
```

Lời giải đang thay đổi object PrintJob đầu vào. Có thể copy để tránh tác dụng phụ. Ngoài ra, một job hoàn thành giữa tick không cho job sau dùng phần công suất dư; đó là giả định mô phỏng cần ghi rõ.

Chương 14 – Sinh ngoặc cân bằng

```
1  def balanced_parentheses(pairs):
2      result = []
3
4      def backtrack(text, opened, closed):
5          if len(text) == 2 * pairs:
6              result.append(text)
7              return
8          if opened < pairs:
9              backtrack(text + "(", opened + 1, closed)
10         if closed < opened:
11             backtrack(text + ")", opened, closed + 1)
12
13     backtrack("", 0, 0)
14     return result
```

Điều kiện `closed < opened` cắt mọi tiền tố có nhiều dấu đóng hơn mở. Chuỗi bất biến tạo bản mới; với n lớn, list ký tự và hoàn tác có thể giảm sao chép nhưng số kết quả vẫn tăng theo số Catalan.

Chương 15 – BFS trên lưới

```
1  from collections import deque
2
3  def grid_path(grid, start, goal):
4      rows = len(grid)
5      cols = len(grid[0]) if rows else 0
```

```
    queue = deque([start])
    parent = {start: None}

    while queue:
        row, col = queue.popleft()
        if (row, col) == goal:
            break
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nxt = (row + dr, col + dc)
            r, c = nxt
            if (0 <= r < rows and 0 <= c < cols
                    and grid[r][c] != "#" and nxt not in parent):
                parent[nxt] = (row, col)
                queue.append(nxt)
    else:
        return None

    path = []
    node = goal
    while node is not None:
        path.append(node)
        node = parent[node]
    return path[::-1]
```

Đánh dấu lúc đưa vào queue, không đợi lúc lấy ra, để một ô không được xếp nhiều lần.

Chương 16 – GradeBook và trách nhiệm

```
class GradeBook:
    def __init__(self):
        self._students = {}

    def add_student(self, student):
        if student.student_id in self._students:
            raise ValueError("Duplicate student ID")
        self._students[student.student_id] = student

    def record(self, student_id, subject, score):
        try:
            student = self._students[student_id]
        except KeyError as error:
            raise KeyError(f"Unknown student {student_id}") from error
        student.add_score(subject, score)
```

Student bảo vệ miền điểm và cách lưu điểm một người. GradeBook bảo vệ ID duy nhất

và tra cứu tập hợp. Trách nhiệm đặt tại nơi có đủ dữ liệu để duy trì bất biến.

Chương 17 – Chuyển tiền nguyên tử trong bộ nhớ

```
def transfer(source, destination, amount):
    if source is destination:
        raise ValueError("Hai tai khoan phai khac nhau")
    if amount <= 0:
        raise ValueError("Amount phai duong")
    if source.balance < amount:
        raise ValueError("Khong du so du")

    # Sau khi moi dieu kien da qua, hai thao tac duoi khong duoc phep loi.
    source._withdraw_validated(amount)
    try:
        destination._deposit_validated(amount)
    except Exception:
        source._deposit_validated(amount)
        raise
```

Rollback thủ công mong manh nếu bước bù cũng lỗi hoặc có nhiều luồng. Trong hệ thống thật, đặt cả hai thay đổi trong transaction của kho dữ liệu. Ví dụ giúp thấy ranh giới bất biến liên đối tượng.

Chương 18 – Contract test cho repository

```
def repository_contract(make_repository):
    repo = make_repository()
    task = Task("T1", "Hoc")

    assert repo.get("T1") is None
    repo.save(task)
    assert repo.get("T1").title == "Hoc"

    task.title = "Hoc Python"
    repo.save(task)
    assert repo.get("T1").title == "Hoc Python"

    assert repo.delete("T1") is True
    assert repo.delete("T1") is False
```

Chạy cùng contract với MemoryRepository, JsonRepository, SqliteRepository. Cần quyết định repo trả bản sao hay object đang được theo dõi; contract phải kiểm tra semantics chứ không chỉ tên phương thức.

Chương 19 – Money bất biến

```python
from dataclasses import dataclass

@dataclass(frozen=True, slots=True)
class Money:
    minor_units: int
    currency: str = "VND"

    def __post_init__(self):
        if not isinstance(self.minor_units, int):
            raise TypeError("minor_units phai la int")
        object.__setattr__(self, "currency", self.currency.upper())

    def __add__(self, other):
        if not isinstance(other, Money):
            return NotImplemented
        if self.currency != other.currency:
            raise ValueError("Khac don vi tien")
        return Money(self.minor_units + other.minor_units, self.currency)
```

`object.__setattr__` trong post-init là lối kiểm soát để chuẩn hóa frozen dataclass lúc tạo. Không dùng float cho số tiền nếu nghiệp vụ cần chính xác đơn vị nhỏ.

Chương 20 – Batch một iterable

```python
from itertools import islice

def batched(iterable, size):
    if size < 1:
        raise ValueError("size phai >= 1")
    iterator = iter(iterable)
    while True:
        batch = tuple(islice(iterator, size))
        if not batch:
            return
        yield batch
```

Hàm nhận cả generator, không cần len/chỉ số. Batch là tuple snapshot; lô cuối ngắn. Một số phiên bản Python hiện đại đã có `itertools.batched`; tự viết để hiểu giao thức.

Chương 21 – Decorator đếm lời gọi

```python
from functools import wraps
```

```

def count_calls(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        wrapper.calls += 1
        return func(*args, **kwargs)
    wrapper.calls = 0
    return wrapper

@count_calls
def square(x):
    return x * x
```

Biến đếm là trạng thái chung cho hàm và không thread-safe. Nếu dùng đồng thời, cần khóa hoặc hệ thống metrics. Test nên kiểm tra giá trị trả, metadata và số đếm cả khi hàm lỗi (hiện vẫn đếm lần thử).

Chương 22 – Transaction list bằng context manager

```
from contextlib import contextmanager
import copy

@contextmanager
def list_transaction(values):
    snapshot = copy.deepcopy(values)
    try:
        yield values
    except Exception:
        values[:] = snapshot
        raise
```

Khôi phục bằng slice giữ danh tính list ngoài. Deepcopy có giới hạn và tốn bộ nhớ; đây là mô hình học tập, không thay transaction DB. Ngoại lệ không bị nuốt.

Chương 23 – Worker queue có kết thúc

```
import asyncio

STOP = object()

async def worker(name, queue, results):
    while True:
        item = await queue.get()
        try:
```

```
            if item is STOP:
                return
            results[item] = await process_item(item)
        finally:
            queue.task_done()

async def run(items, workers=3):
    queue = asyncio.Queue(maxsize=workers * 2)
    results = {}
    tasks = [asyncio.create_task(worker(str(i), queue, results))
             for i in range(workers)]
    for item in items:
        await queue.put(item)
    for _ in tasks:
        await queue.put(STOP)
    await queue.join()
    await asyncio.gather(*tasks)
    return results
```

Nếu `process_item` lỗi, worker kết thúc và producer có thể kẹt. Thiết kế hoàn chỉnh cần TaskGroup, kênh lỗi và hủy cả nhóm. Bài tập: sửa để lỗi đầu tiên hủy an toàn.

Chương 24 – Phát hiện file trùng theo hash

```
from collections import defaultdict
from hashlib import sha256
from pathlib import Path

def file_hash(path, block_size=1024 * 1024):
    digest = sha256()
    with Path(path).open("rb") as file:
        while block := file.read(block_size):
            digest.update(block)
    return digest.hexdigest()

def duplicate_groups(folder):
    by_size = defaultdict(list)
    for path in Path(folder).rglob("*"):
        if path.is_file():
            by_size[path.stat().st_size].append(path)

    by_hash = defaultdict(list)
    for paths in by_size.values():
        if len(paths) < 2:
            continue
        for path in paths:
```

```
            by_hash[file_hash(path)].append(path)
    return [paths for paths in by_hash.values() if len(paths) > 1]
```

Nhóm kích thước trước tránh băm tệp chắc chắn khác. Công cụ chỉ báo cáo; quyết định xóa cần xem symlink, hardlink, quyền và bản sao lưu.

Chương 25 – Repository SQLite và transaction test

```
import sqlite3

def test_transaction_rolls_back():
    conn = sqlite3.connect(":memory:")
    conn.execute("CREATE TABLE account(id TEXT PRIMARY KEY, balance INTEGER)")
    conn.executemany(
        "INSERT INTO account VALUES (?, ?)",
        [("A", 100), ("B", 0)],
    )
    conn.commit()

    try:
        with conn:
            conn.execute("UPDATE account SET balance=balance-50 WHERE id='A'")
            raise RuntimeError("Simulated failure")
            conn.execute("UPDATE account SET balance=balance+50 WHERE id='B'")
    except RuntimeError:
        pass

    rows = dict(conn.execute("SELECT id, balance FROM account"))
    assert rows == {"A": 100, "B": 0}
```

Test cố ý lỗi giữa hai bước và chứng minh không có trạng thái nửa giao dịch.

Chương 26 – Parser CLI có thể test

```
import argparse

def build_parser():
    parser = argparse.ArgumentParser(prog="blanket-tasks")
    commands = parser.add_subparsers(dest="command", required=True)

    add = commands.add_parser("add")
    add.add_argument("title")
    add.add_argument("--priority", type=int, choices=(1, 2, 3), default=2)

    complete = commands.add_parser("complete")
```

```
    complete.add_argument("id")
    return parser

def test_add_parser():
    args = build_parser().parse_args(["add", "Hoc Python", "--priority", "1"])
    assert args.command == "add"
    assert args.title == "Hoc Python"
    assert args.priority == 1
```

Parser chỉ kiểm tra cú pháp CLI. Tiêu đề rỗng sau strip và quy tắc domain vẫn thuộc model/service.

Chương 27 – Kiểm tra planner không vượt sức chứa

```
def plan(tasks, capacity, score):
    if capacity < 0:
        raise ValueError("capacity khong am")
    budget = int(capacity * 0.8)
    selected = []
    used = 0
    for task in sorted(tasks, key=score, reverse=True):
        if task.estimate_minutes is None:
            continue
        if used + task.estimate_minutes <= budget:
            selected.append(task)
            used += task.estimate_minutes
    return selected

def test_plan_respects_buffer(tasks):
    selected = plan(tasks, capacity=300, score=lambda t: t.priority_score)
    assert sum(t.estimate_minutes for t in selected) <= 240
```

Greedy không đảm bảo tổ hợp điểm tối ưu giống knapsack, nhưng minh bạch và đủ cho MVP. Hãy kiểm tra thêm: không chọn task blocked, kết quả xác định khi hòa, capacity 0, estimate lớn hơn budget và mọi task không estimate.

Cách tự chấm lời giải

Với mỗi bài, chấm theo năm tiêu chí, mỗi tiêu chí 0–2 điểm:

1. **Đúng**: đúng ví dụ thường và biên đã định nghĩa.
2. **Rõ**: tên, cấu trúc và hợp đồng có thể giải thích.
3. **An toàn**: không làm mất dữ liệu, nuốt lỗi hoặc lộ bí mật vô cớ.

4. **Phù hợp**: cấu trúc và độ phức tạp đáp ứng quy mô.

5. **Có bằng chứng**: test, assertion hoặc đối chiếu độc lập.

Điểm 10 không có nghĩa lời giải duy nhất. Nó nghĩa lời giải đã nói rõ mình làm gì, không làm gì và vì sao ta có lý do để tin.

Phụ lục B

Bảng tra cứu nhanh

Phần này dùng để gợi nhớ sau khi đã học, không thay cho hiểu khái niệm. Khi không nhớ một phương thức, hãy tra; khi không biết chọn cấu trúc nào, quay lại câu hỏi về ý nghĩa dữ liệu.

Cấu trúc dữ liệu

Kiểu	Thao tác thường dùng	Ghi chú
list	append, extend, insert, pop, remove, sort, copy	có thứ tự, mutable, cho trùng; đọc chỉ số $O(1)$, tìm $O(n)$
tuple	indexing, slicing, unpacking	có thứ tự, bất biến lớp ngoài, có thể làm khóa nếu mọi phần tử hashable
set	add, discard, remove, union, intersection	duy nhất, tra cứu trung bình $O(1)$, không dựa thứ tự
dict	get, items, keys, values, pop, setdefault, update	khóa duy nhất/hashable, tra cứu trung bình $O(1)$
deque	append, appendleft, pop, popleft	queue/stack hai đầu hiệu quả
heap	heappush, heappop, nlargest, nsmallest	min-heap, push/pop $O(\log n)$

Mẫu duyệt

```
for value in values:
    ...

for index, value in enumerate(values, start=1):
    ...

for left, right in zip(a, b, strict=True):
    ...
```

```

for key, value in mapping.items():
    ...

result = [transform(x) for x in values if predicate(x)]
```

Tệp và JSON

```
from pathlib import Path
import json

path = Path("data") / "items.json"
path.parent.mkdir(parents=True, exist_ok=True)

with path.open(encoding="utf-8") as file:
    data = json.load(file)

text = json.dumps(data, ensure_ascii=False, indent=2)
temporary = path.with_suffix(path.suffix + ".tmp")
temporary.write_text(text, encoding="utf-8")
temporary.replace(path)
```

Ngoại lệ

```
try:
    value = operation()
except SpecificError as error:
    recover_or_add_context(error)
else:
    use(value)
finally:
    cleanup_if_not_using_context_manager()

raise DomainError("clear context") from error
```

Chỉ bắt lỗi xử lý được; không dùng bare `except`; không im lặng. Giữ vùng try nhỏ.

Lớp và dataclass

```
from dataclasses import dataclass, field

@dataclass
```

```
class Item:
    id: str
    name: str
    tags: list[str] = field(default_factory=list)

    def __post_init__(self):
        self.name = self.name.strip()
        if not self.name:
            raise ValueError("name is empty")
```

Đặt mutable default bằng factory. Chỉ overload dunder khi ngữ nghĩa quen thuộc. Ưu tiên hợp thành và Protocol cho cộng sự thay thế.

Thuật toán

Bài toán	Công cụ gợi ý	Điều kiện/ghi chú
Tra cứu ID nhiều lần	dict	ID ổn định và duy nhất
Loại trùng/kiểm tra có	set	không cần giữ số lần; giữ thứ tự riêng nếu cần
Tìm trong dãy sắp	binary search/bisect	cùng khóa và cùng thứ tự sắp
Lấy phần tử ưu tiên	heap	xử lý khóa hòa, update có thể push bản mới
Đường ít cạnh	BFS	đồ thị không trọng số
Đường chi phí nhỏ	Dijkstra	trọng số không âm
Phụ thuộc	topological sort	đồ thị có hướng không chu trình
Thử tổ hợp	backtracking	cắt tỉa, kích thước đầu ra có thể lũy thừa
Dữ liệu rất lớn	generator hoặc stream	dùng một lần, lỗi xuất hiện khi tiêu thụ

Kiểm thử

```
def test_behavior():
    # Arrange
    object_under_test = ...
    # Act
    result = object_under_test.do(...)
    # Assert
    assert result == expected
```

Danh sách biên: rỗng, một, trùng, âm, min/max, Unicode, thiếu tệp, hỏng dữ liệu, timeout, ngày giao tháng/năm. Một lỗi đã sửa cần regression test.

Checklist trước khi phát hành

☐ README tái tạo môi trường và chạy ứng dụng từ máy sạch.

☐ Test cốt lõi chạy thành công; không phụ thuộc dữ liệu cá nhân.

☐ Cấu hình và bí mật tách khỏi mã; không có token trong lịch sử phát hành.

☐ Thao tác ghi có backup/transaction phù hợp; lỗi không bị coi là dữ liệu rỗng.

☐ Log có mức, không chứa bí mật; mã thoát CLI có ý nghĩa.

☐ Schema/migration có phiên bản và đã thử trên bản sao.

☐ License/phụ thuộc được kiểm tra; tài liệu nói rõ giới hạn.

Phụ lục C

Từ điển thuật ngữ

API
Giao diện mà một thành phần công bố để thành phần khác sử dụng; có thể là hàm, lớp hoặc dịch vụ HTTP.

Argument / đối số
Giá trị thực tế truyền vào khi gọi hàm.

Asymptotic complexity
Cách chi phí tăng khi kích thước đầu vào tiến lớn, thường mô tả bằng Big-O.

Atomic / nguyên tử
Một nhóm thay đổi hoặc xảy ra toàn bộ, hoặc không thay đổi nào được giữ.

Backtracking / quay lui
Thử một lựa chọn, khám phá, rồi hoàn tác khi nhánh không phù hợp.

Call stack
Cấu trúc giữ các lời gọi hàm đang chờ hoàn tất, gồm biến cục bộ và vị trí quay về.

Closure
Hàm giữ quyền truy cập biến từ phạm vi bao quanh kể cả sau khi hàm ngoài kết thúc.

Composition / hợp thành
Xây một đối tượng bằng cách chứa và ủy quyền cho các đối tượng khác.

Concurrency / đồng thời
Nhiều tác vụ cùng có tiến triển trong khoảng thời gian chồng nhau.

Context manager
Đối tượng quản lý bước vào/ra một phạm vi `with`, thường để dọn tài nguyên.

Coroutine
Tác vụ có thể tạm dừng và tiếp tục, là đơn vị nền của asyncio.

CSV
Định dạng dữ liệu dạng bảng với quy tắc phân cách và quoting; không nên tự tách bằng `split(',')`.

Dataclass
Công cụ sinh code nghi thức cho lớp chủ yếu giữ dữ liệu.

Decorator
Callable nhận một hàm/lớp và trả phiên bản được bọc hoặc thay đổi.

Dependency injection
Cung cấp cộng sự từ bên ngoài thay vì để đối tượng tự tạo phụ thuộc cứng.

Dictionary
Ánh xạ khóa hashable duy nhất đến giá trị.

Dry-run
Chế độ lập/in kế hoạch nhưng không thực hiện thay đổi thật.

Dunder method
Phương thức tên hai dấu gạch dưới ở hai đầu tham gia giao thức ngôn ngữ.

Encoding
Quy tắc ánh xạ ký tự thành byte và ngược lại.

Exception
Đối tượng biểu diễn luồng thất bại bất thường có thể được phát sinh và bắt.

Generator
Iterator được tạo thuận tiện bằng hàm có `yield` hoặc biểu thức generator.

Graph / đồ thị
Tập đỉnh và cạnh mô tả quan hệ.

Hashable
Đối tượng có hash ổn định và equality phù hợp, có thể làm khóa dict/phần tử set.

Heap
Cấu trúc giữ phần tử nhỏ nhất (hoặc lớn nhất qua biến đổi) dễ lấy theo ưu tiên.

Idempotent
Chạy thao tác lặp lại cho cùng trạng thái cuối như chạy một lần.

Immutable / bất biến
Đối tượng không thay đổi trạng thái quan sát được sau khi tạo.

Invariant / bất biến điều kiện
Mệnh đề phải luôn đúng tại các ranh giới xác định của đối tượng/thuật toán.

Iterable
Đối tượng có thể tạo iterator.

Iterator
Đối tượng trả từng phần tử và nhớ vị trí giữa các lần `next`.

JSON
Định dạng cây dữ liệu gồm object, array, string, number, boolean và null.

Logging
Ghi sự kiện có cấu trúc/mức độ để vận hành và chẩn đoán.

Memoization
Lưu kết quả bài toán con theo đầu vào để tránh tính lặp.

Migration
Bước chuyển dữ liệu hoặc schema từ phiên bản cũ sang mới.

Module
Một tệp Python tạo không gian tên và có thể được import.

Mutable
Đối tượng có thể thay đổi tại chỗ trong khi giữ danh tính.

Package
Cấu trúc nhóm các module dưới một không gian tên.

Parameter / tham số
Tên biến khai báo trong định nghĩa hàm để nhận đối số.

Polymorphism / đa hình
Nhiều kiểu đáp ứng cùng giao diện để mã gọi không cần biết chi tiết cụ thể.

Protocol
Mô tả tĩnh một tập hành vi/thuộc tính mà đối tượng cần có.

Queue
Cấu trúc vào trước ra trước.

Race condition
Kết quả phụ thuộc thứ tự xen kẽ không được kiểm soát giữa các tác vụ.

Recursion / đệ quy
Hàm giải bài toán bằng cách gọi phiên bản nhỏ hơn của chính bài toán.

Regression test
Test giữ lại tình huống của lỗi cũ để ngăn lỗi quay lại.

Repository
Giao diện che cách lưu/tải đối tượng miền.

Schema
Hợp đồng về hình dạng, kiểu và ràng buộc dữ liệu.

Serialization
Chuyển đối tượng sang dạng có thể lưu/truyền và khôi phục theo quy ước.

Set Tập hợp phần tử hashable duy nhất.

Stack
Cấu trúc vào sau ra trước.

Transaction
Phạm vi thay đổi với commit khi thành công và rollback khi thất bại.

Type hint
Annotation mô tả kiểu mong đợi cho người/công cụ, thường không ép kiểm tra runtime.

Unicode
Tiêu chuẩn gán code point cho ký tự của nhiều hệ chữ; UTF-8 là một encoding Unicode.

Unit test
Test một đơn vị hành vi nhỏ, cô lập các phụ thuộc không cần thiết.

Virtual environment

Môi trường Python riêng cho interpreter/phụ thuộc của một dự án.

Phụ lục D

Tài liệu tham khảo và đường học tiếp

Nguồn chính thức

1. Python Software Foundation. *The Python Tutorial*. https://docs.python.org/3/tutorial/
2. Python Software Foundation. *Python Standard Library*. https://docs.python.org/3/library/
3. Python Software Foundation. *Python Language Reference*. https://docs.python.org/3/reference/
4. Python Enhancement Proposals. https://peps.python.org/
5. Python Packaging Authority. *Python Packaging User Guide*. https://packaging.python.org/
6. SQLite. *Official Documentation*. https://www.sqlite.org/docs.html

Sách và tài liệu nền

1. Ramalho, L. *Fluent Python*. O'Reilly Media.
2. Sweigart, A. *Automate the Boring Stuff with Python*. No Starch Press.
3. Matthes, E. *Python Crash Course*. No Starch Press.
4. Sedgewick, R., Wayne, K. *Algorithms*. Addison-Wesley.
5. Cormen, T. H. et al. *Introduction to Algorithms*. MIT Press.
6. Martin, R. C. *Clean Architecture*. Pearson.

Cách đọc tài liệu chính thức

Khi gặp một công cụ mới, bắt đầu ở tutorial để thấy bức tranh, sau đó mở library reference để xem chữ ký, ngoại lệ, độ phức tạp hoặc thay đổi phiên bản. Language reference dành cho

câu hỏi chính xác về cú pháp và data model. PEP giải thích động cơ của thay đổi, nhưng không phải mọi PEP đều đã được chấp nhận hoặc áp dụng ở phiên bản bạn chạy.

Luôn kiểm tra thanh phiên bản trên tài liệu. Một ví dụ của Python mới có thể không chạy trên môi trường cũ. Trong dự án, ghi `requires-python`, chạy test trên các phiên bản hỗ trợ và không dựa vào "máy tôi chạy được".

Ba hướng sau Tập 2

- **Khoa học dữ liệu/AI**: NumPy, pandas, trực quan hóa, xác suất, đại số tuyến tính, quy trình dữ liệu và PyTorch. Đừng bỏ qua kiểm tra dữ liệu và khả năng tái lập.
- **Backend**: HTTP sâu hơn, framework web, PostgreSQL, authentication, cache, queue, container, quan sát hệ thống và bảo mật.
- **Khoa học máy tính**: cấu trúc dữ liệu nâng cao, quy hoạch động, đồ thị, hệ điều hành, mạng, cơ sở dữ liệu và lý thuyết tính toán.

Hãy chọn một hướng theo dự án muốn xây, không theo số lượng từ khóa có thể ghi vào CV. Nền tảng của Tập 2 sẽ tiếp tục xuất hiện dưới tên mới: một DataLoader vẫn là iterator; một API vẫn là hợp đồng và ranh giới lỗi; một mô hình AI vẫn cần dữ liệu có schema, test và đo lường.

Nguồn chính thống dùng để kiểm chứng bản sâu

1. Python Software Foundation. *The Python Tutorial*, Python 3.14: https://docs.python.org/3/tutorial/
2. Python Software Foundation. *Python Data Model*: https://docs.python.org/3/reference/datamodel.html
3. Python Software Foundation. *Built-in Types*: https://docs.python.org/3/library/stdtypes.html
4. Python Software Foundation. *Errors and Exceptions*: https://docs.python.org/3/tutorial/errors.html
5. Python Software Foundation. *Input and Output*: https://docs.python.org/3/tutorial/inputoutput.html